

Major concepts for logistic regression

- Used for **binary classification**
 - Outcome: EVENT or NON-EVENT
- Requires a **DIFFERENT** Likelihood compared to regression
- Requires a transformation in order to predict the event PROBABILITY
- Transformation allows **GENERALIZING** linear modeling concepts to handle non-continuous outputs!
 - Logistic regression is a **Generalized Linear Model (GLM)**
- Regularization is an important tool to limit overfitting
- Un-regularized logistic regression cannot handle LINEARLY separated classes relative to the FEATURES

Revisit binary classification

- When we introduced predicting binary outcomes, we discussed that something in the model **had to change** compared to regression.
- A binary outcome...can take on 1 of 2 possible states:
- If the EVENT occurs: $y_n = 1$
- If the EVENT does **NOT** occur: $y_n = 0$

We cannot use a Gaussian likelihood!

- A Gaussian is well suited for a continuous response.
- Allows all continuous numbers.
- About 99.7% of the values are within 3 standard deviations around the mean.
- **A binary outcome can only be 0 or 1! Nothing else!**

Likelihood functions

- Regression problems use Gaussian likelihoods:

$$y_n \mid \mu_n, \sigma \sim \text{normal}(y_n \mid \mu_n, \sigma)$$

- Binary classification use Bernoulli likelihoods:

$$p(y_n \mid \mu_n) = \text{Bernoulli}(y_n \mid \mu_n) = \mu_n^{y_n} (1 - \mu_n)^{1-y_n}$$

Likelihood functions – and expected values

- Regression problems use Gaussian likelihoods:

$$y_n \mid \mu_n, \sigma \sim \text{normal}(y_n \mid \mu_n, \sigma)$$
$$\mu_n = f(\mathbf{x}_{n,:}, \boldsymbol{\beta})$$

- Binary classification use Bernoulli likelihoods:

$$p(y_n \mid \mu_n) = \text{Bernoulli}(y_n \mid \mu_n) = \mu_n^{y_n} (1 - \mu_n)^{1-y_n}$$
$$\mu_n = f(\mathbf{x}_{n,:}, \boldsymbol{\beta})$$

We briefly touched on this earlier in the semester when we wrote

$$\text{EVENT PROBABILITY} = f(\mathbf{x}, \boldsymbol{\beta})$$

- The parameters, $\boldsymbol{\beta}$, are written explicitly within the relationship above.
- The LOSS FUNCTION considers the LIKELIHOOD of the observed response, $y = 1$ or $y = 0$, GIVEN the modeled EVENT PROBABILITY.
- The parameters are learned by MINIMIZING the LOSS.

Binary classification cannot use the same formulas/expressions for the mean as regression

- If we have a single input, x , we CANNOT use a simple linear relationship for the probability of the event.

$$\mu_n = \beta_0 + \beta_1 x_n$$

What's special about a probability?

Binary classification cannot use the same formulas/expressions for the mean as regression

- If we have a single input, x , we CANNOT use a simple linear relationship for the probability of the event.

$$\mu_n = \beta_0 + \beta_1 x_n$$

What's special about a probability?

Probability is bounded between 0 and 1!!

The simple linear relationship does NOT respect the bounds on the probability

- To overcome this limitation, we will NOT directly model μ .
- Instead, we will **TRANSFORM** from a bounded variable to an **UNBOUNDED variable**.
- We will call the unbounded or transformed variable η .

There are multiple transformations we could use, but a popular one is the **logit transformation**

- Logit or **LOG-ODDS** transformation:

$$\eta = \text{logit}(\mu) = \log \left[\frac{\mu}{1 - \mu} \right]$$

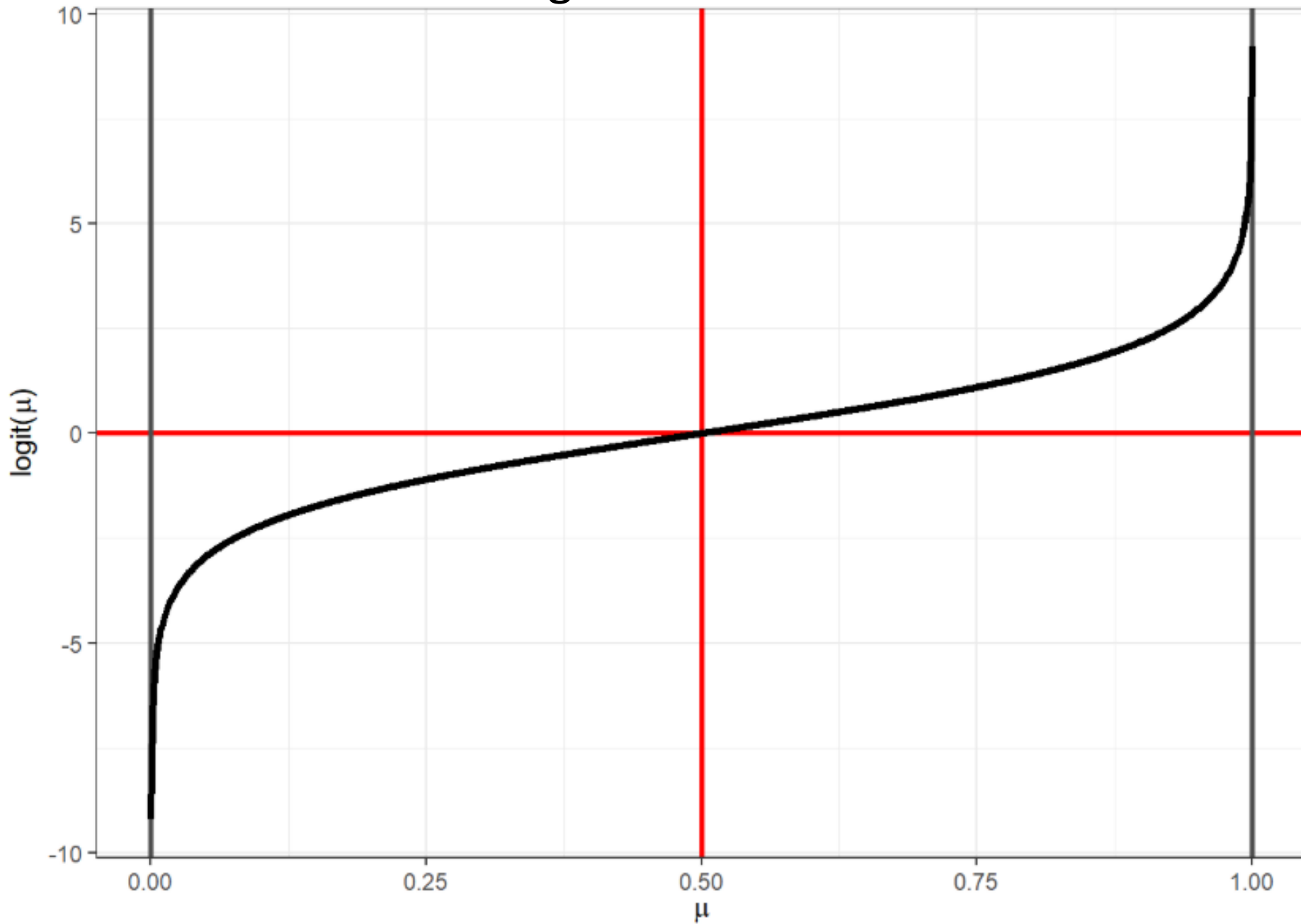
There are multiple transformations we could use, but a popular one is the **logit transformation**

- Logit or **LOG-ODDS** transformation:

$$\eta = \text{logit}(\mu) = \log \left[\frac{\mu}{1 - \mu} \right]$$

Odds ratio, the probability divided
by 1 minus the probability

Logit transformation



By modeling the new unbounded variable, the logit function serves as the link to the **linear predictor**

- The **linear predictor** is just the linear model we are familiar with:

$$\eta_n = \beta_0 + \beta_1 x_n$$

- We are generalizing the linear model!

To recover the probability we need to invert the link function

- The inverse logit is called the **logistic function**:

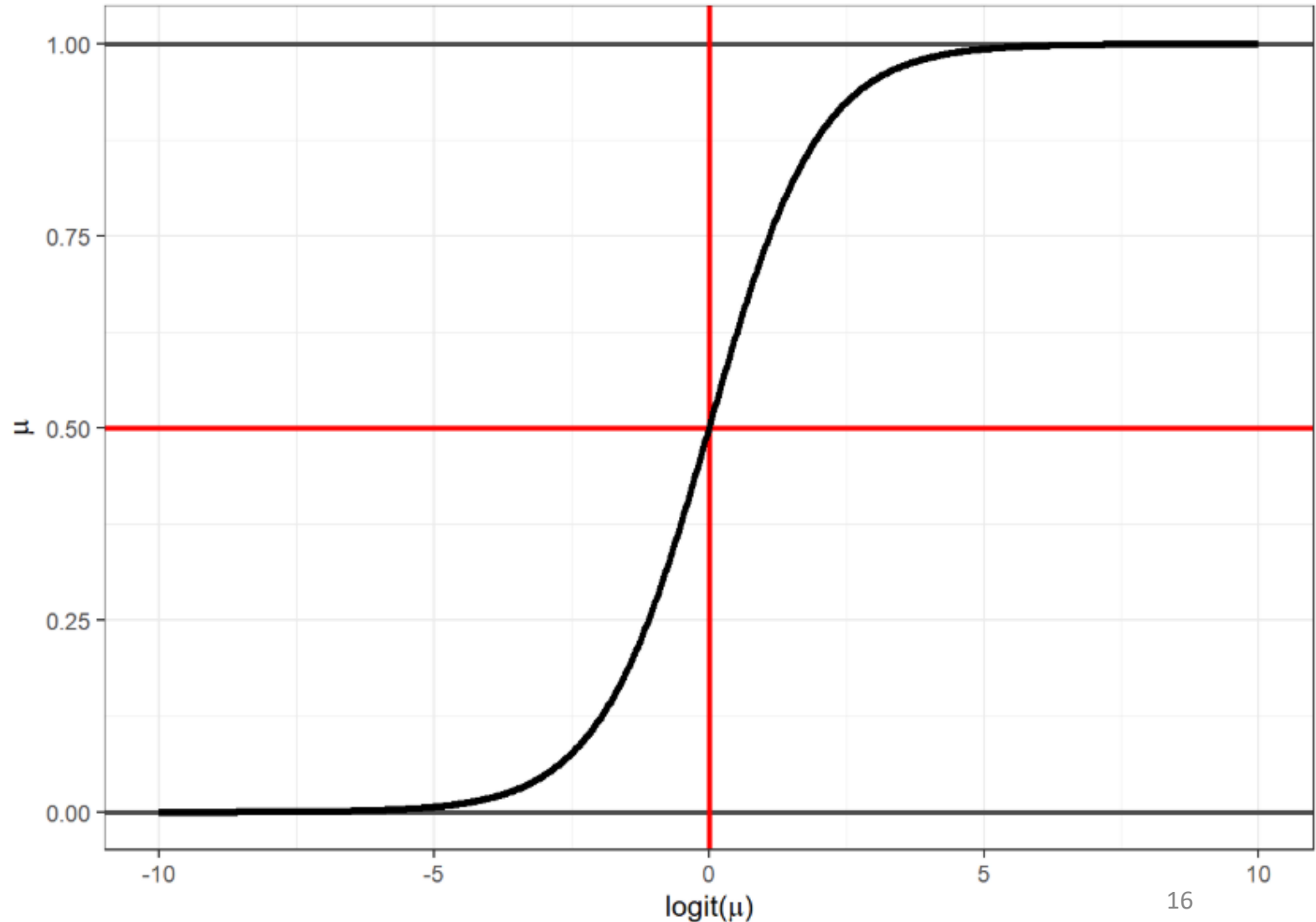
$$\mu = \text{logit}^{-1}[\eta] = \frac{1}{1 + \exp(-\eta)} = \frac{\exp(\eta)}{1 + \exp(\eta)}$$

I use the R function `boot::inv.logit()` to calculate the logistic function

$$\mu = \text{logit}^{-1}[\eta] = \frac{1}{1 + \exp(-\eta)} = \frac{\exp(\eta)}{1 + \exp(\eta)}$$

The logistic function is an **S-curve** or sigmoid function.

It “*squashes*” the unbounded variable to be constrained between 0 and 1.



The n -th Likelihood for the Generalized Linear Model (GLM):

$$y_n | \mu_n \sim \text{Bernoulli}(y_n | \mu_n)$$

$$\mu_n = \text{logit}^{-1}[\eta_n]$$

$$\eta_n = \beta_0 + \beta_1 x_n$$

OR:

$$y_n | \beta_0, \beta_1, x_n \sim \text{Bernoulli}(y_n | \text{logit}^{-1}[\beta_0 + \beta_1 x_n])$$

Because the inverse link is the logistic function, this GLM is referred to as LOGISTIC REGRESSION:

$$y_n | \mu_n \sim \text{Bernoulli}(y_n | \mu_n)$$

$$\mu_n = \text{logit}^{-1}[\eta_n]$$

$$\eta_n = \beta_0 + \beta_1 x_n$$

OR:

$$y_n | \beta_0, \beta_1, x_n \sim \text{Bernoulli}(y_n | \text{logit}^{-1}[\beta_0 + \beta_1 x_n])$$

The complete model is then the product of the N individual likelihoods:

$$\mathbf{y}|\beta_0, \beta_1, \mathbf{x} \sim \prod_{n=1}^N (\text{Bernoulli}(y_n | \text{logit}^{-1}[\beta_0 + \beta_1 x_n]))$$

Why can we use the PRODUCT of the N likelihoods?

What are we assuming?

We generalized our linear modeling toolkit

- Everything we talked about in terms of matrix math and basis functions also apply!

Consider D inputs stored in a design matrix $\mathbf{X}$

- The linear predictor evaluated at all N observations is just:

$$\boldsymbol{\eta} = \mathbf{X}\boldsymbol{\beta}$$

- The linear predictor at the n -th observation:

$$\eta_n = \mathbf{x}_{n,:}\boldsymbol{\beta}$$

Consider D inputs stored in a design matrix $\mathbf{X}$

- The linear predictor evaluated at all N observations is just:

$$\boldsymbol{\eta} = \mathbf{X}\boldsymbol{\beta}$$

REMEMBER: First column in $\mathbf{X}$ is the “intercept” column of 1’s

- The linear predictor at the n -th observation:

$$\eta_n = \mathbf{x}_{n,:}\boldsymbol{\beta}$$

Consider a J -th order basis stored in a basis design matrix Φ

- The linear predictor evaluated at all N observations is just:

$$\boldsymbol{\eta} = \Phi \boldsymbol{\beta}$$

- The n -th linear predictor is:

$$\eta_n = \phi_{n,:} \boldsymbol{\beta}$$

Consider a J -th order basis stored in a basis design matrix Φ

- The linear predictor evaluated at all N observations is just:

$$\eta = \Phi\beta$$

REMEMBER: First column in Φ is the “intercept” column of 1’s

- The n -th linear predictor is:

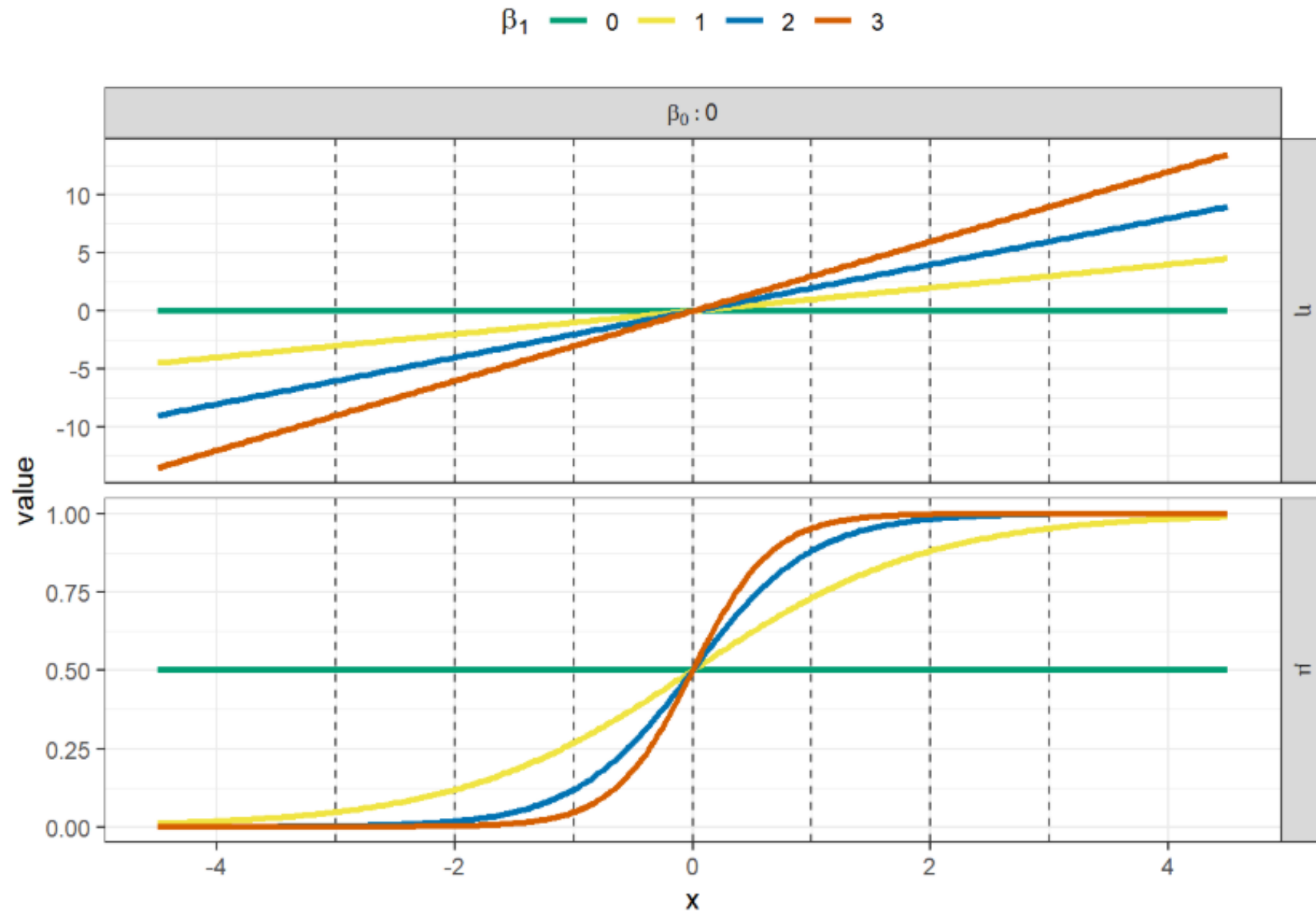
$$\eta_n = \phi_{n,:}\beta$$

Let's now look at trends of the linear predictor for the single input setting

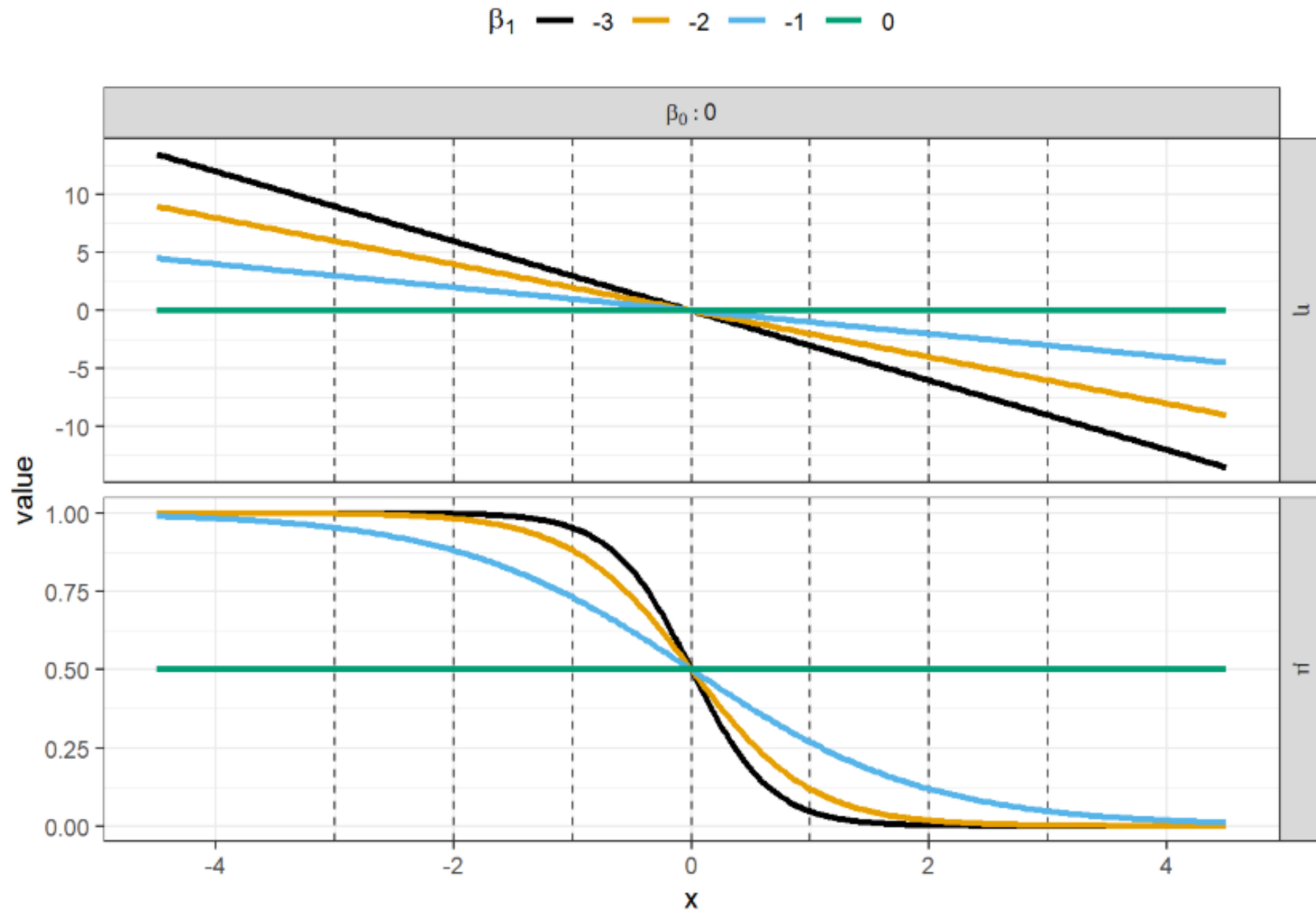
- Everything can apply to more complicated relationships...but let's keep things simple for now.
- We will continue to use a single input with a linear relationship to the linear predictor:

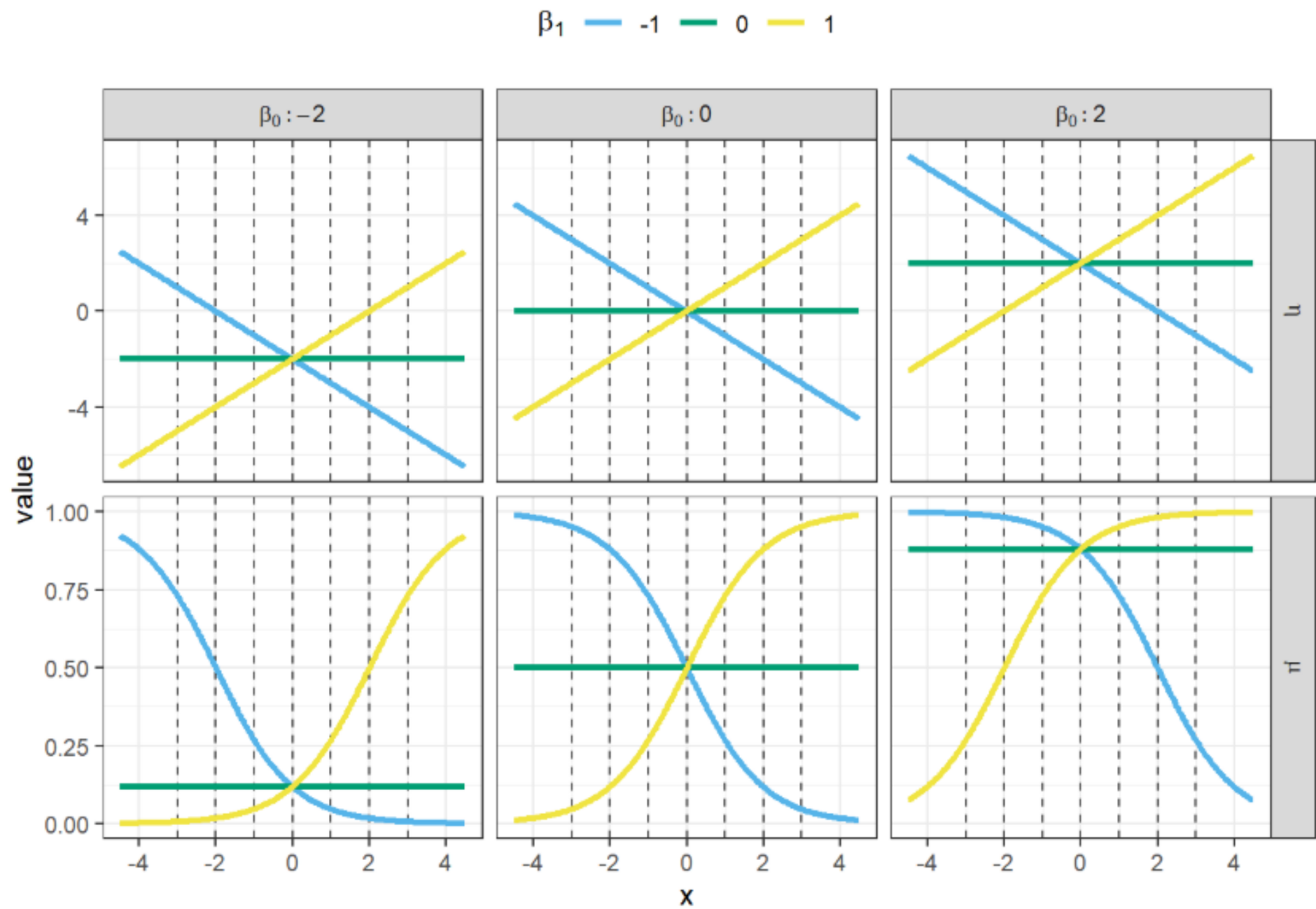
$$\eta_n = \beta_0 + \beta_1 x_n$$

Trends of the linear predictor and probability of the event



Trends of the linear predictor and probability of the event





With the probability of the event known, we can generate random binary outcomes

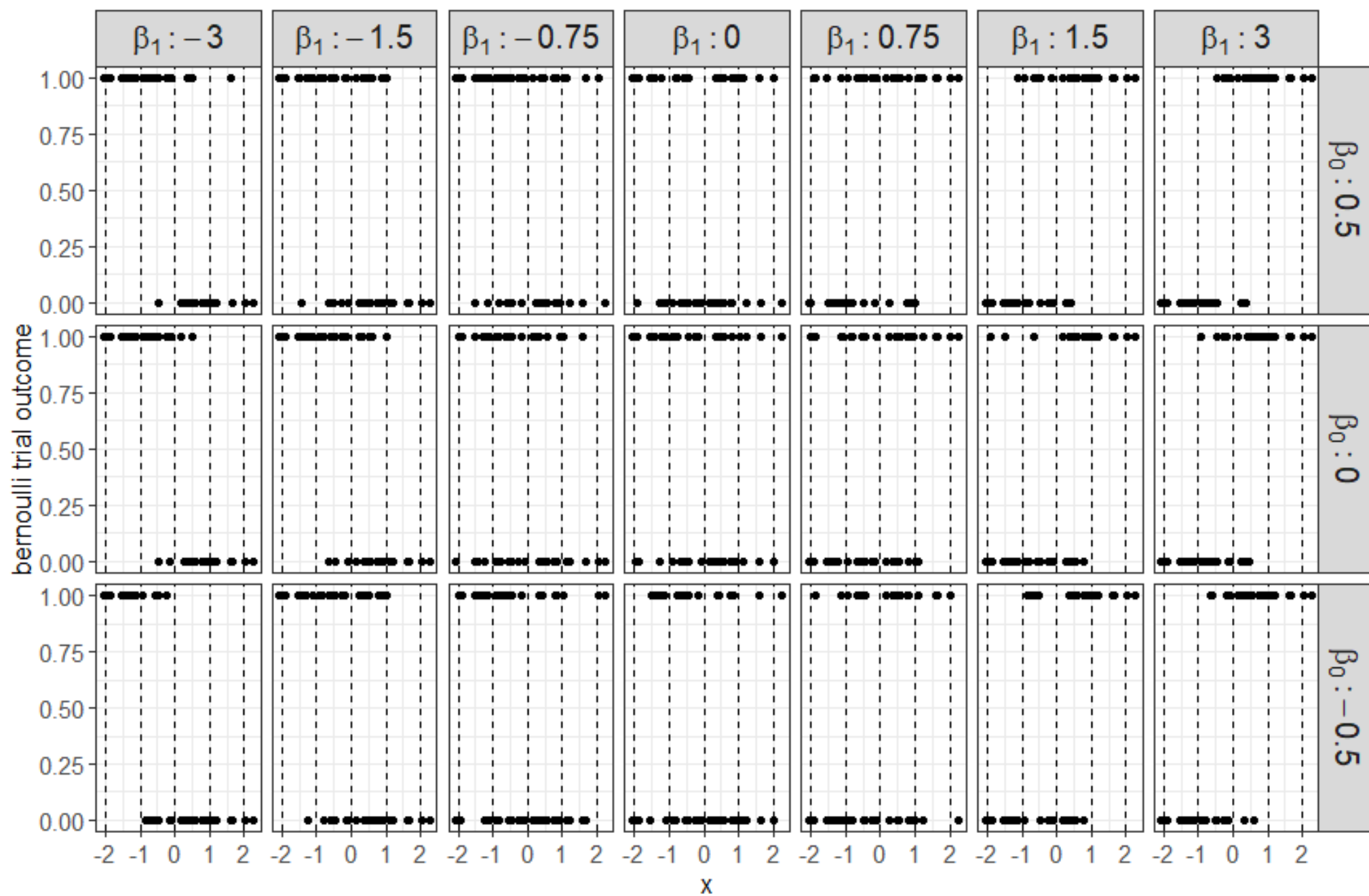
- **The likelihood is a Bernoulli:**

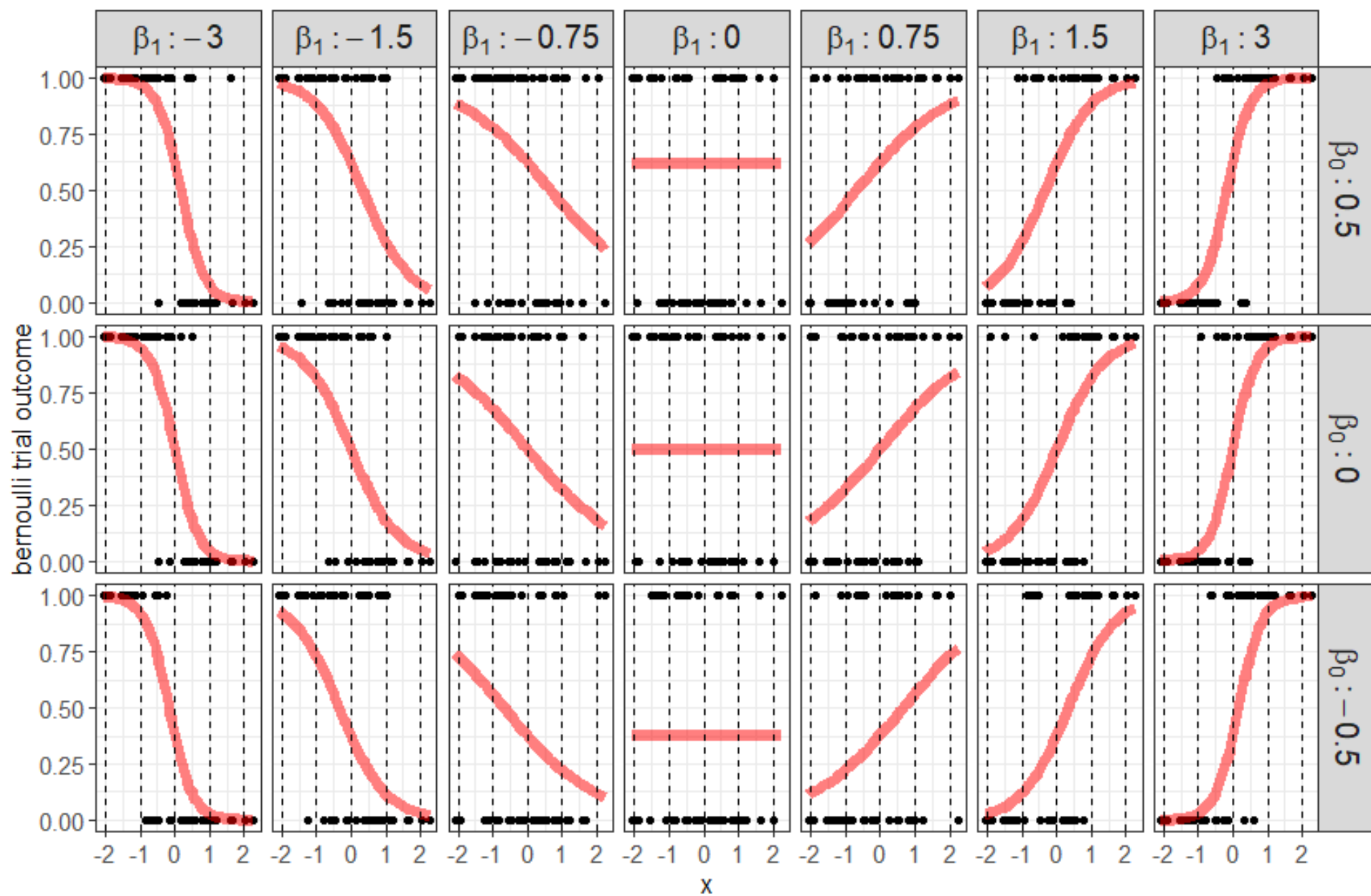
$$p(y_n \mid \mathbf{x}_{n,:}, \boldsymbol{\beta}) = \text{Bernoulli}(y_n \mid \text{logit}^{-1}(\beta_0 + \beta_1 x_n))$$

- In R, if we have calculated the probability of the event and assigned the result to the vector `mu`, a random Bernoulli trial is generated by calling the `rbinom()` function:

```
y <- rbinom(n = length(mu), size = 1, prob = mu)
```

The vector `y` will contain 0's and 1's with length equal to the length of `mu`





Before going through the math of fitting a logistic regression model...

- Let's create a simple synthetic data problem.
- We will specify a true relationship, generate data, then try and fit a model to the data.

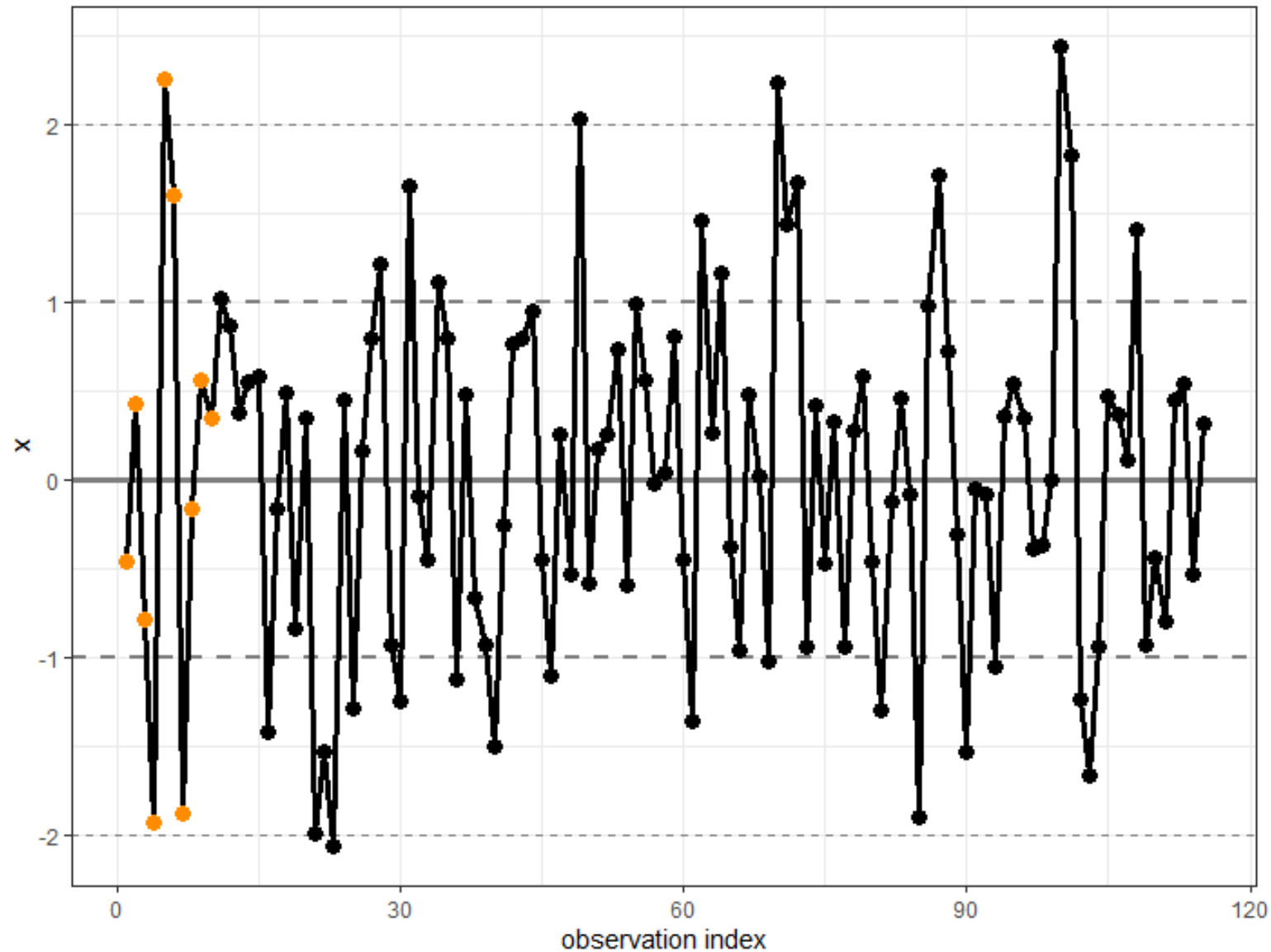
Synthetic data

```
### define the true parameter values
beta0_true <- -0.1
beta1_true <- 1.35

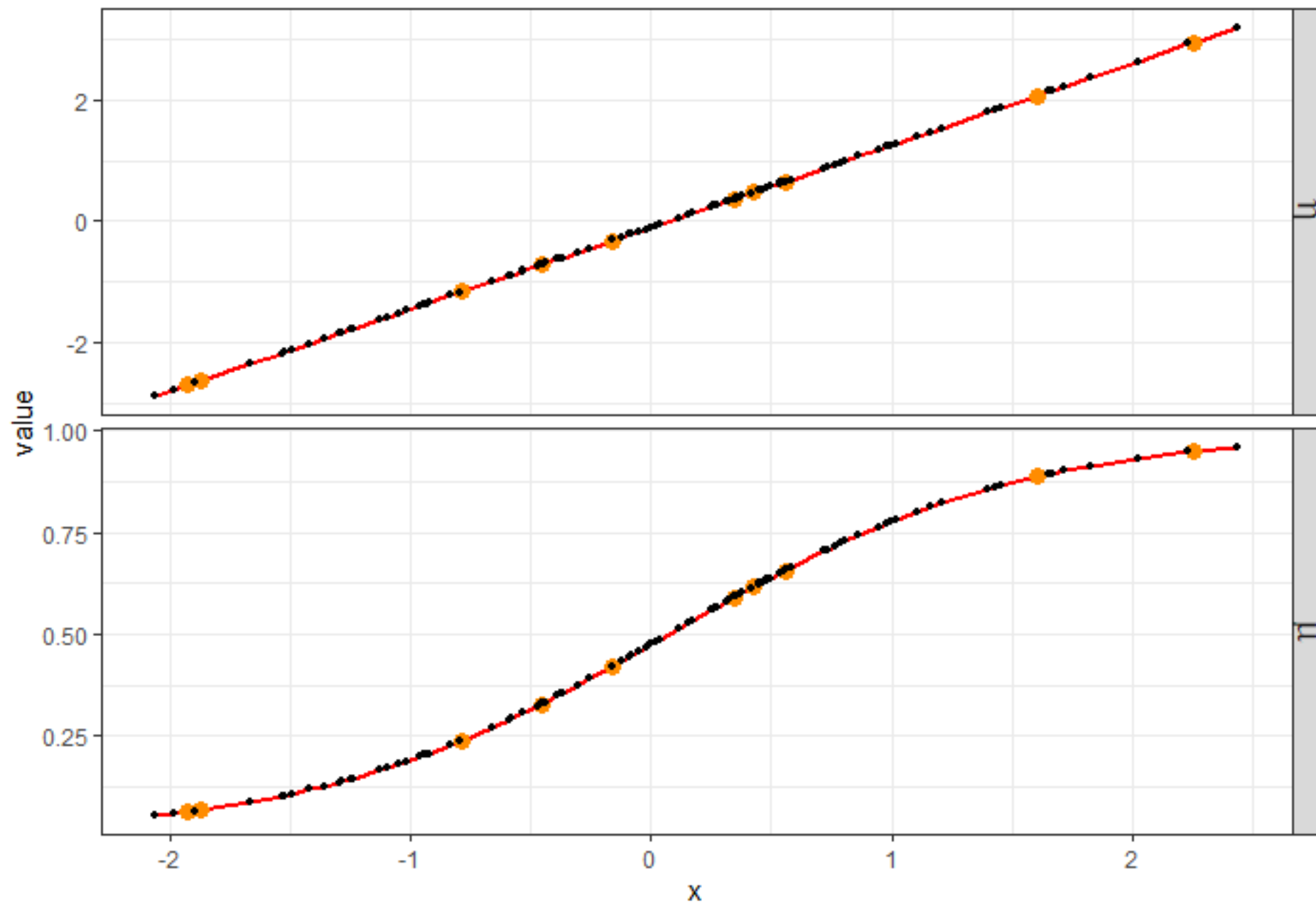
### generate 115 random input observations, evaluate the
### linear predictor, calculate the probability of the event
### and generate the random bernoulli outcomes
set.seed(412)

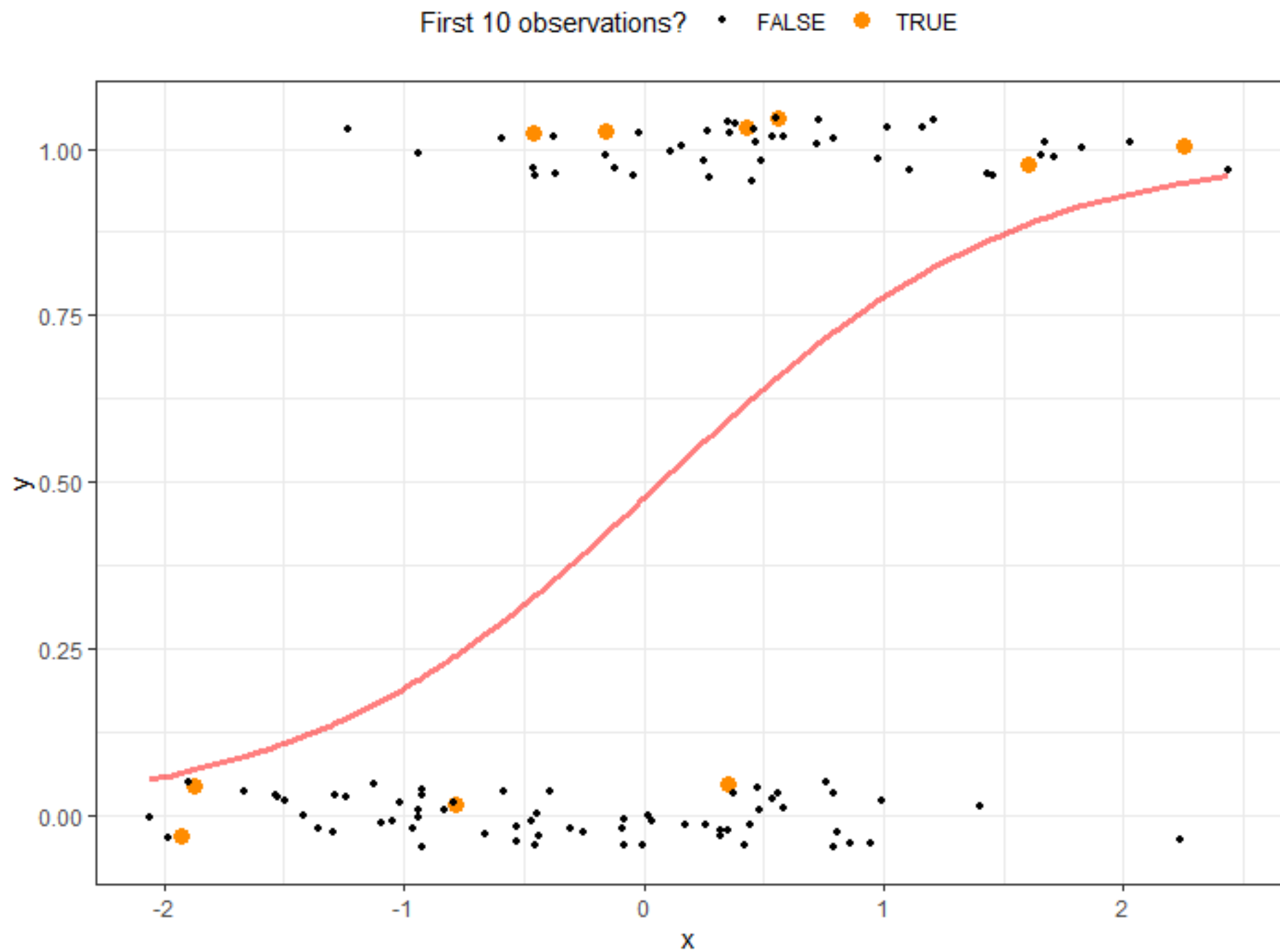
demo_df <- tibble::tibble(
  x = rnorm(n = 115)
) %>%
  mutate(eta = beta0_true + beta1_true * x,
         mu = boot::inv.logit(eta),
         y = rbinom(n = n(), size = 1, prob = mu))
```

Random input run chart



First 10 observations? • FALSE • TRUE





This data set was shown to you previously when we introduced binary classification

- At the beginning of the semester, we fit a logistic regression model with the `glm()` in R.
- But now...let's understand how fitting works with logistic regression!

This data set was shown to you previously when we introduced binary classification

- At the beginning of the semester, we fit a logistic regression model with the `glm()` in R.
- We will take a Bayesian viewpoint, compared to `glm()` which is a classic Maximum Likelihood Estimate (MLE) approach.

Before going through the formal derivation, let's visualize the learning problem.

- We need to learn the unknowns of our model given the observations.
- How many unknowns are there?

Likelihood is a Bernoulli distribution

$$y_n | \mu_n \sim \text{Bernoulli}(y_n | \mu_n)$$

EQUIVALENT TO

$$y_n | \text{size} = 1, \mu_n \sim \text{Binomial}(y_n | \text{size} = 1, \mu_n)$$

Likelihood is a Bernoulli distribution

$$y_n | \mu_n \sim \text{Bernoulli}(y_n | \mu_n)$$

EQUIVALENT TO

$$y_n | \text{size} = 1, \mu_n \sim \text{Binomial}(y_n | \text{size} = 1, \mu_n)$$

What about σ ????

Likelihood is a Bernoulli distribution

$$y_n | \mu_n \sim \text{Bernoulli}(y_n | \mu_n)$$

EQUIVALENT TO

$$y_n | \text{size} = 1, \mu_n \sim \text{Binomial}(y_n | \text{size} = 1, \mu_n)$$

The Binomial distribution does NOT have σ !!!!

With logistic regression, the only parameters we need to learn are the β -parameters!

- In general the joint posterior distribution is:

$$p(\boldsymbol{\beta} \mid \mathbf{y}, \mathbf{X}) \propto p(\mathbf{y} \mid \boldsymbol{\beta}, \mathbf{X})p(\boldsymbol{\beta})$$

- Specifically, for our single input, linear relationship example, the joint posterior is:

$$p(\beta_0, \beta_1 \mid \mathbf{y}, \mathbf{x}) \propto p(\mathbf{y} \mid \beta_0, \beta_1, \mathbf{x})p(\beta_0, \beta_1)$$

Let's use independent Gaussian priors on the unknown $\boldsymbol{\beta}$ -parameters

- Typically shared or common prior means and standard deviations are used:

$$p(\boldsymbol{\beta}) = \prod_{d=0}^D \left(\text{normal}(\beta_d \mid \mu_{\beta}, \tau_{\beta}) \right)$$

- Usually the shared prior mean is set to zero, $\mu_{\beta} = 0$

The complete probability model assuming standard normal priors

$$y_n | \mu_n \sim \text{Bernoulli}(y_n | \mu_n)$$

$$\mu_n = \text{logit}^{-1}(\eta_n)$$

$$\eta_n = \beta_0 + \beta_1 x_n = \mathbf{x}_{n,:} \boldsymbol{\beta}$$

$$\beta_0 \sim \text{normal}(\beta_0 \mid 0, 1)$$

$$\beta_1 \sim \text{normal}(\beta_1 \mid 0, 1)$$

Since there are only 2 unknowns to our problem, we can easily visualize the log-posterior surface

- This is an important step when moving from standard linear models to logistic regression.
- In linear models, the log-posterior surface was UNIMODAL.
- Will that still be the case with logistic regression?

```

### define the log-posterior function
logistic_logpost <- function(unknowns, my_info)
{
  # unpack the parameter vector
  beta_0 <- unknowns[1]
  beta_1 <- unknowns[2]

  # calculate linear predictor
  eta <- beta_0 + beta_1 * my_info$xobs

  # calculate the event probability
  mu <- boot::inv.logit(eta)

  # evaluate the log-likelihood
  log_lik <- sum(dbinom(x = my_info$yobs,
                        size = 1,
                        prob = mu,
                        log = TRUE))

  # evaluate the log-prior
  log_prior <- sum(dnorm(x = c(beta_0, beta_1),
                           mean = my_info$mu_beta,
                           sd = my_info$tau_beta,
                           log = TRUE))

  # sum together
  log_lik + log_prior
}

```

```

#### define the log-posterior function
logistic_logpost <- function(unknowns, my_info)
{
  # unpack the parameter vector
  beta_0 <- unknowns[1]
  beta_1 <- unknowns[2]

  # calculate linear predictor
  eta <- beta_0 + beta_1 * my_info$xobs

  # calculate the event probability
  mu <- boot::inv.logit(eta)

  # evaluate the log-likelihood
  log_lik <- sum(dbinom(x = my_info$yobs,
                        size = 1,
                        prob = mu,
                        log = TRUE))

  # evaluate the log-prior
  log_prior <- sum(dnorm(x = c(beta_0, beta_1),
                           mean = my_info$mu_beta,
                           sd = my_info$tau_beta,
                           log = TRUE))

  # sum together
  log_lik + log_prior
}

```

Unpack unknowns into the intercept and slope

```

#### define the log-posterior function
logistic_logpost <- function(unknowns, my_info)
{
  # unpack the parameter vector
  beta_0 <- unknowns[1]
  beta_1 <- unknowns[2]

  # calculate linear predictor
  eta <- beta_0 + beta_1 * my_info$xobs   Calculate linear predictor

  # calculate the event probability
  mu <- boot::inv.logit(eta)

  # evaluate the log-likelihood
  log_lik <- sum(dbinom(x = my_info$yobs,
                        size = 1,
                        prob = mu,
                        log = TRUE))

  # evaluate the log-prior
  log_prior <- sum(dnorm(x = c(beta_0, beta_1),
                          mean = my_info$mu_beta,
                          sd = my_info$tau_beta,
                          log = TRUE))

  # sum together
  log_lik + log_prior
}

```

```

#### define the log-posterior function
logistic_logpost <- function(unknowns, my_info)
{
  # unpack the parameter vector
  beta_0 <- unknowns[1]
  beta_1 <- unknowns[2]

  # calculate linear predictor
  eta <- beta_0 + beta_1 * my_info$xobs

  # calculate the event probability
  mu <- boot::inv.logit(eta)

  # evaluate the log-likelihood
  log_lik <- sum(dbinom(x = my_info$yobs,
                        size = 1,
                        prob = mu,
                        log = TRUE))

  # evaluate the log-prior
  log_prior <- sum(dnorm(x = c(beta_0, beta_1),
                           mean = my_info$mu_beta,
                           sd = my_info$tau_beta,
                           log = TRUE))

  # sum together
  log_lik + log_prior
}

```

NEW!!!! Calculate the probability via the inverse link function

```

#### define the log-posterior function
logistic_logpost <- function(unknowns, my_info)
{
  # unpack the parameter vector
  beta_0 <- unknowns[1]
  beta_1 <- unknowns[2]

  # calculate linear predictor
  eta <- beta_0 + beta_1 * my_info$xobs

  # calculate the event probability
  mu <- boot::inv.logit(eta)

  # evaluate the log-likelihood
  log_lik <- sum(dbinom(x = my_info$yobs,
                        size = 1,
                        prob = mu,
                        log = TRUE))

  # evaluate the log-prior
  log_prior <- sum(dnorm(x = c(beta_0, beta_1),
                           mean = my_info$mu_beta,
                           sd = my_info$tau_beta,
                           log = TRUE))

  # sum together
  log_lik + log_prior
}

```

Log-likelihood...what's different compared to the linear model?

```

#### define the log-posterior function
logistic_logpost <- function(unknowns, my_info)
{
  # unpack the parameter vector
  beta_0 <- unknowns[1]
  beta_1 <- unknowns[2]

  # calculate linear predictor
  eta <- beta_0 + beta_1 * my_info$xobs

  # calculate the event probability
  mu <- boot::inv.logit(eta)

  # evaluate the log-likelihood
  log_lik <- sum(dbinom(x = my_info$yobs,
                        size = 1,
                        prob = mu,
                        log = TRUE))

  # evaluate the log-prior
  log_prior <- sum(dnorm(x = c(beta_0, beta_1),
                          mean = my_info$mu_beta,
                          sd = my_info$tau_beta,
                          log = TRUE))

  # sum together
  log_lik + log_prior
}

```

Log-prior...is there a difference with the linear model?

```

#### define the log-posterior function
logistic_logpost <- function(unknowns, my_info)
{
  # unpack the parameter vector
  beta_0 <- unknowns[1]
  beta_1 <- unknowns[2]

  # calculate linear predictor
  eta <- beta_0 + beta_1 * my_info$xobs

  # calculate the event probability
  mu <- boot::inv.logit(eta)

  # evaluate the log-likelihood
  log_lik <- sum(dbinom(x = my_info$yobs,
                        size = 1,
                        prob = mu,
                        log = TRUE))

  # evaluate the log-prior
  log_prior <- sum(dnorm(x = c(beta_0, beta_1),
                           mean = my_info$mu_beta,
                           sd = my_info$tau_beta,
                           log = TRUE))

  # sum together
  log_lik + log_prior
}

```

WHAT ABOUT THE DERIVATIVE ADJUSTMENT???

We do NOT need to worry about a derivative adjustment!!!!

- Even though we are applying a non-linear transformation, we are NOT transforming an unknown!

$$\mu_n = \text{logit}^{-1}(\eta_n)$$

- The transformation is applied to a calculated or derived quantity.

$$\eta_n = \beta_0 + \beta_1 x_n$$

- If we know the β -parameters and x , we can calculate η .

We do NOT need to worry about a derivative adjustment!!!!

- Even though we are applying a non-linear transformation, we are NOT transforming an unknown!

$$\mu_n = \text{logit}^{-1}(\eta_n)$$

- The transformation is applied to a calculated or derived quantity.

$$\eta_n = \beta_0 + \beta_1 x_n$$

- If we know the β -parameters and x , we can calculate η .

The event probability is DETERMINISTICALLY known if we know the input and the β 's!

We do NOT need to worry about a derivative adjustment!!!!

- Even though we are applying a non-linear transformation, we are NOT transforming an unknown!

$$\mu_n = \text{logit}^{-1}(\eta_n)$$

- The transformation is applied to a calculated or derived quantity.

$$\eta_n = \beta_0 + \beta_1 x_n$$

- If we know the β -parameters and x , we can calculate η .

We are NOT learning the event probability directly!
Instead we are learning the linear predictor coefficients and those coefficients determine the probability!

In a Bayesian setting, if we apply a prior to a parameter then we need to consider the derivative adjustment when a transformation is used!

$$y_n | \mu_n \sim \text{Bernoulli}(y_n | \mu_n)$$

$$\mu_n = \text{logit}^{-1}(\eta_n)$$

$$\eta_n = \beta_0 + \beta_1 x_n = \mathbf{x}_{n,:} \boldsymbol{\beta}$$

Transformation applied
to μ but a prior is NOT
assigned to μ !

$$\beta_0 \sim \text{normal}(\beta_0 \mid 0, 1)$$

$$\beta_1 \sim \text{normal}(\beta_1 \mid 0, 1)$$

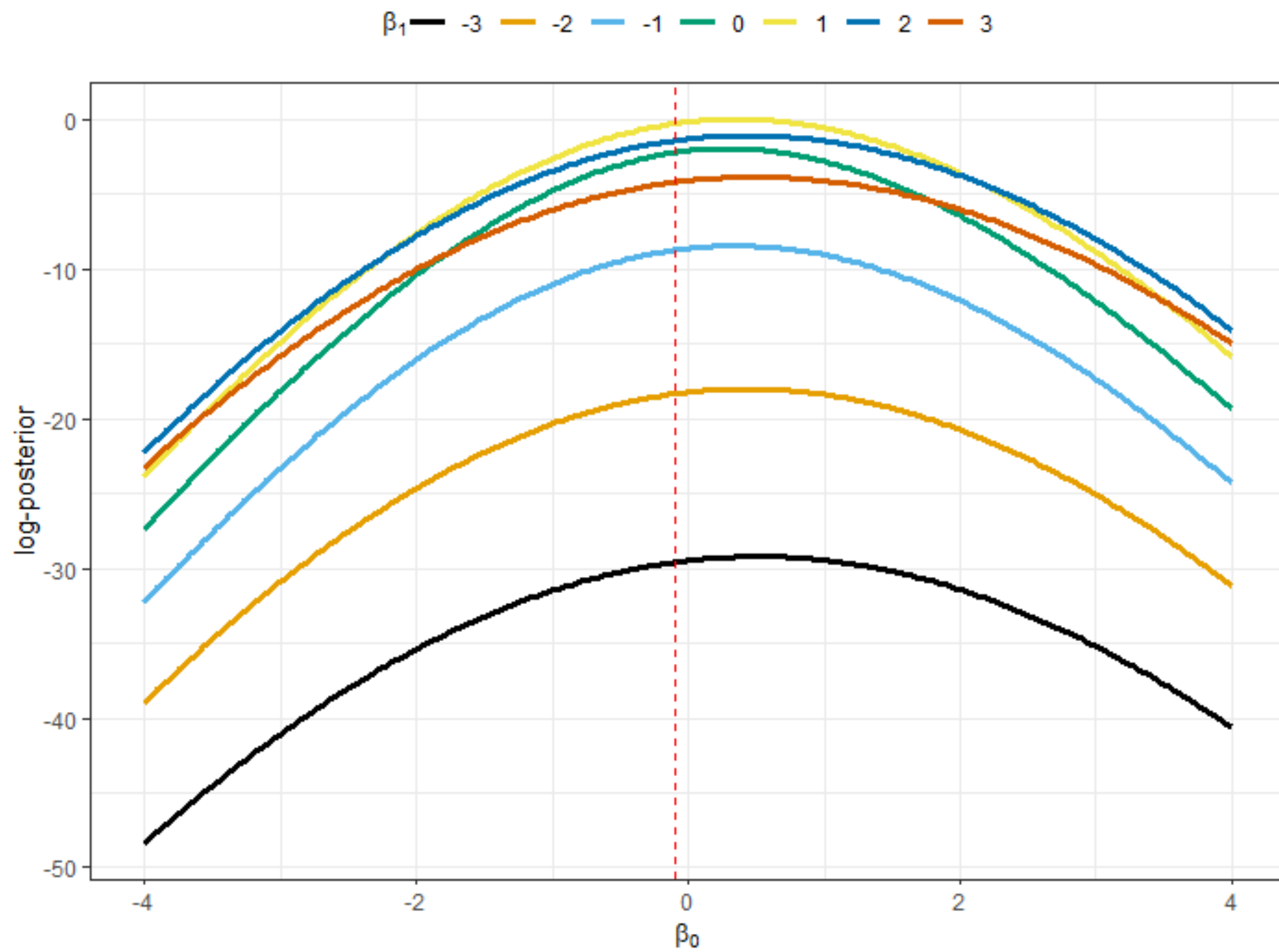
Specify the list of required information,
focusing just on the first 10 observations

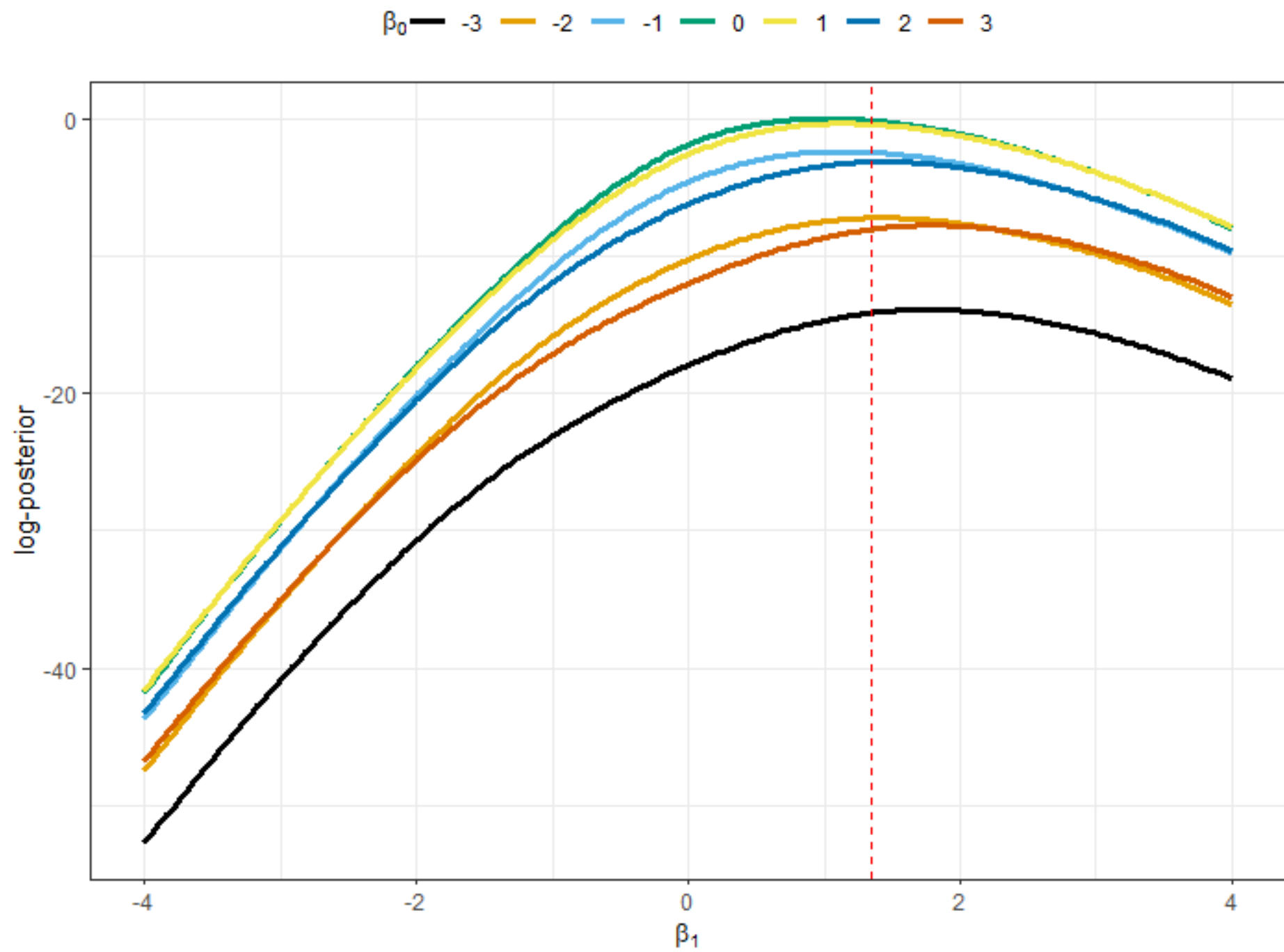
```
### create the training set
train_df <- demo_df %>%
  slice(1:10)

### define the information list
info_use <- list(
  xobs = train_df$x,
  yobs = train_df$y,
  mu_beta = 0,
  tau_beta = 1
)
```

Let's first visualize univariate trends before we consider the complete surface

- We will look at the log-posterior with respect to β_0 at a few specific β_1 values.
- Then we will look at the log-posterior with respect to β_1 at a few specific β_0 values.

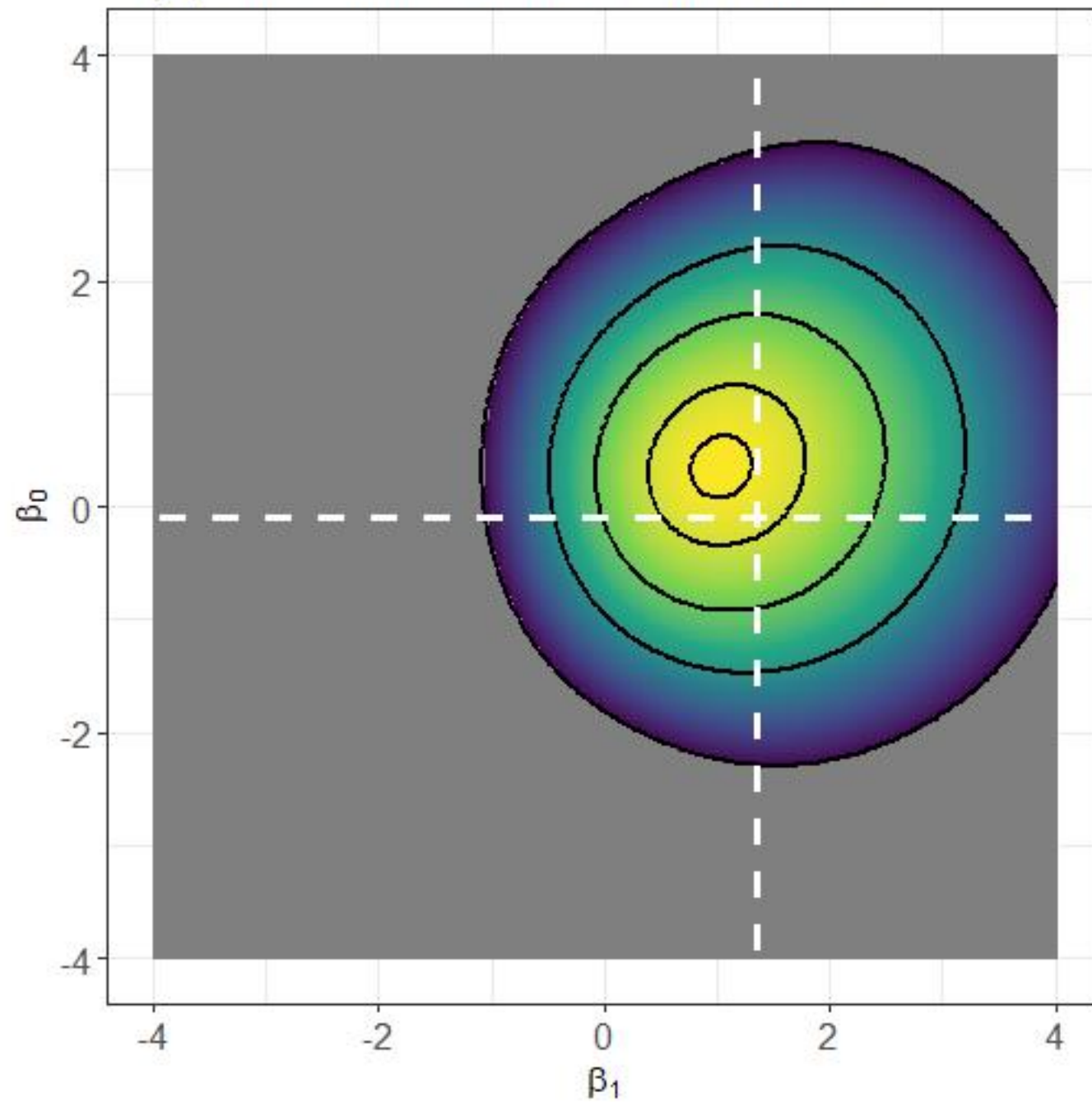




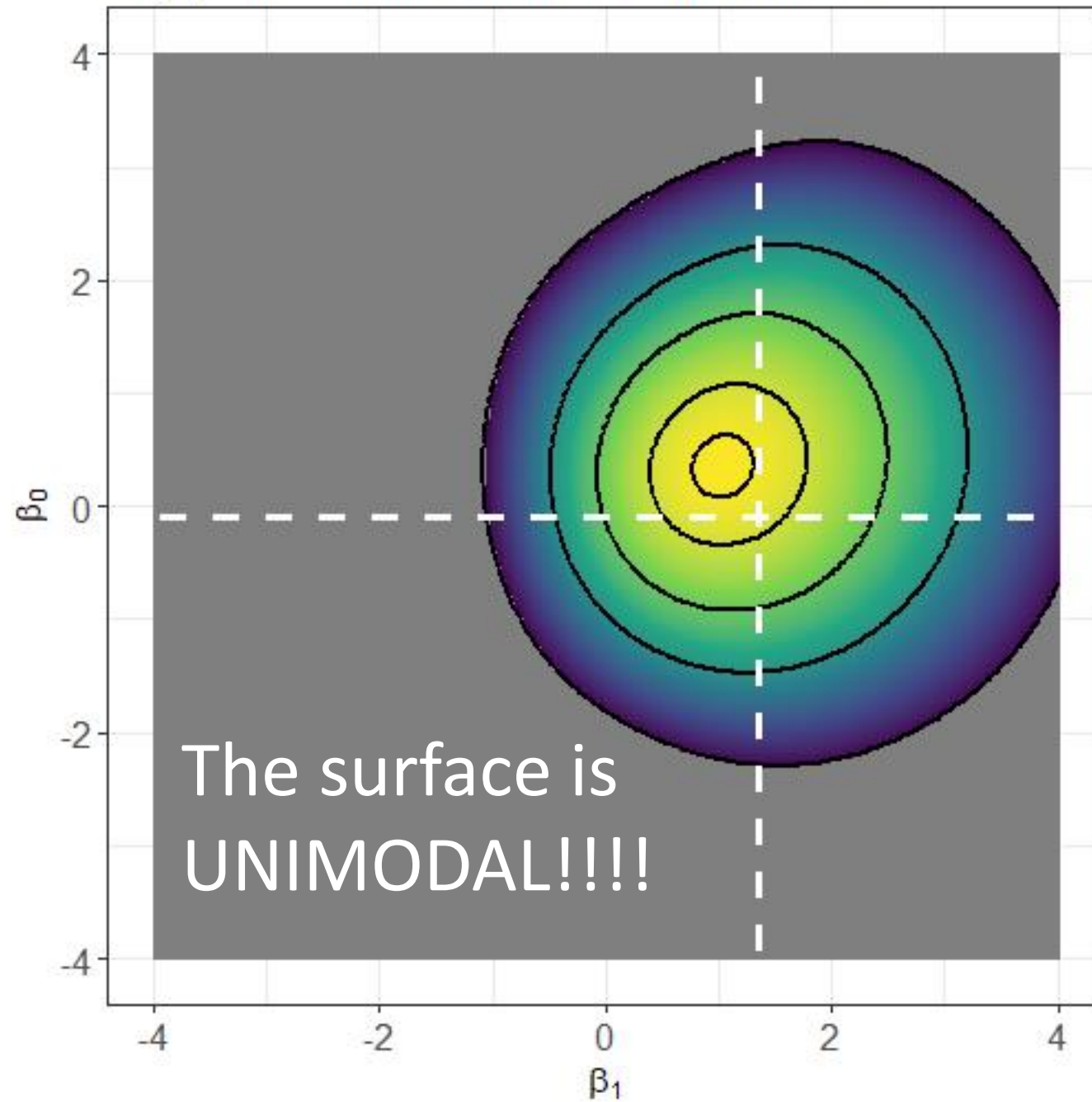
The log-posterior seems to be unimodal...

- Let's look at the complete surface to see if that holds over MANY combinations of the unknown intercept and slope.

Log-posterior based on $N = 10$ observations

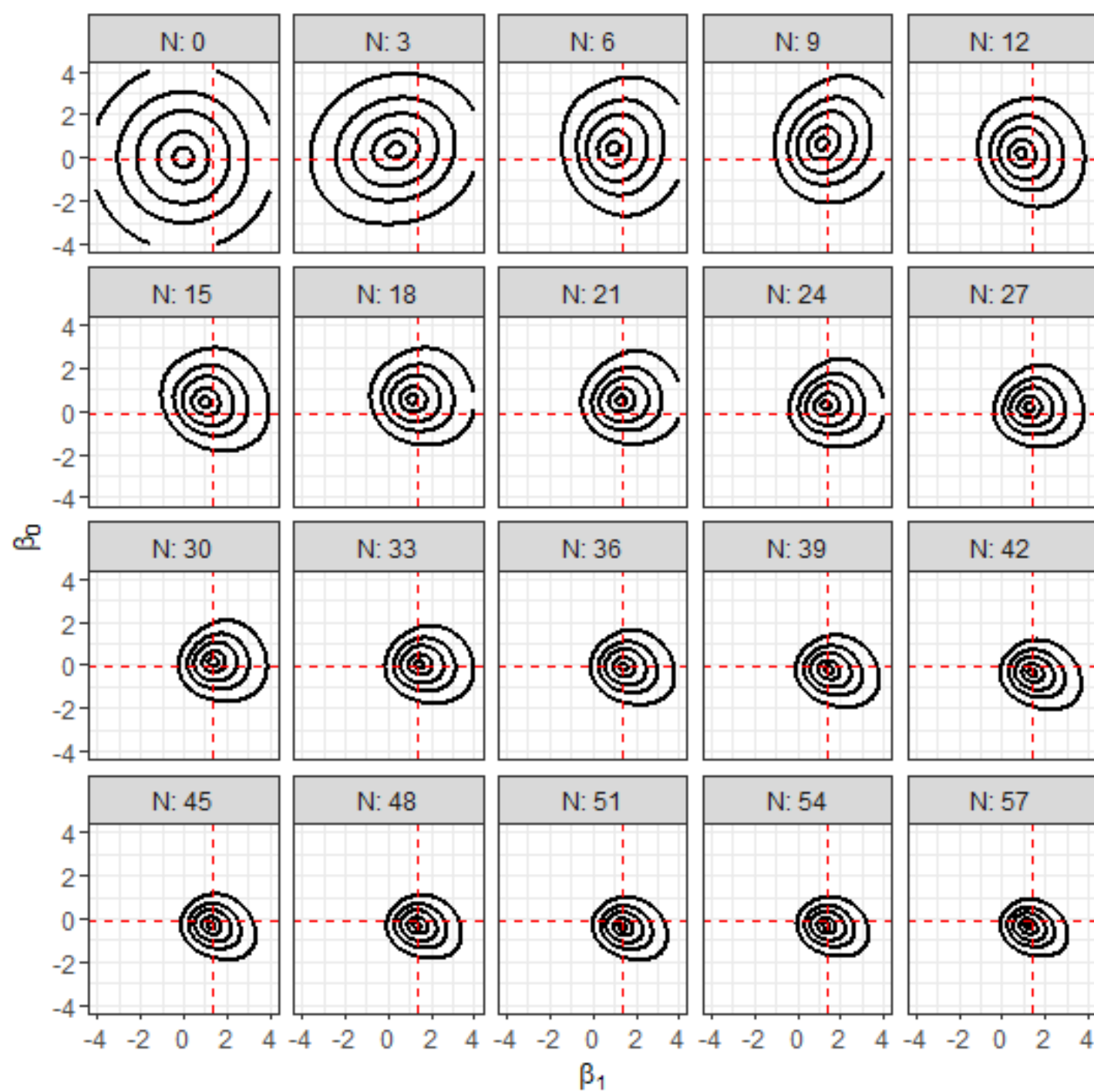


Log-posterior based on $N = 10$ observations

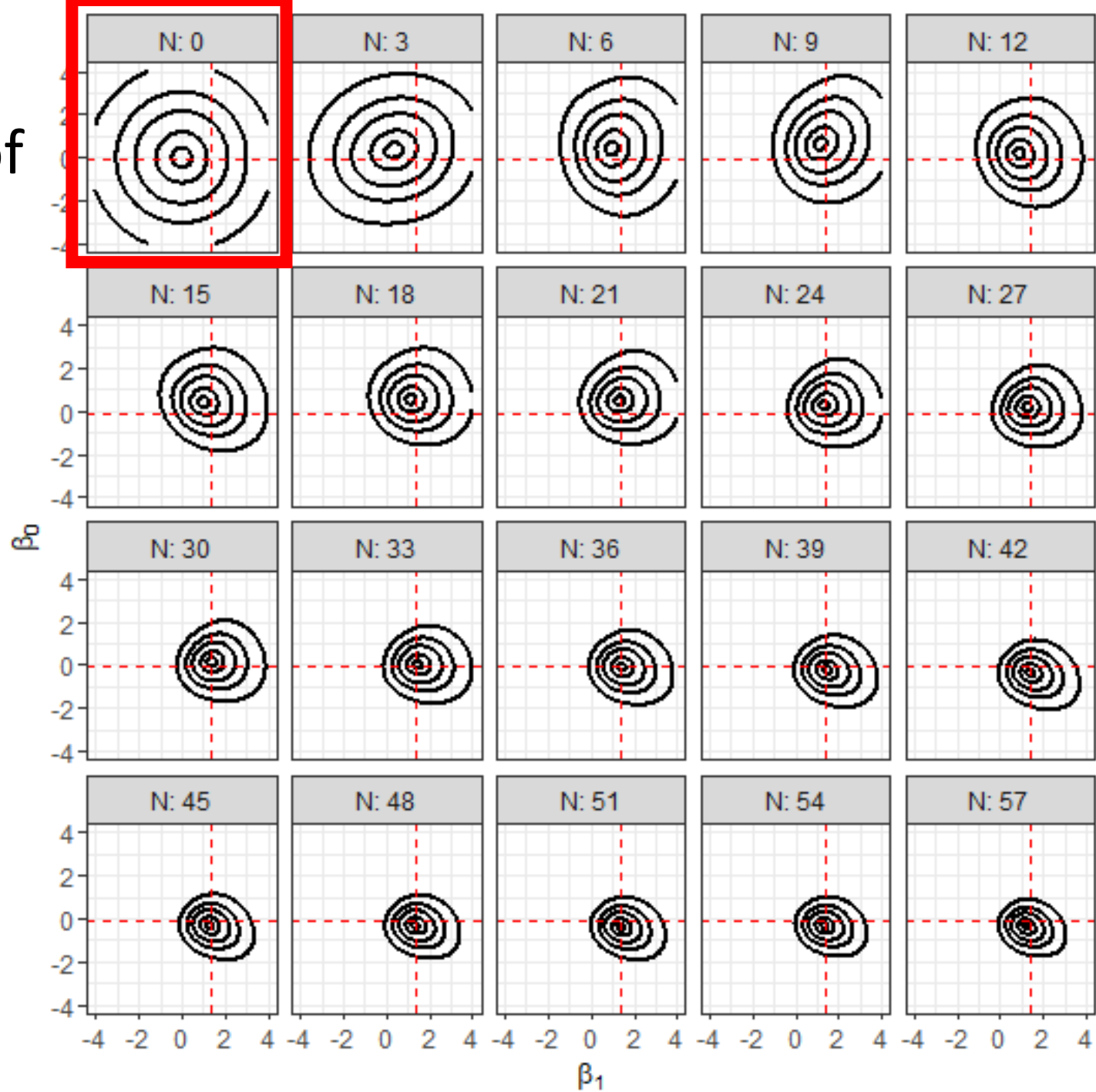


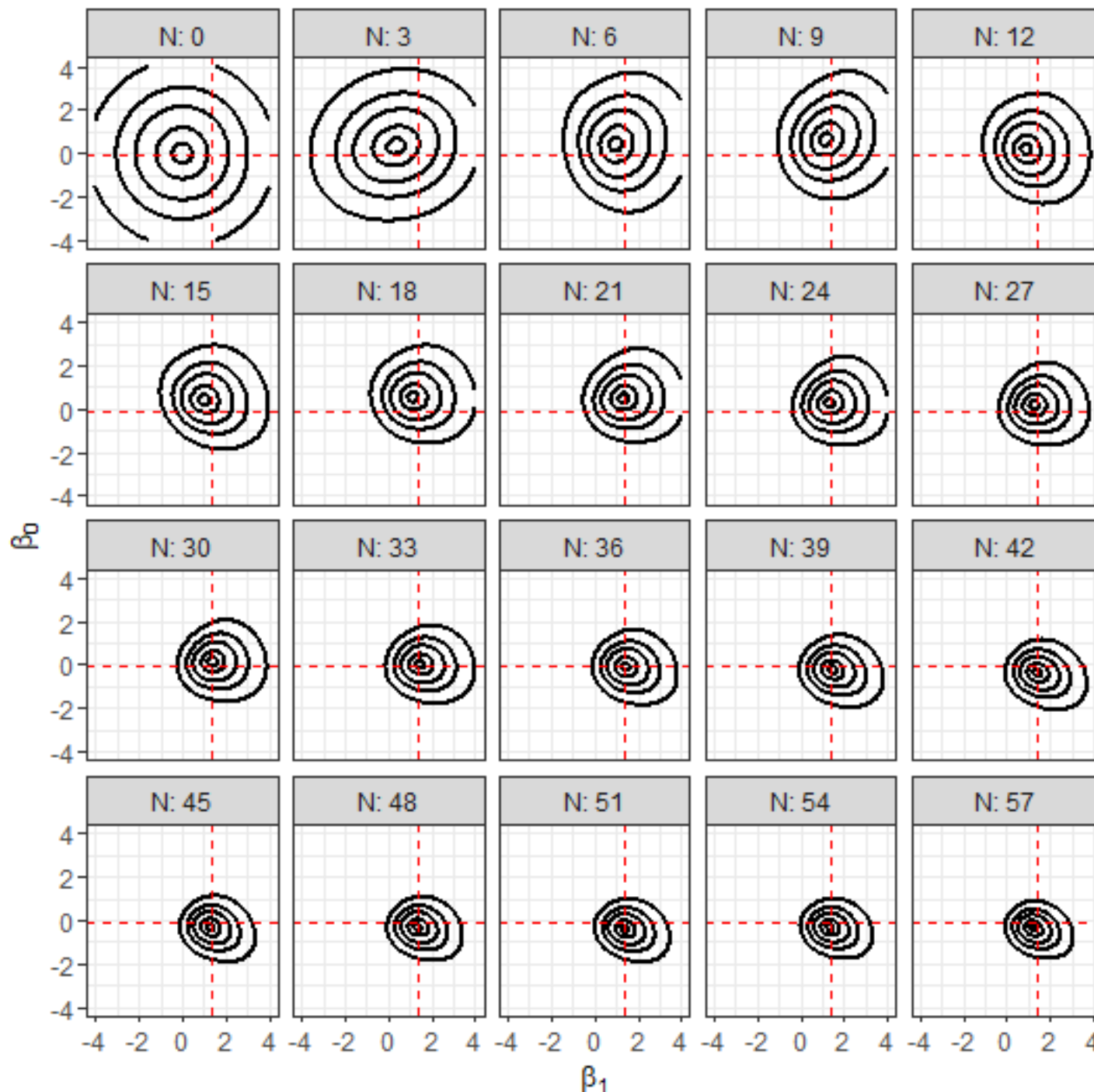
How does the log-posterior surface change as the sample size changes?

- Let's start with the prior and then add 3 data points at a time.



Why does the prior consist of concentric circles?



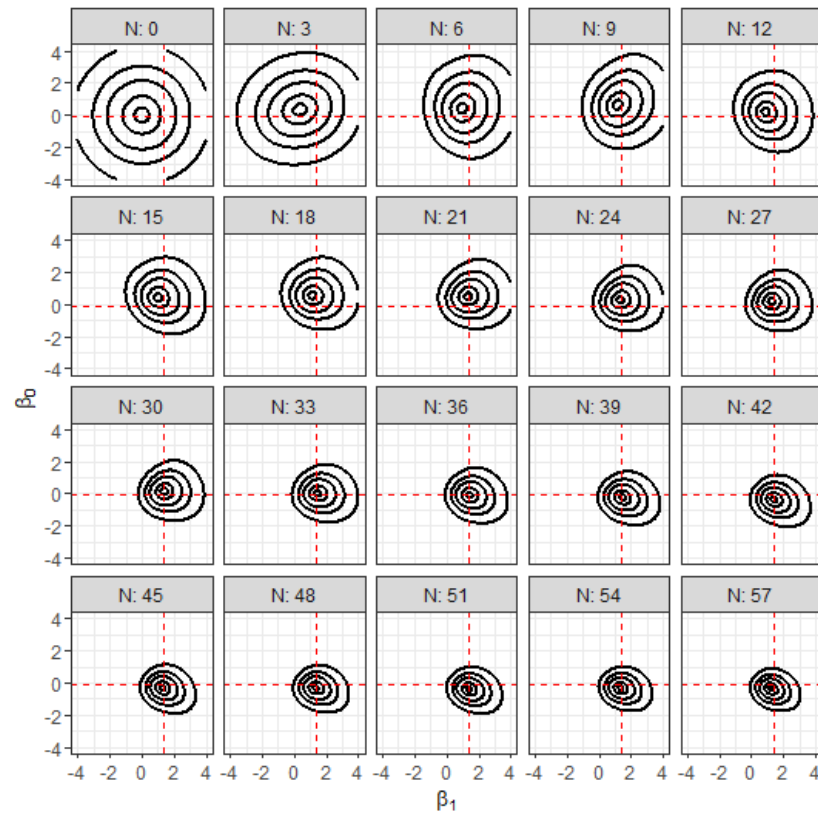


The mode moves closer to the true values as the sample size increases...

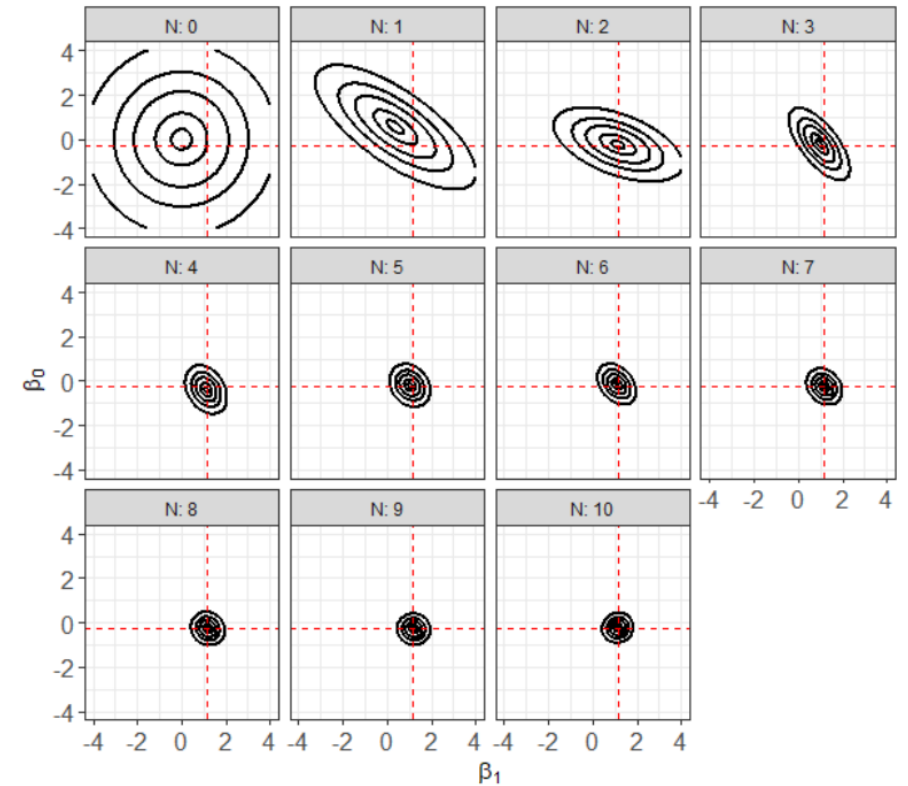
...the precision increases “slower” than the simple single input linear regression problem

...the precision increases “slower” than the simple single input linear regression problem

Logistic regression - GLM



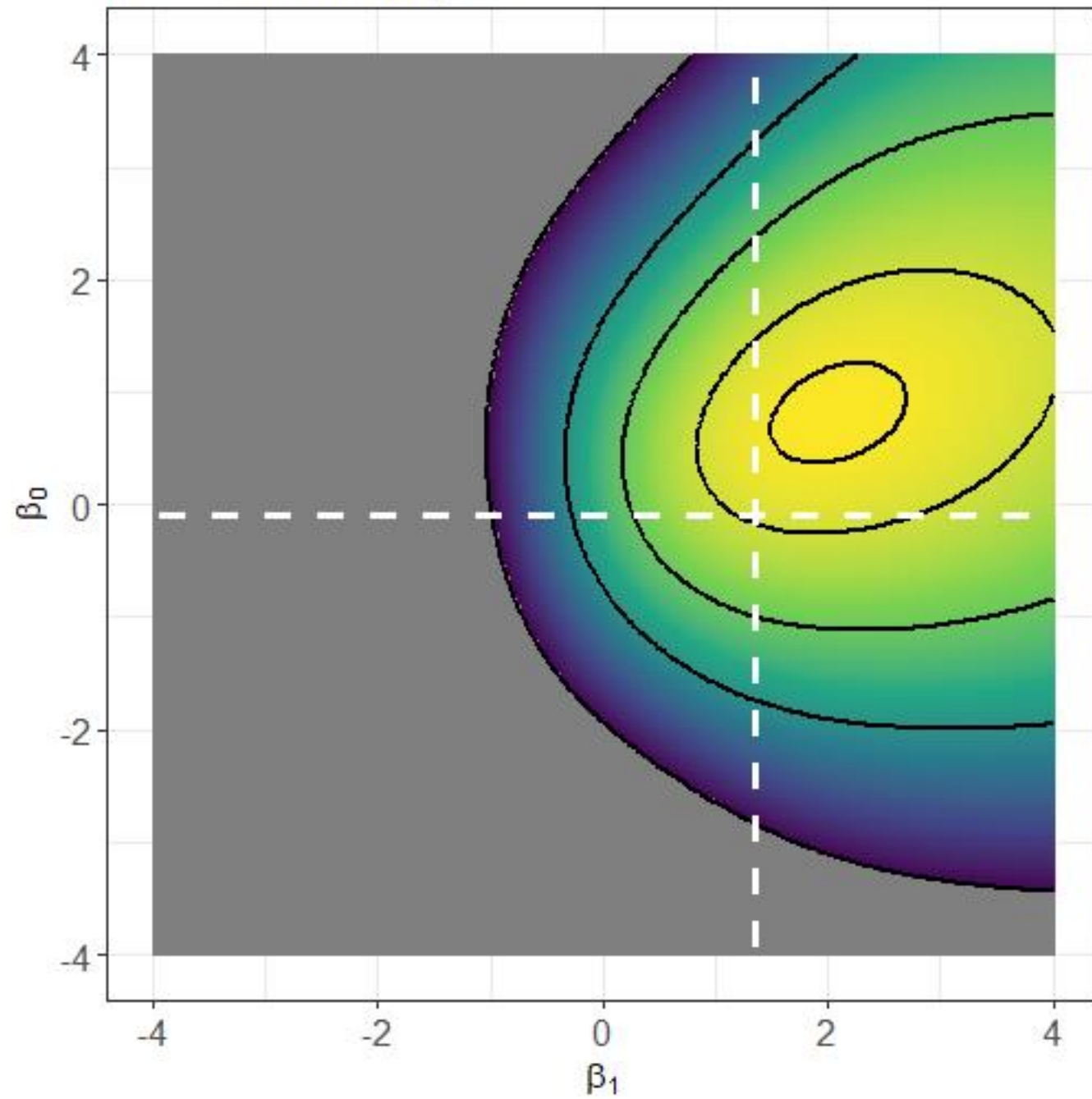
Linear Model



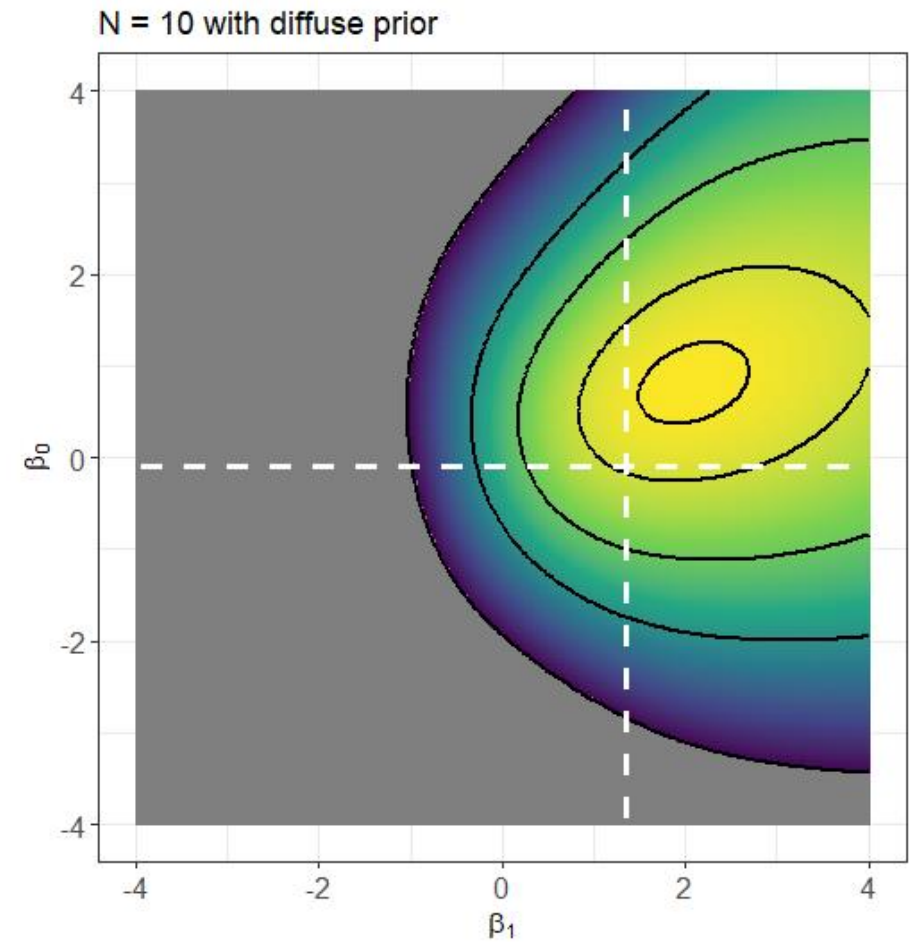
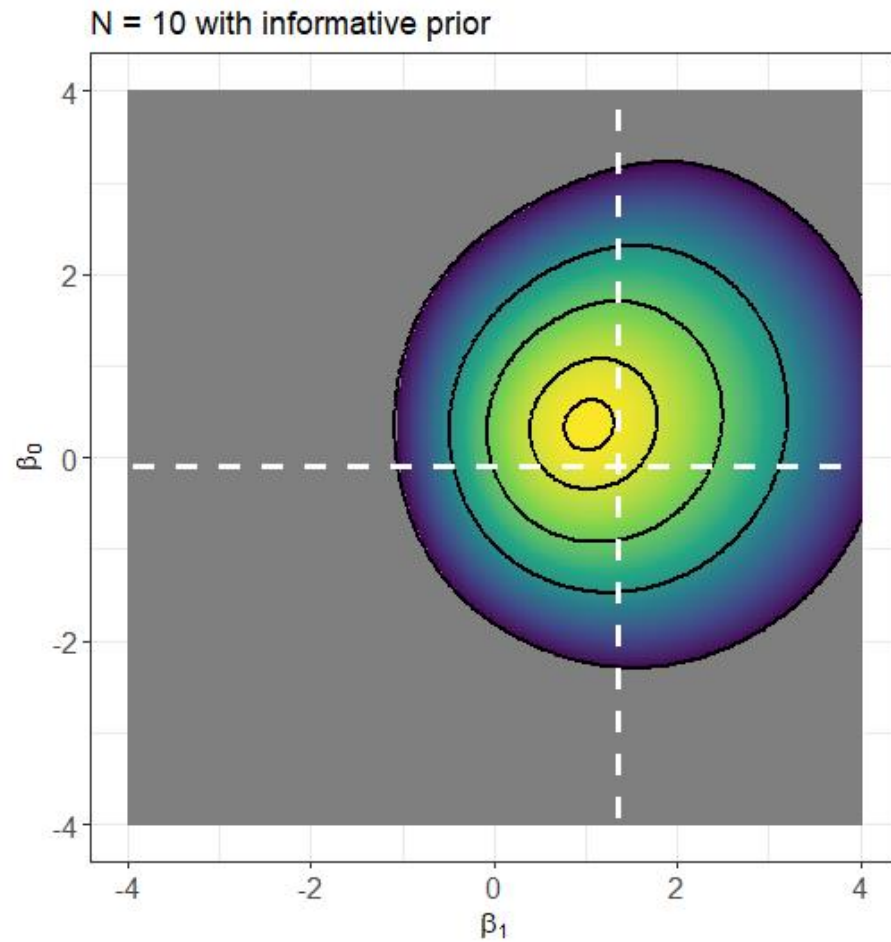
We used a strong prior...could that be why the log-posterior surface is unimodal?

- Let's use a diffuse, vague, very weak prior, with a prior standard deviation of 20, $\tau_\beta = 20$, instead of 1.
- Parameter values from -40 to +40 are allowed!

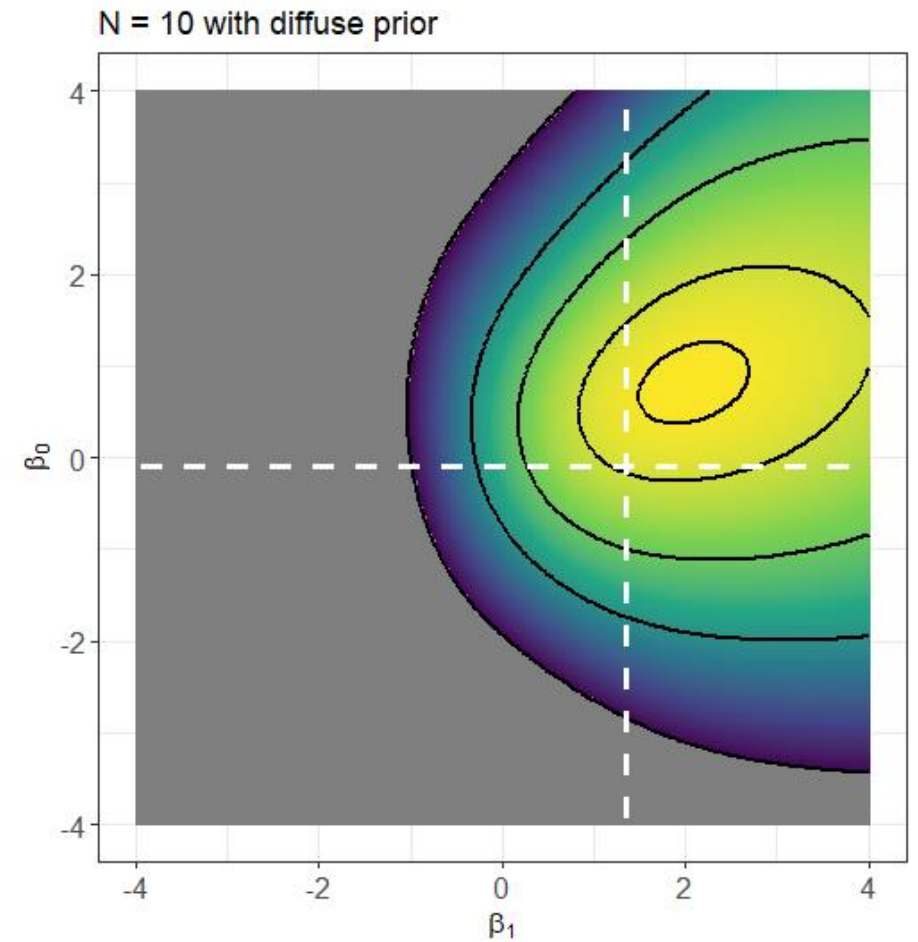
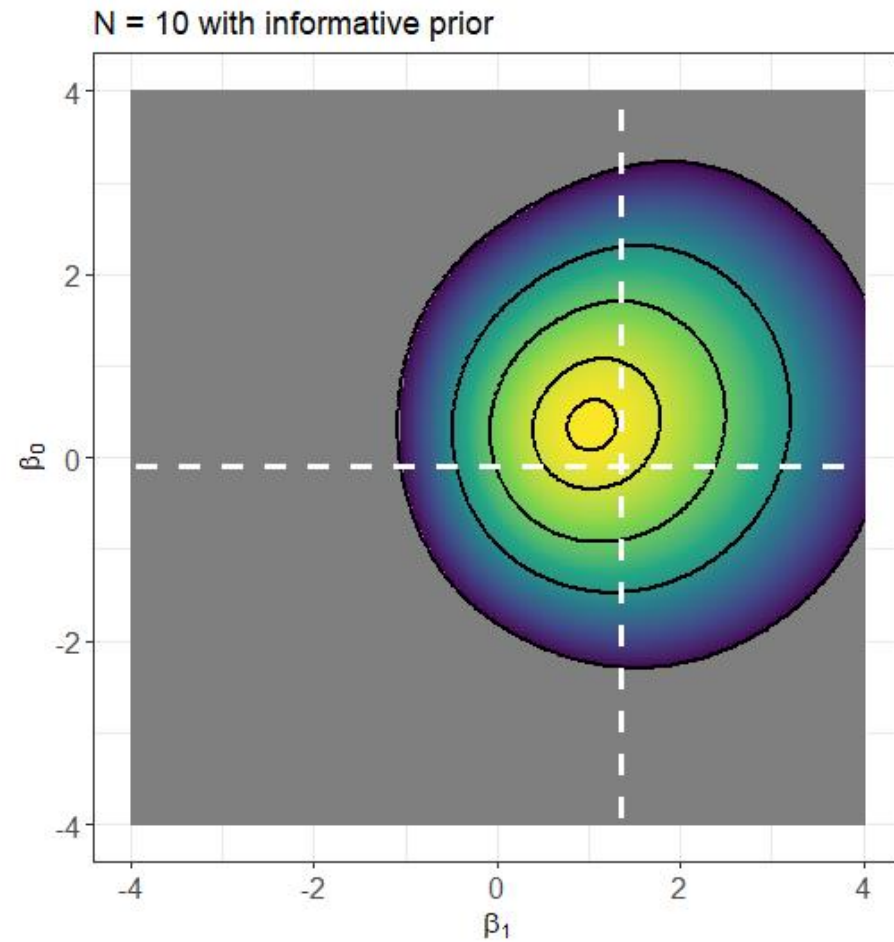
N = 10 with diffuse prior



The two log-posterior surface are different. The result starting from the diffuse prior has greater uncertainty!! Can you see why that is?



The posterior mode depends on our prior.
But, both have just a single posterior mode!!!!

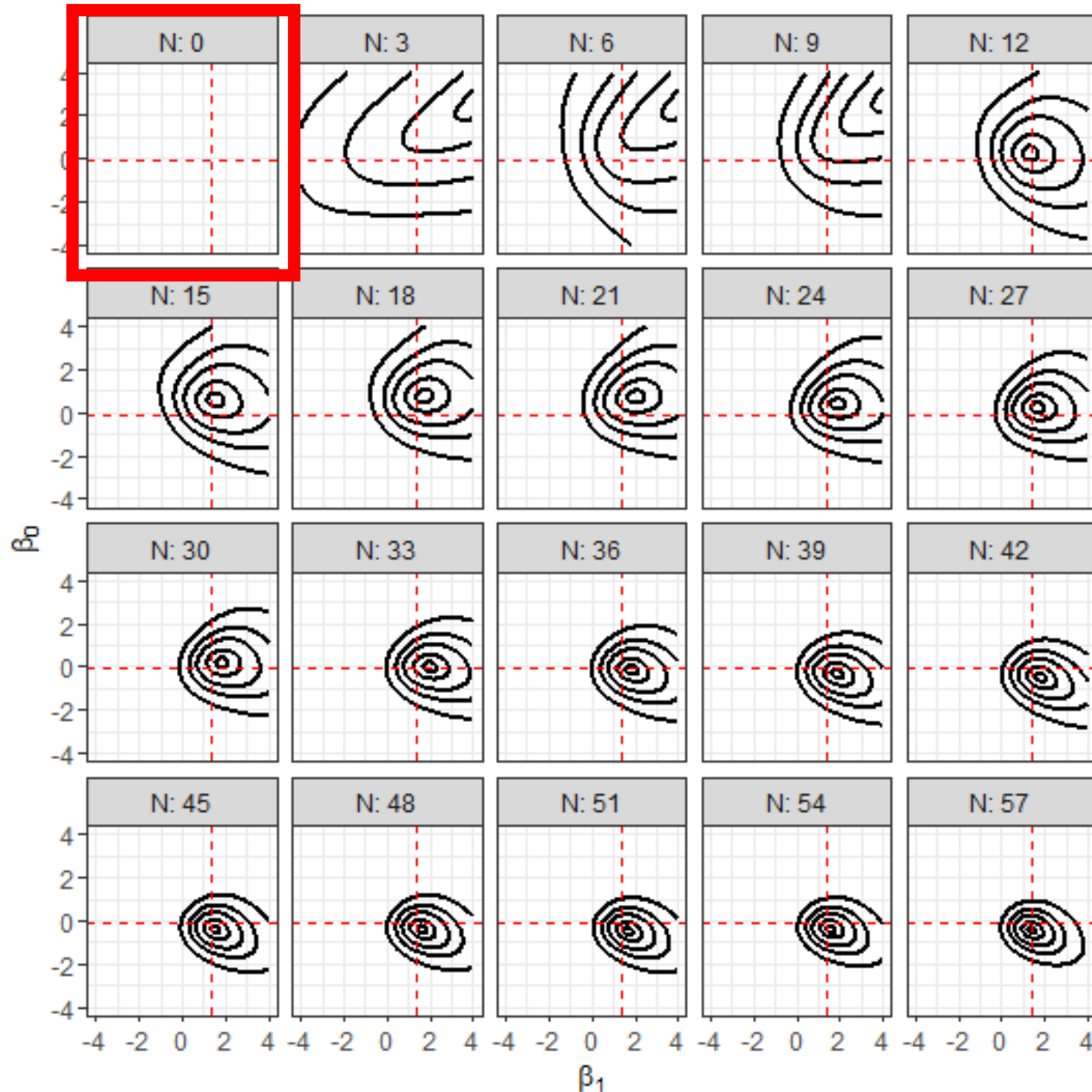


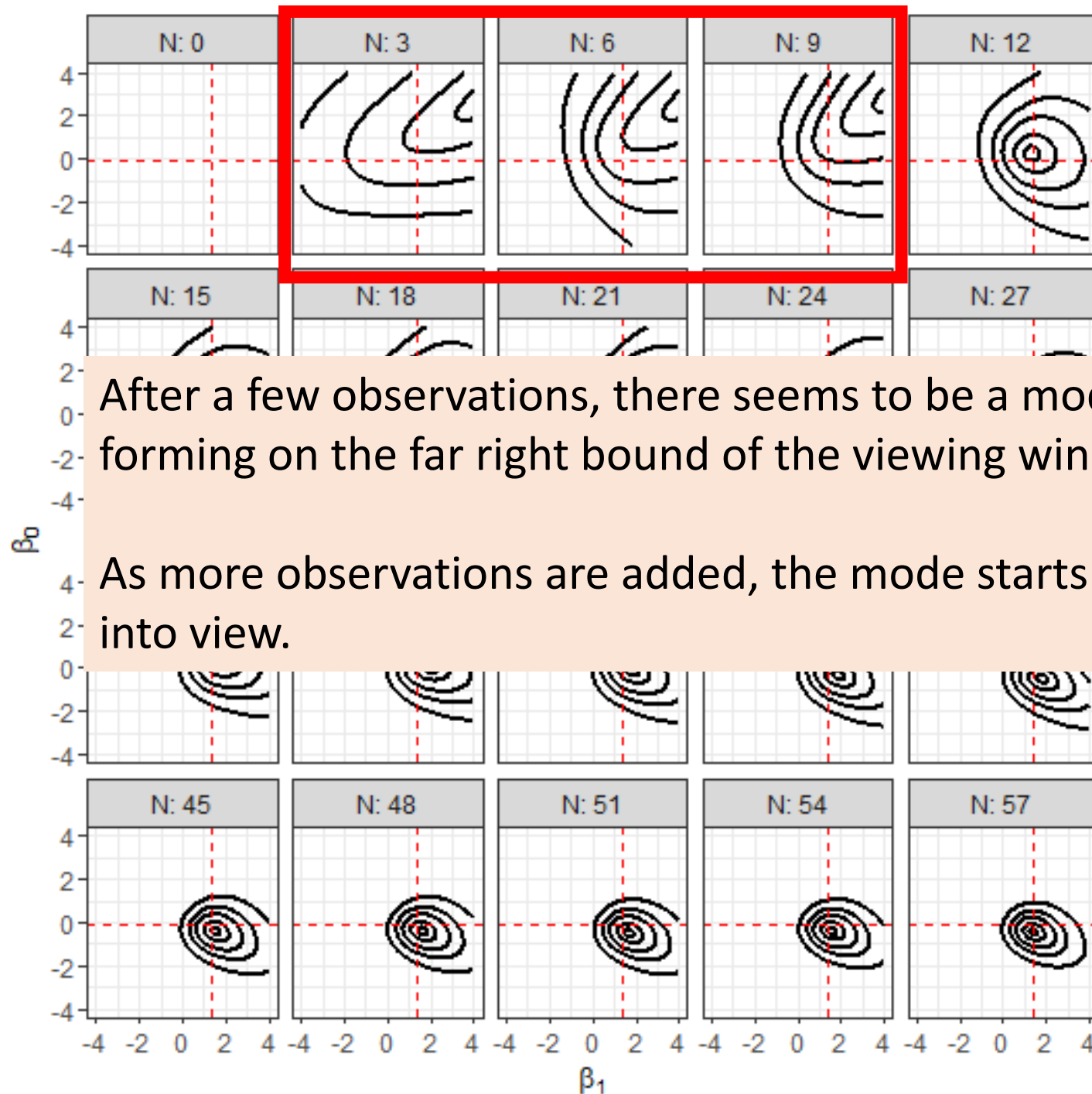
We will now repeat the study of varying the sample size for the diffuse prior case.

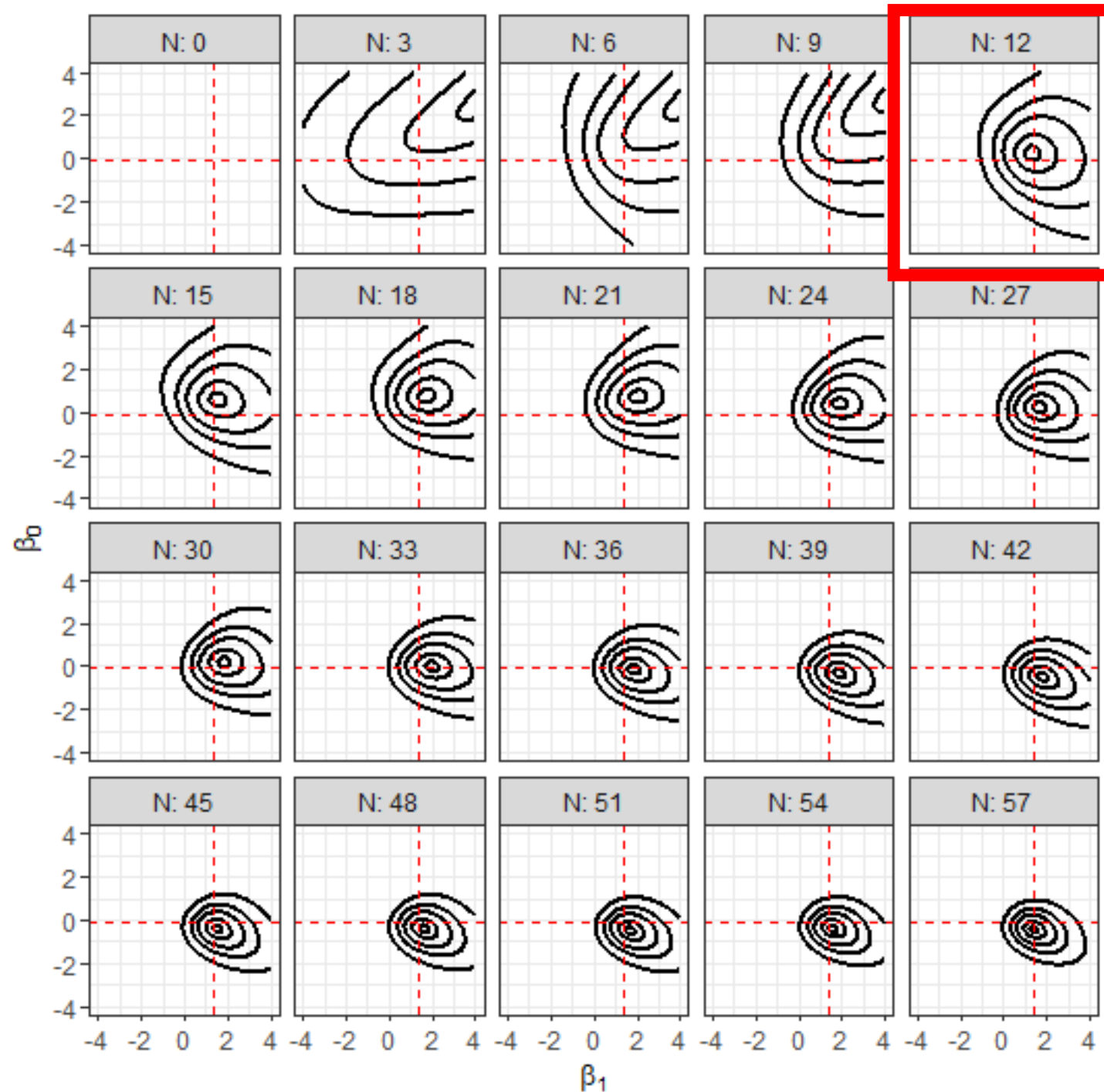
- Start again from the prior and then sequentially add 3 data points at a time.

The prior is so diffuse we do not see any contour lines over our “viewing window”.

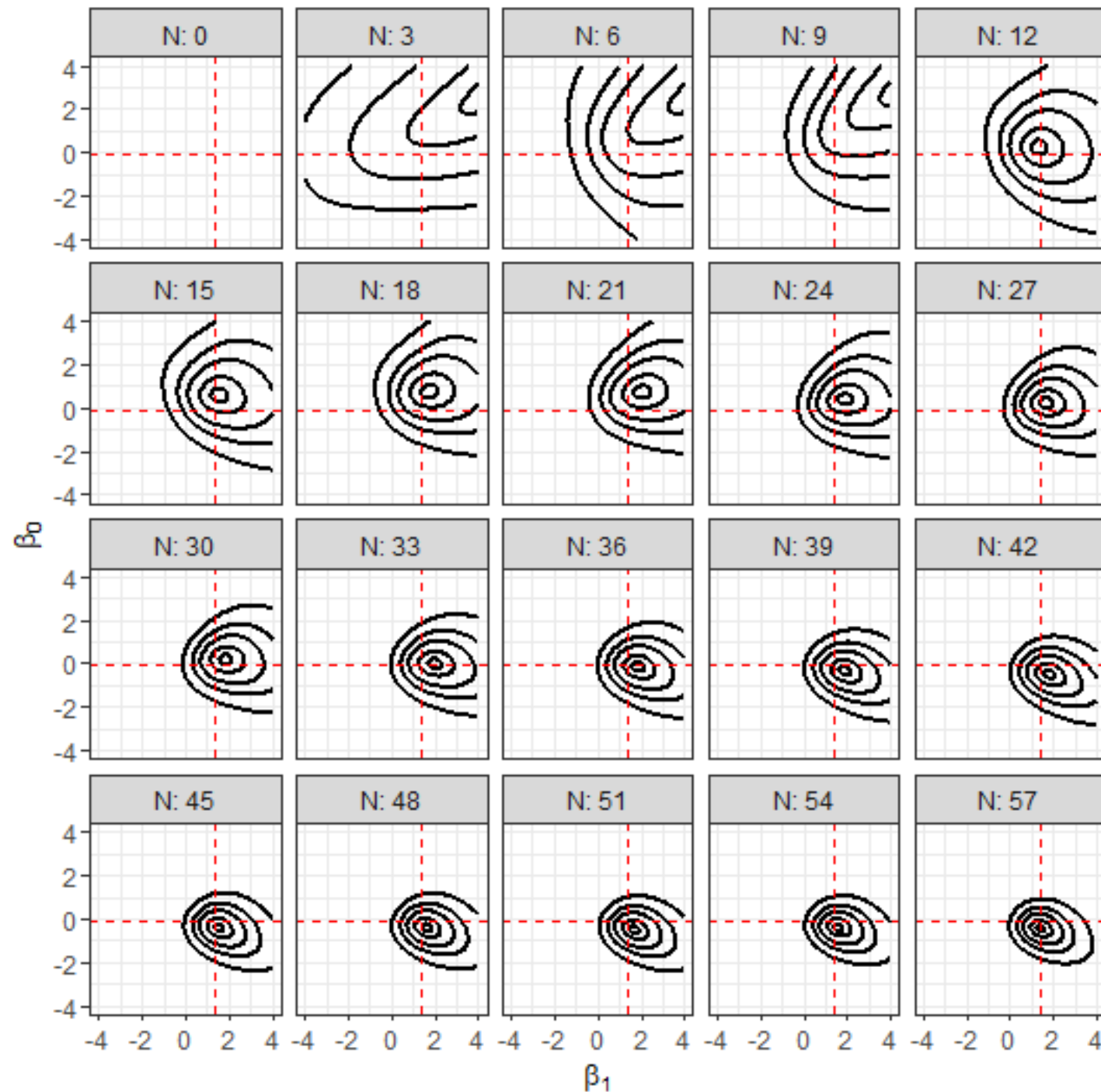
All combinations within the viewing window are essentially equally likely.



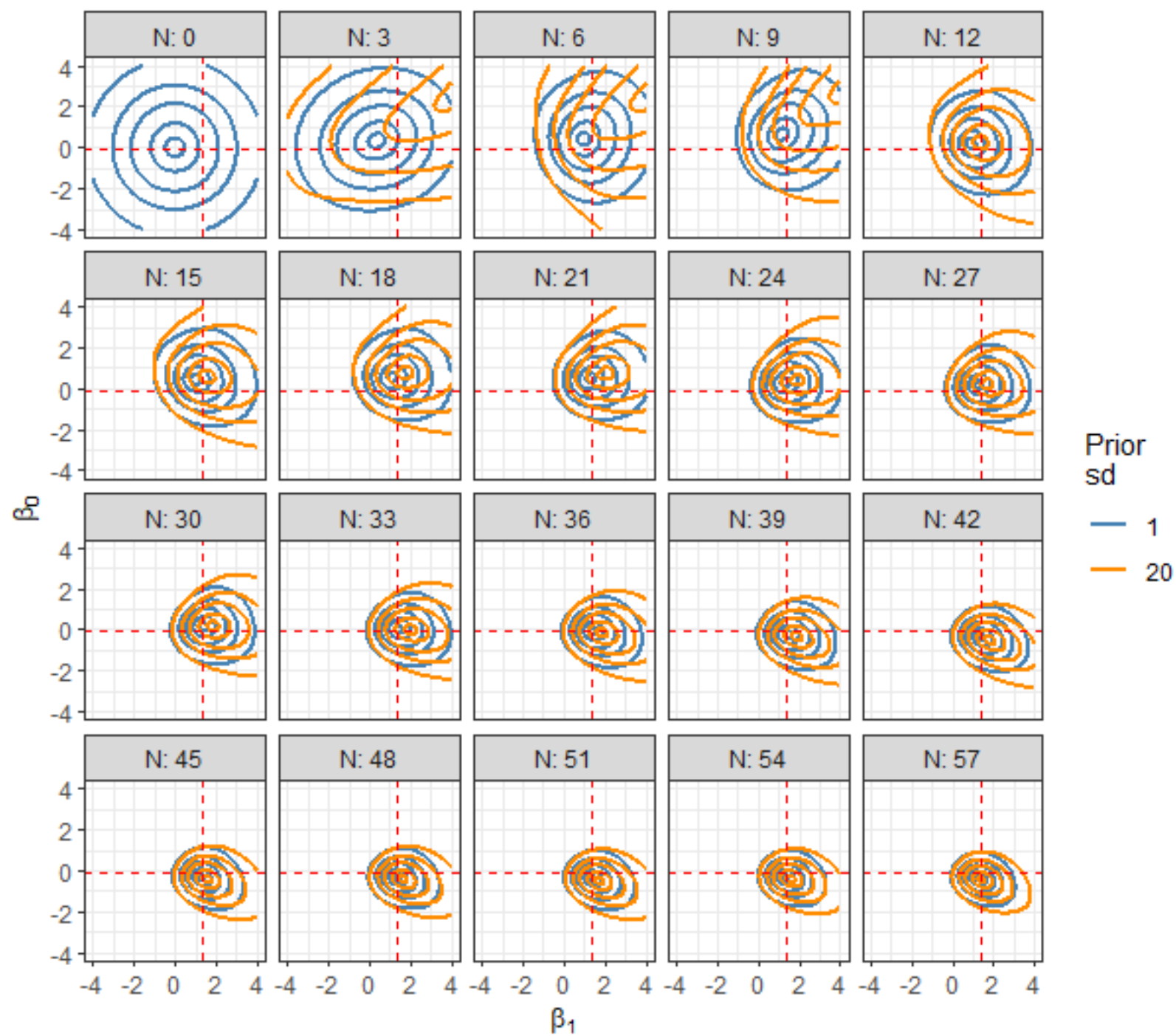


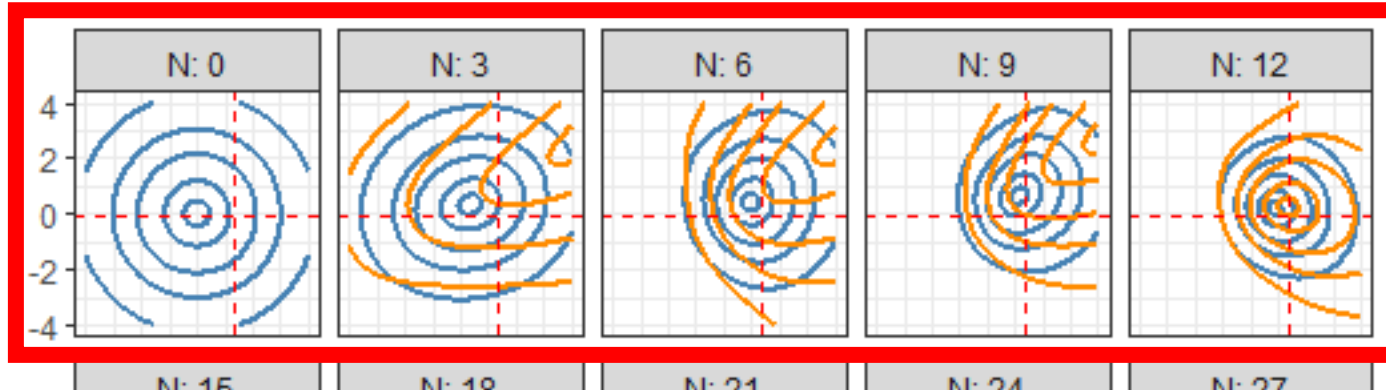


There's clearly just a single mode once we have "enough" observations!



How do the results compare to the posterior from the informative case?





Compromise between the prior and the data still applies to logistic regression!!!

With very few observations, the informative prior is more precise than the “data precision”.

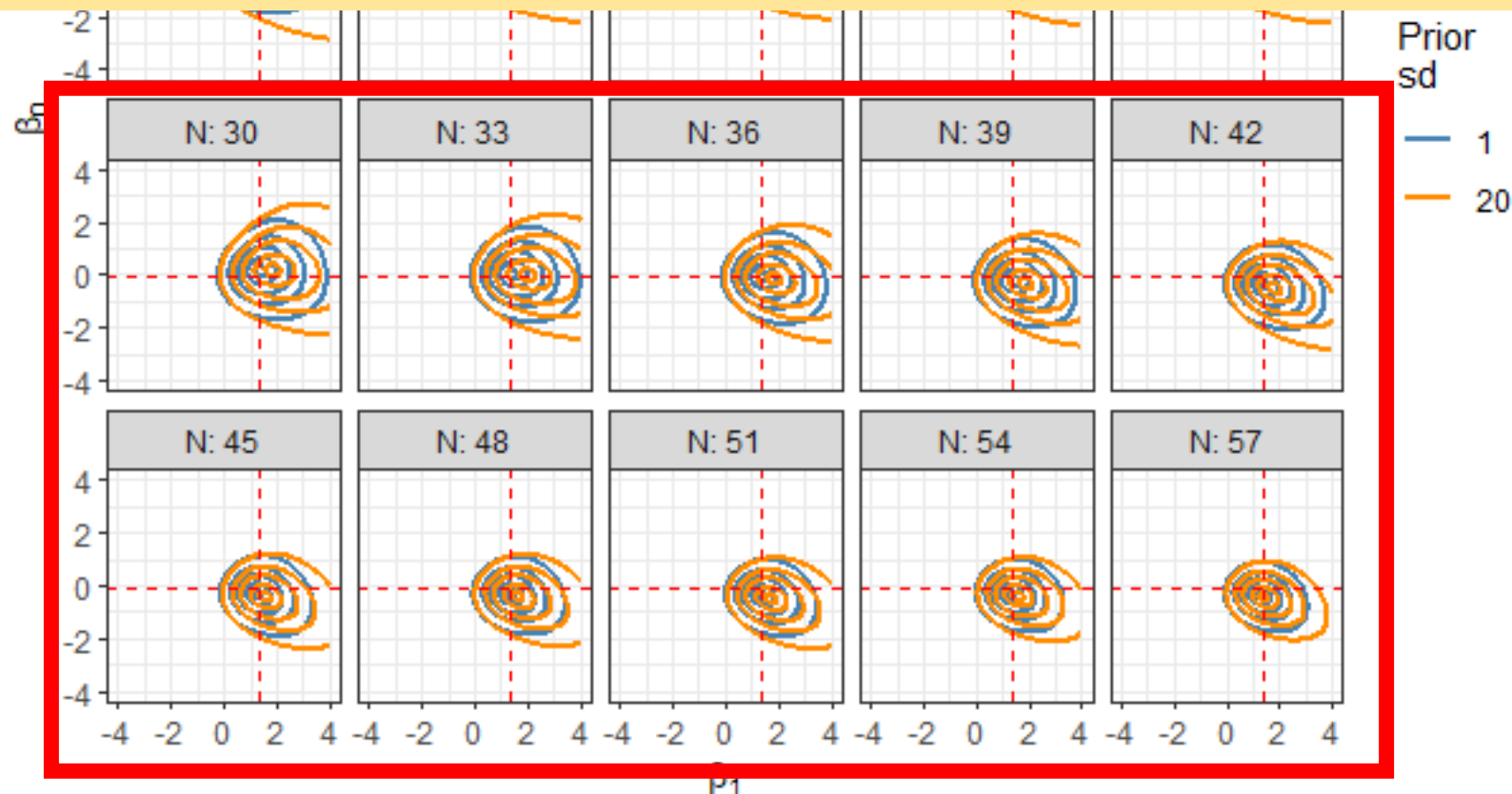
The prior **CONSTRAINS** the **allowable** values the parameters can take.

The posterior therefore follows the prior rather than the likelihood.

Prior
sd
— 1
— 20

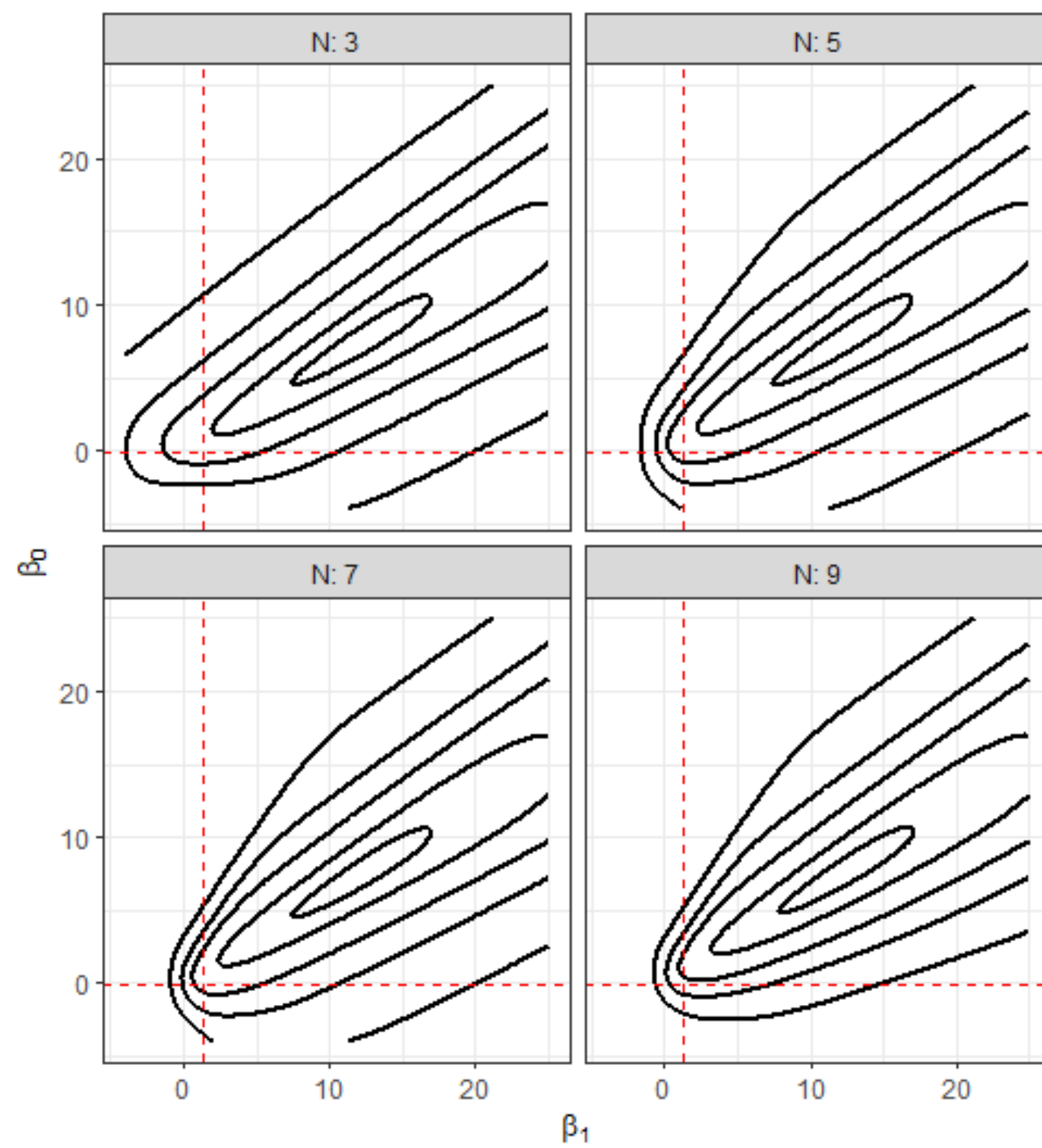
Eventually, the data overwhelms the prior, and so the posterior follows the likelihood.

From this simple study...it seems that logistic regression requires more observations than a standard linear model for the prior to be overwhelmed by the data.

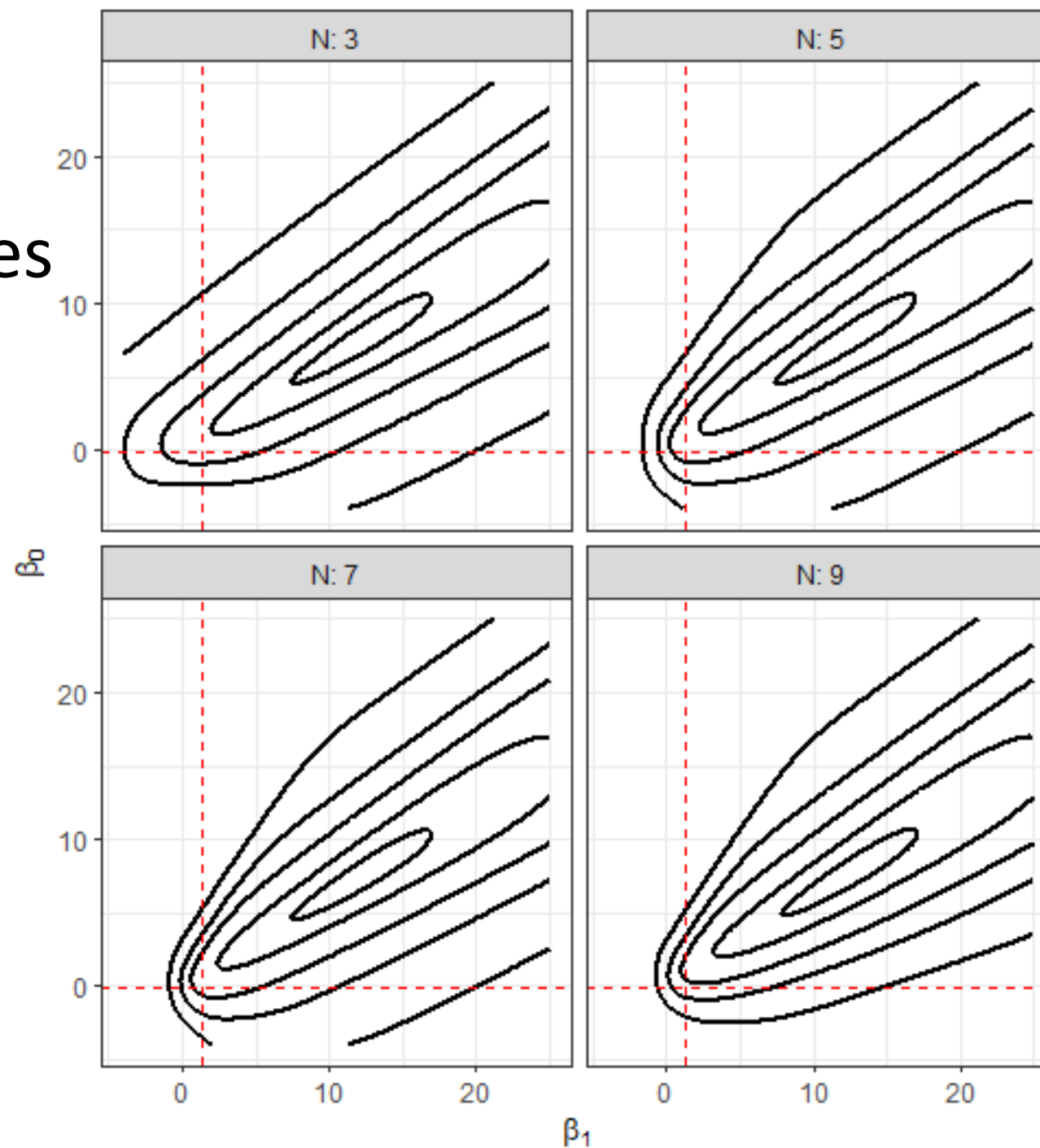


Let's see what's going on with the diffuse prior results when we have very limited data

- It seems like a mode does exist, but it's just "off the screen".
- Expand the grid from -4 to +4 to -4 to +25 for both parameters.

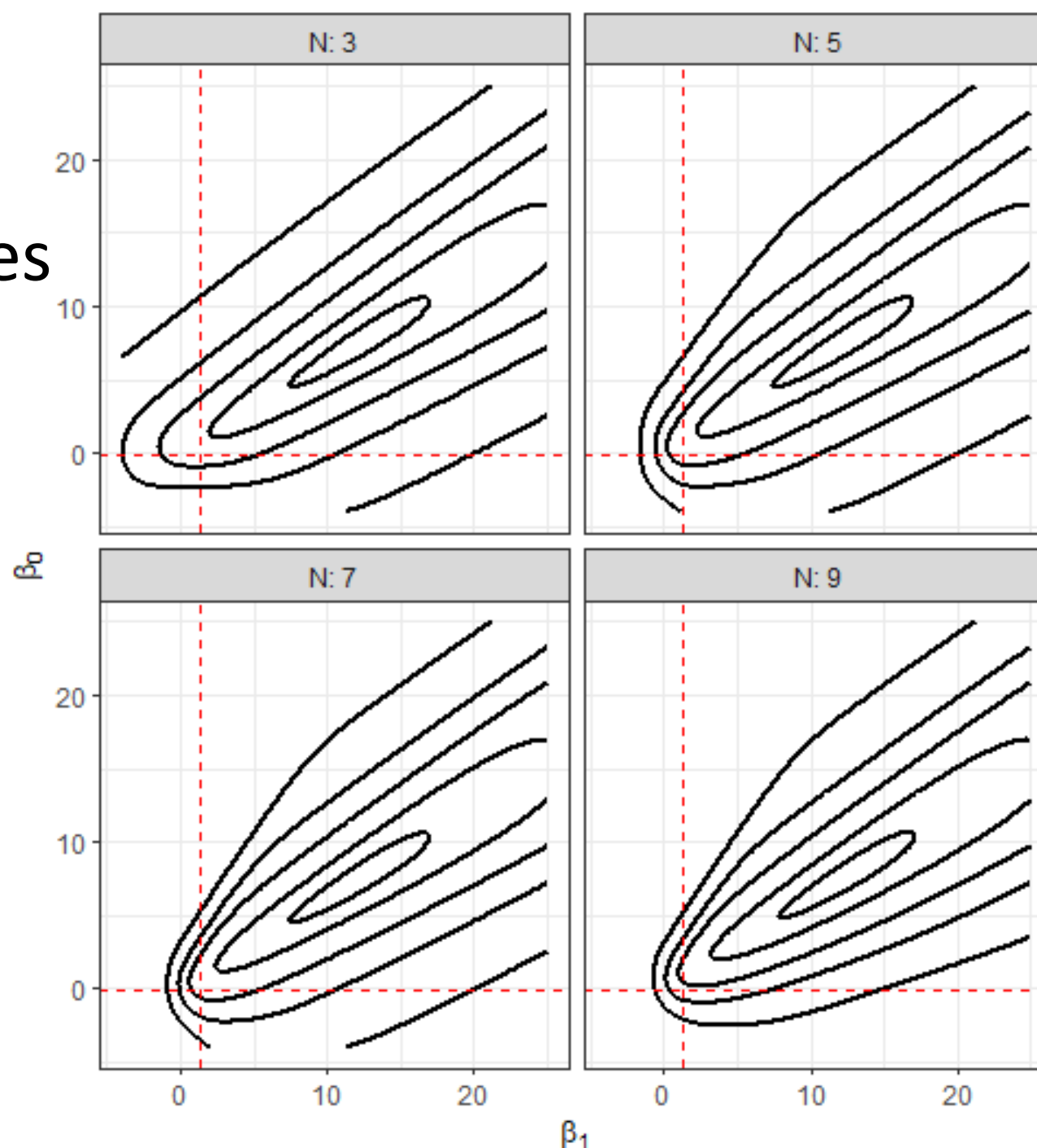


Posterior modes
still exist!!!!



Posterior modes
still exist!!!!!!

Look at the
values of the
modes...

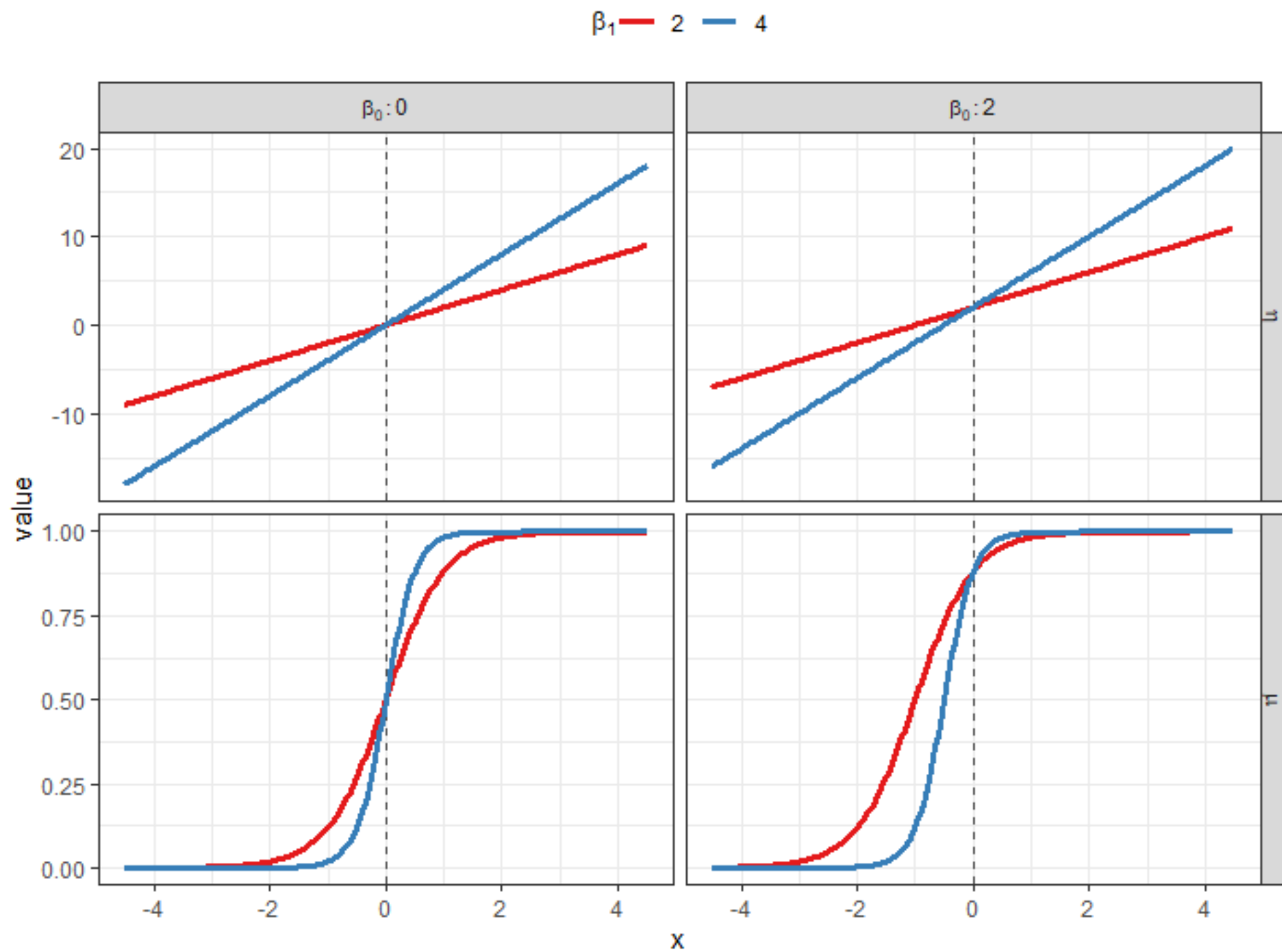


Intercept and
slope appear
very correlated

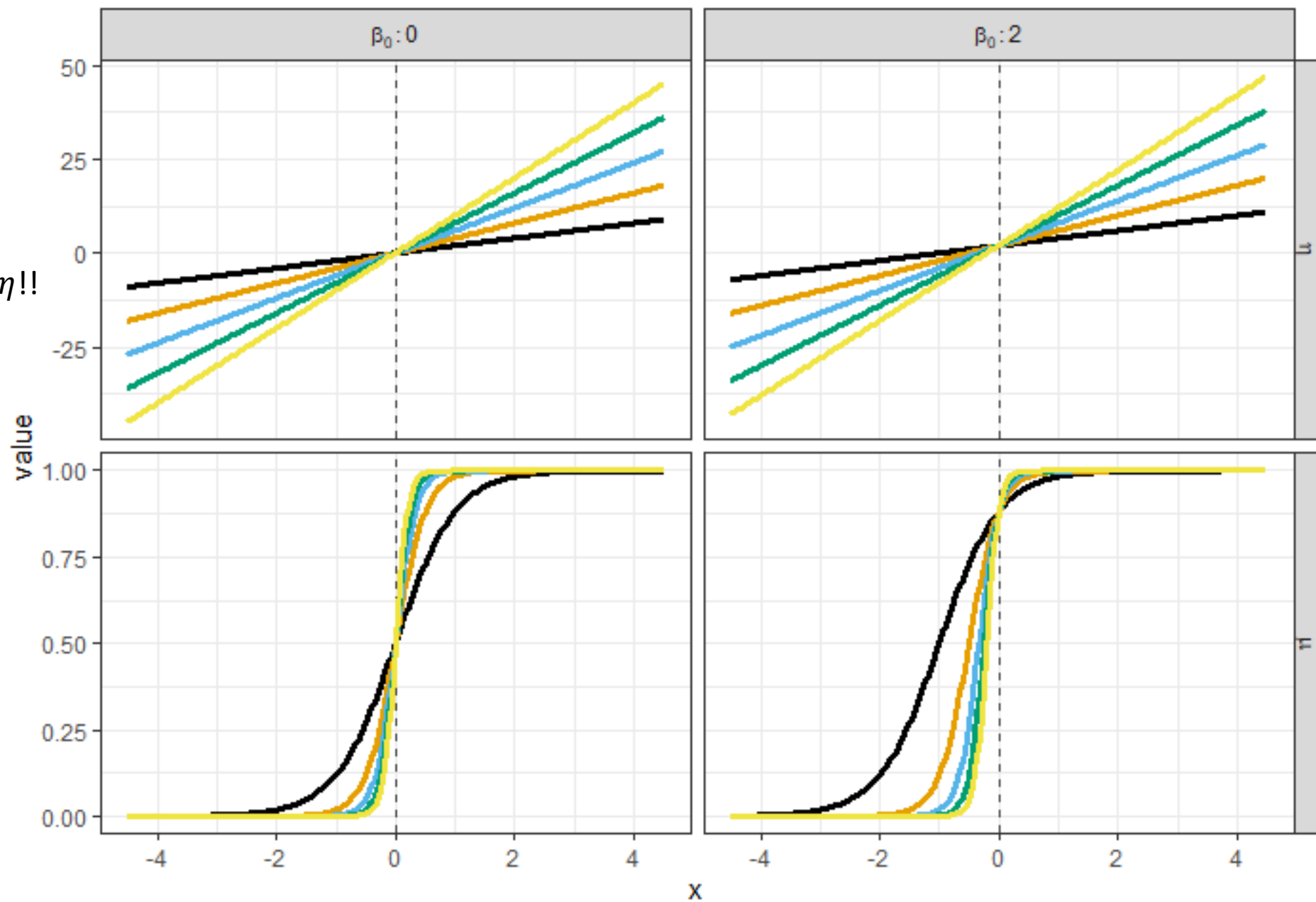
...the modes
seem to be
between
slopes of 10
and 15...

We looked at the trends of the η and μ for “regular” single digit values of the parameters

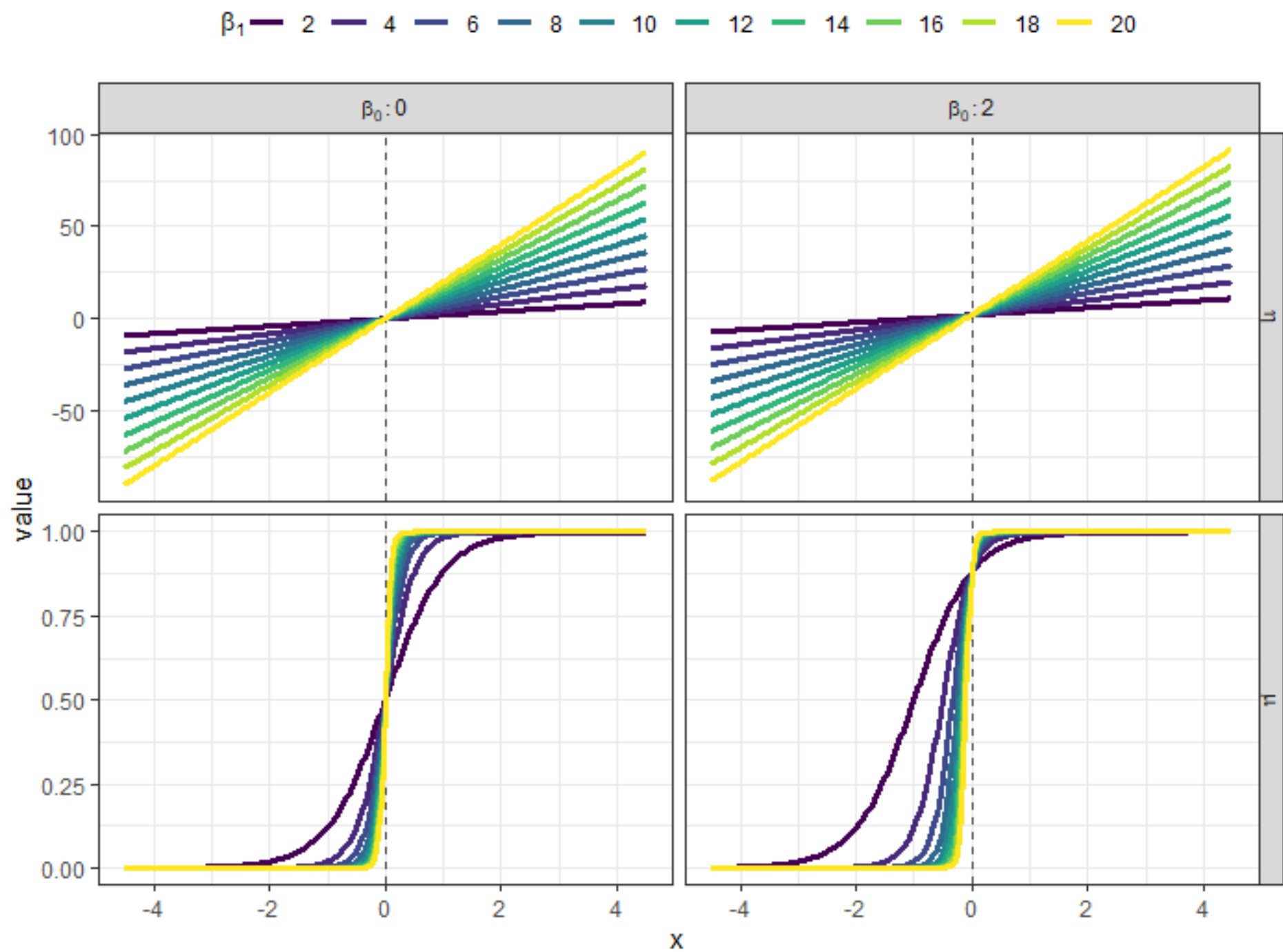
- Let’s now look to see what about when the intercept and slope are at larger double digit values.
- As a refresher, the next slide shows the trends when the slope is 2 and 4 and the intercept is 0 and 2.

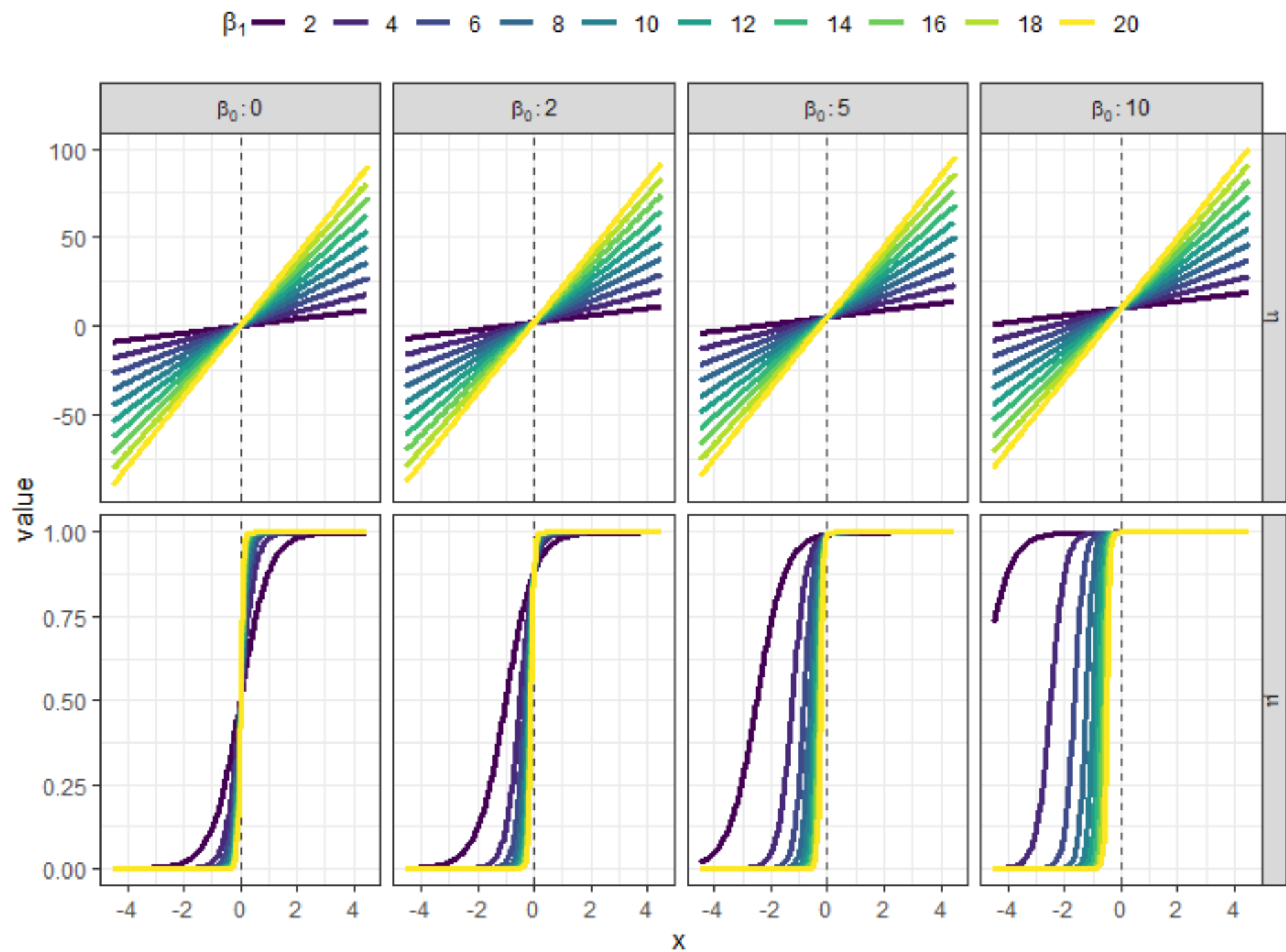


β_1 — 2 — 4 — 6 — 8 — 10



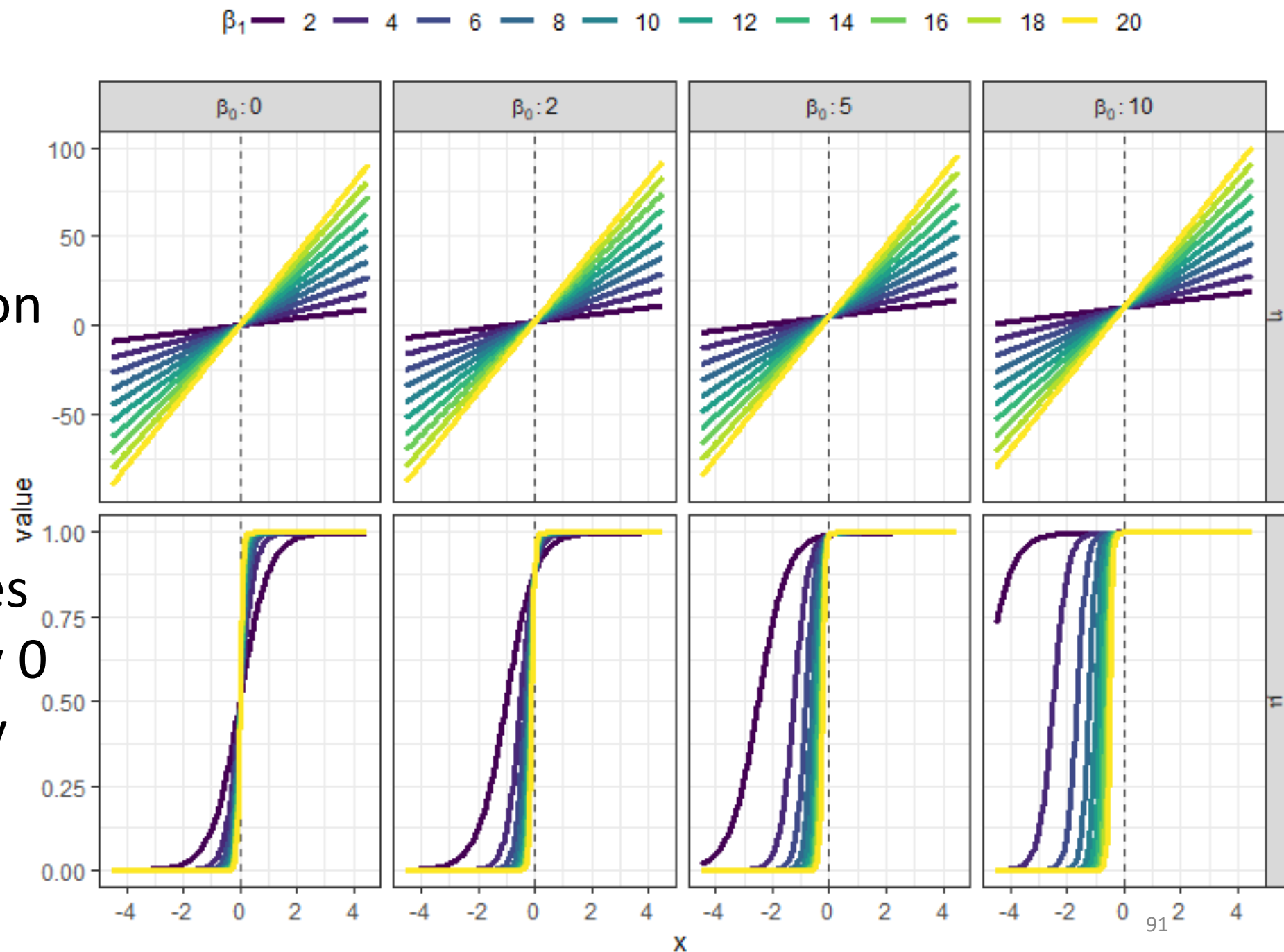
y-axis scale has
changed
substantially for η !!





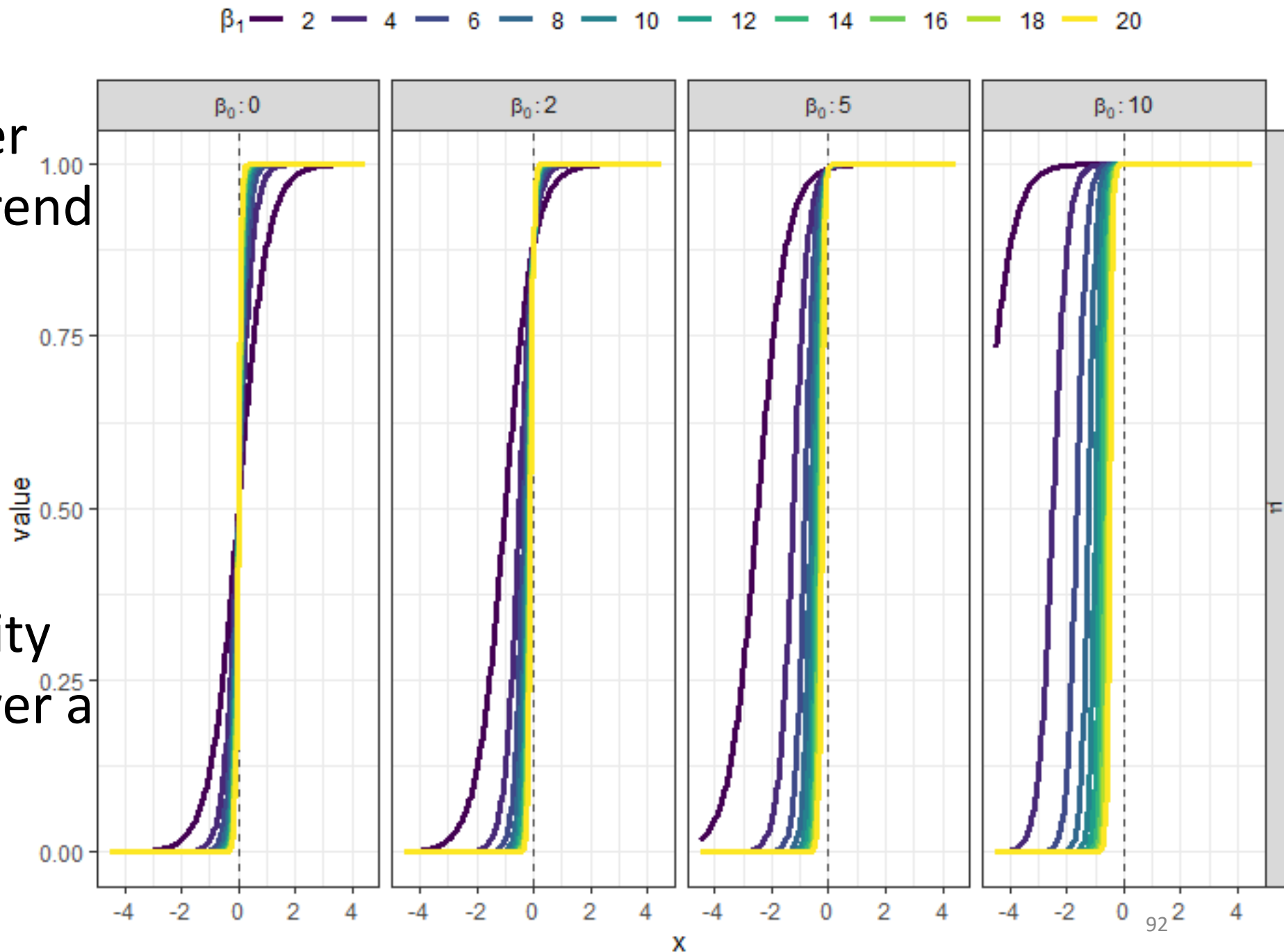
With double digit intercepts and slopes, η can take on large values for moderate x values.

Those large η values convert to basically 0 and 1 in probability space!!!

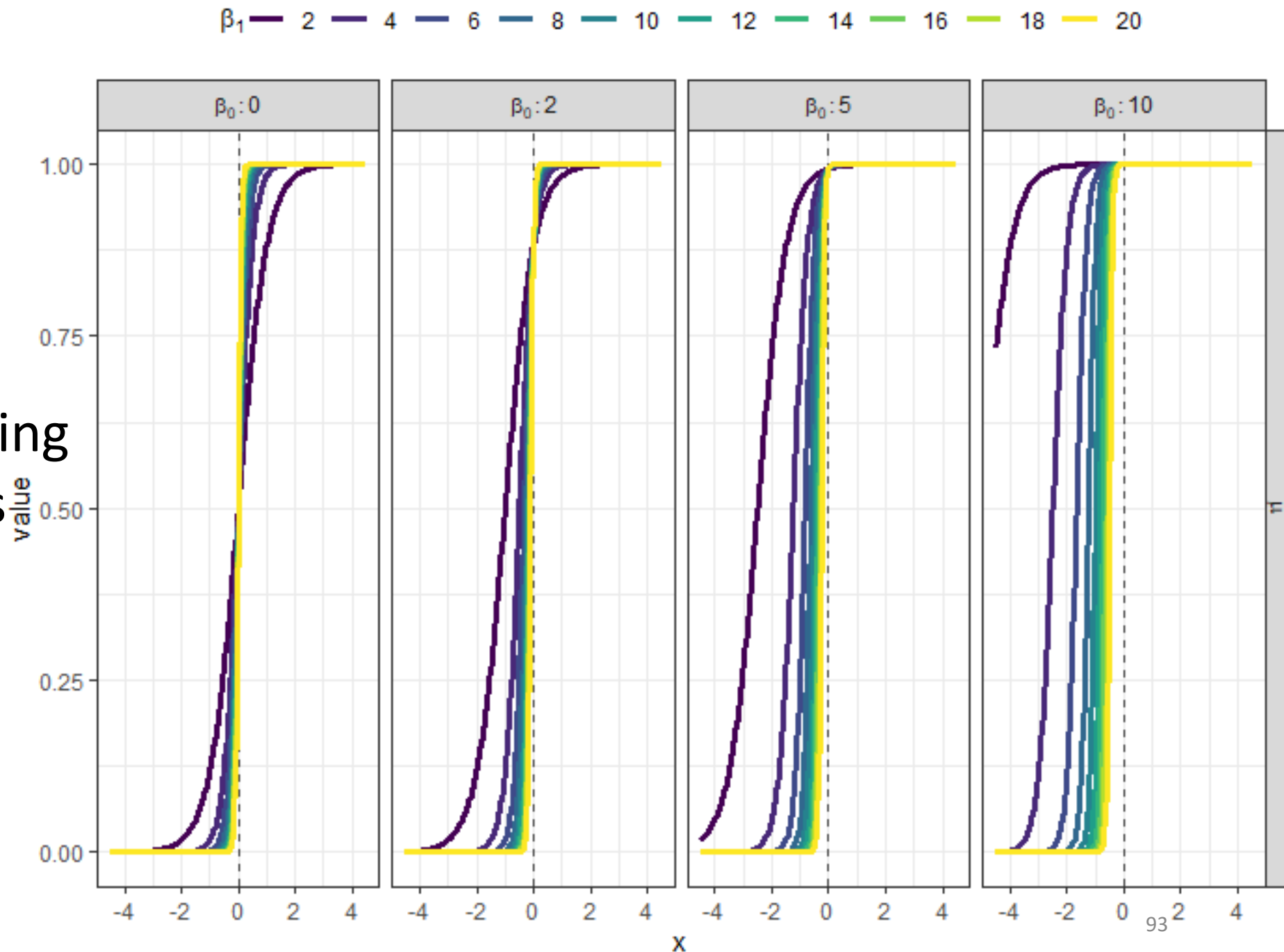


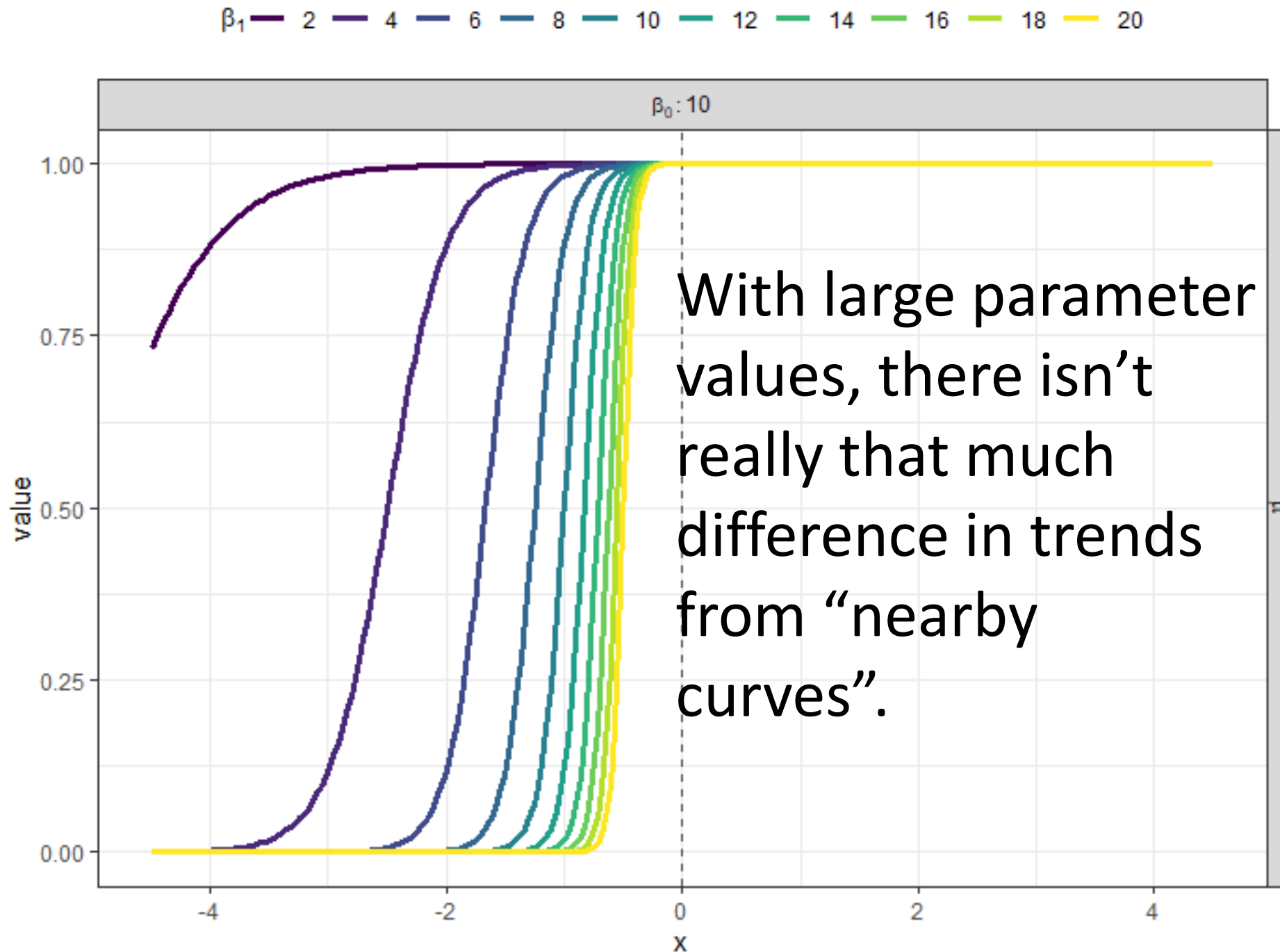
The large parameter values turn the μ trend into a **step change**.

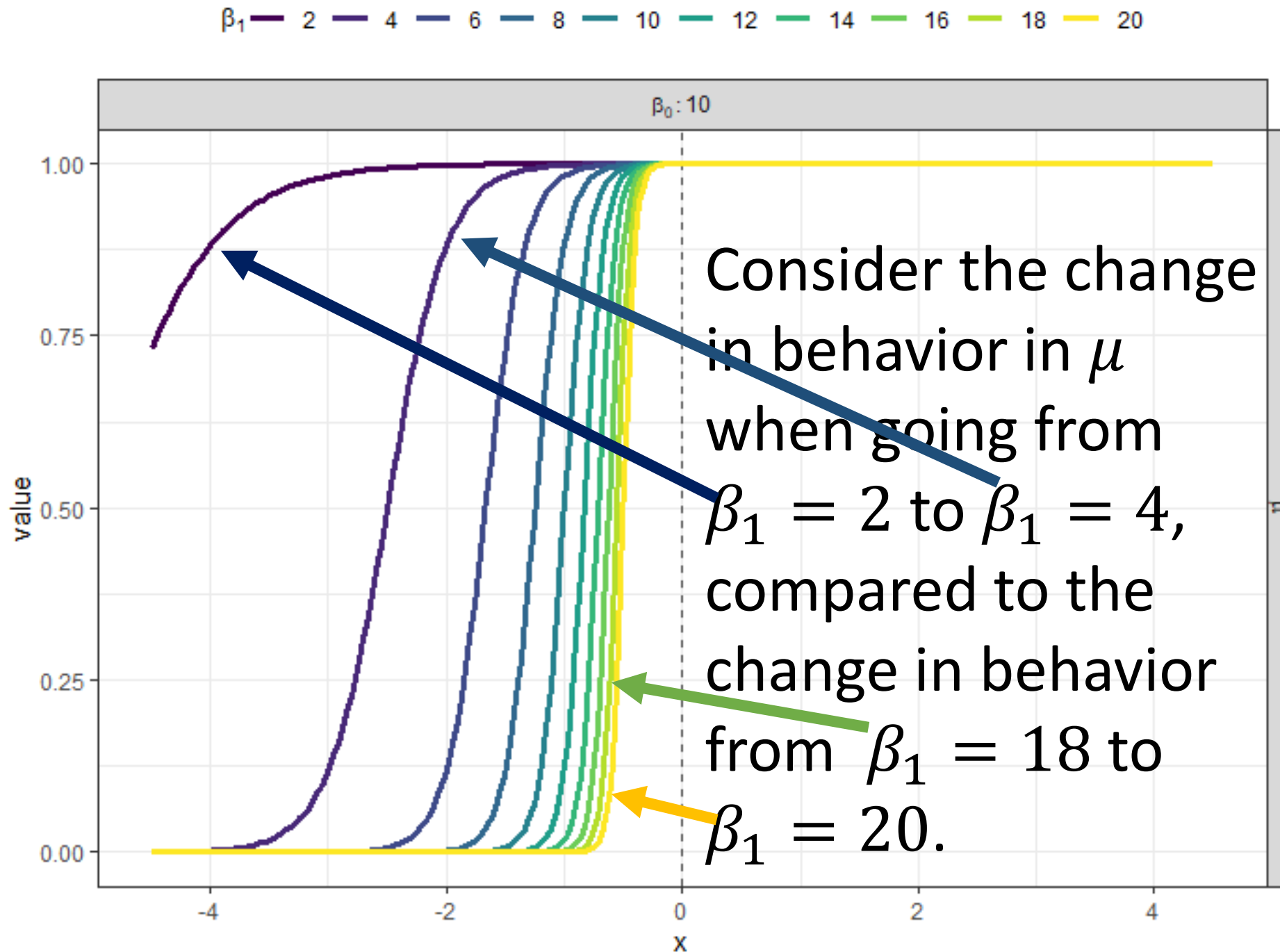
The event probability goes from 0 to 1 over a very **narrow input interval**.

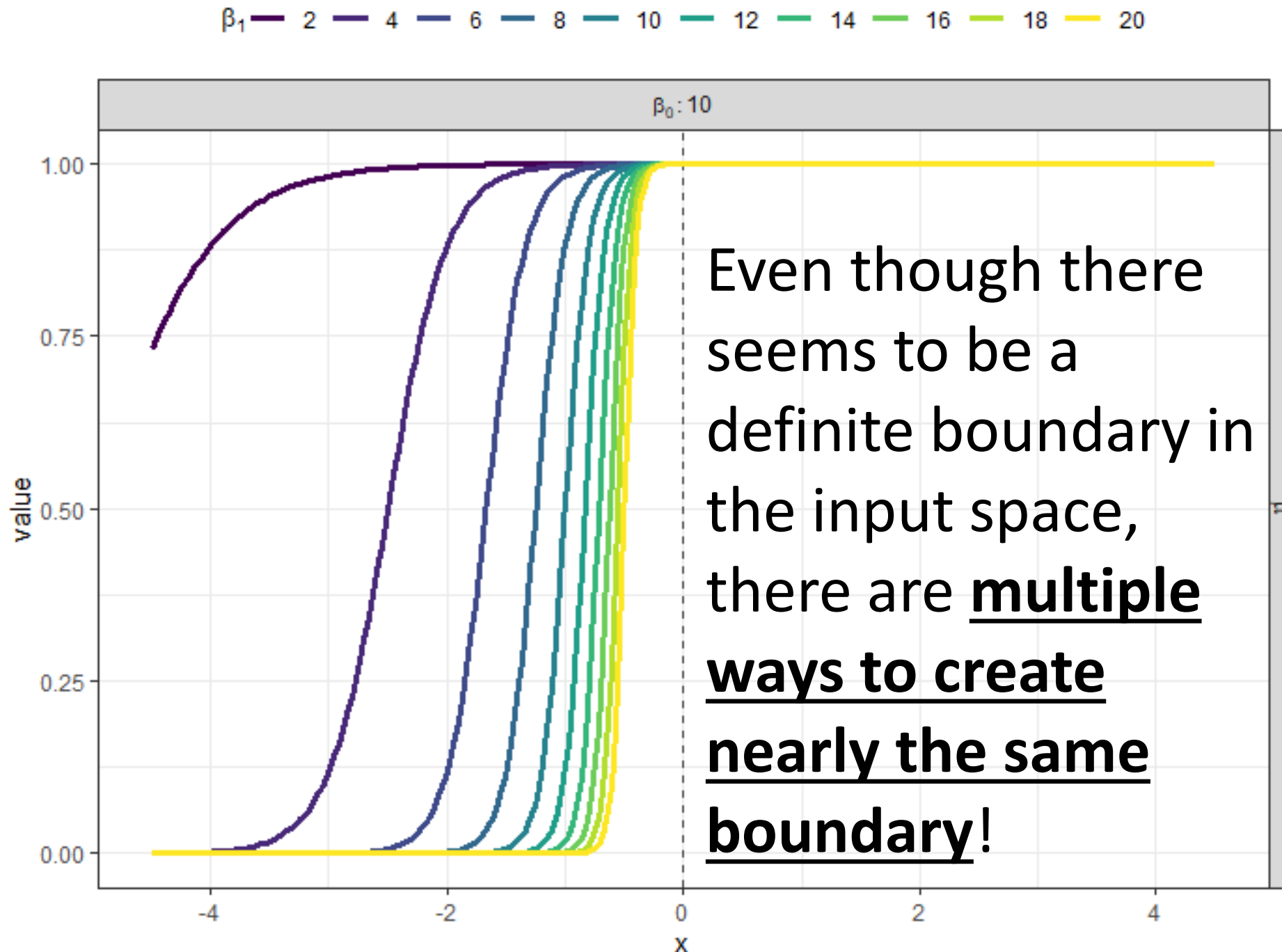


It's like there's a
definite boundary
between never seeing
the event to always
seeing the event...

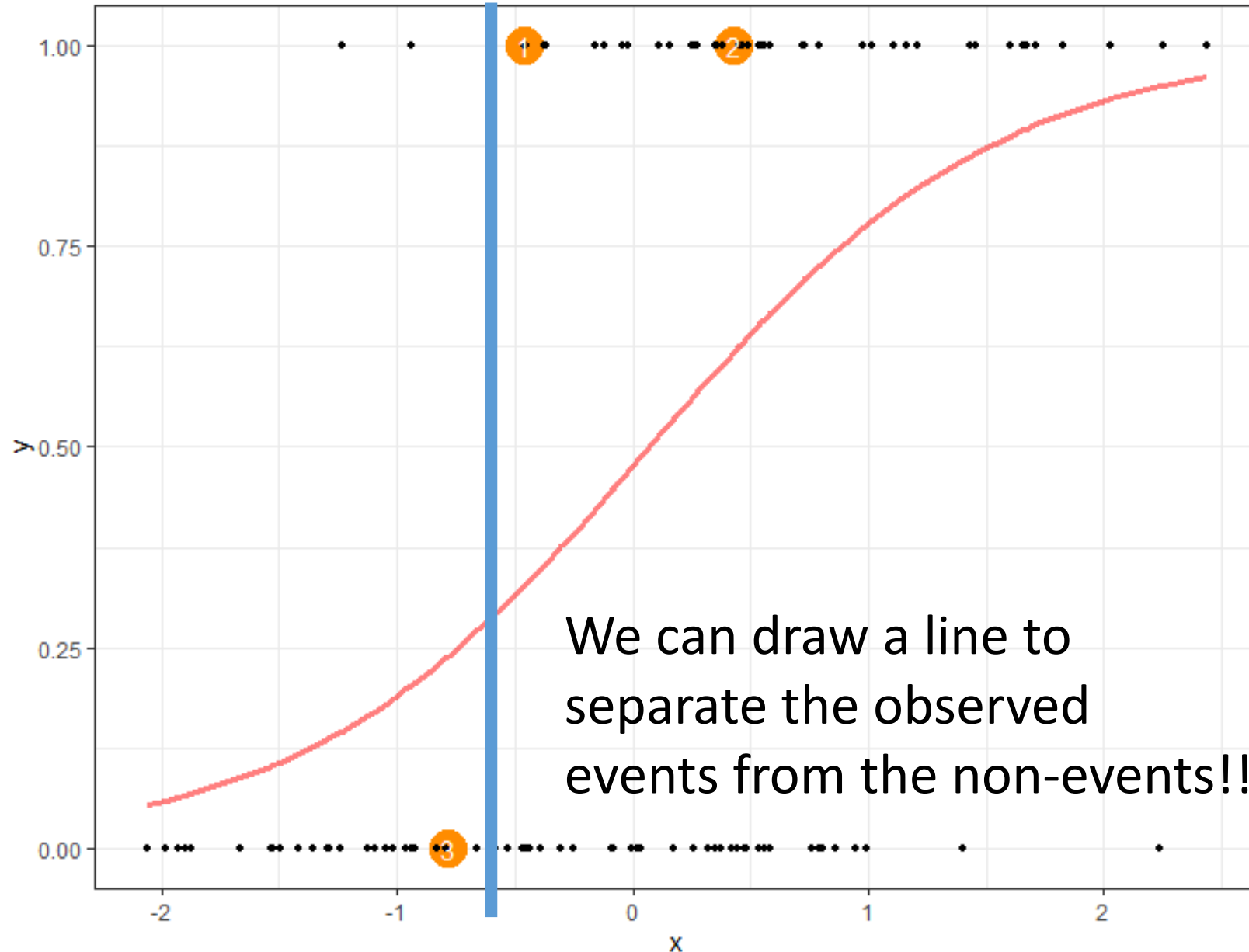




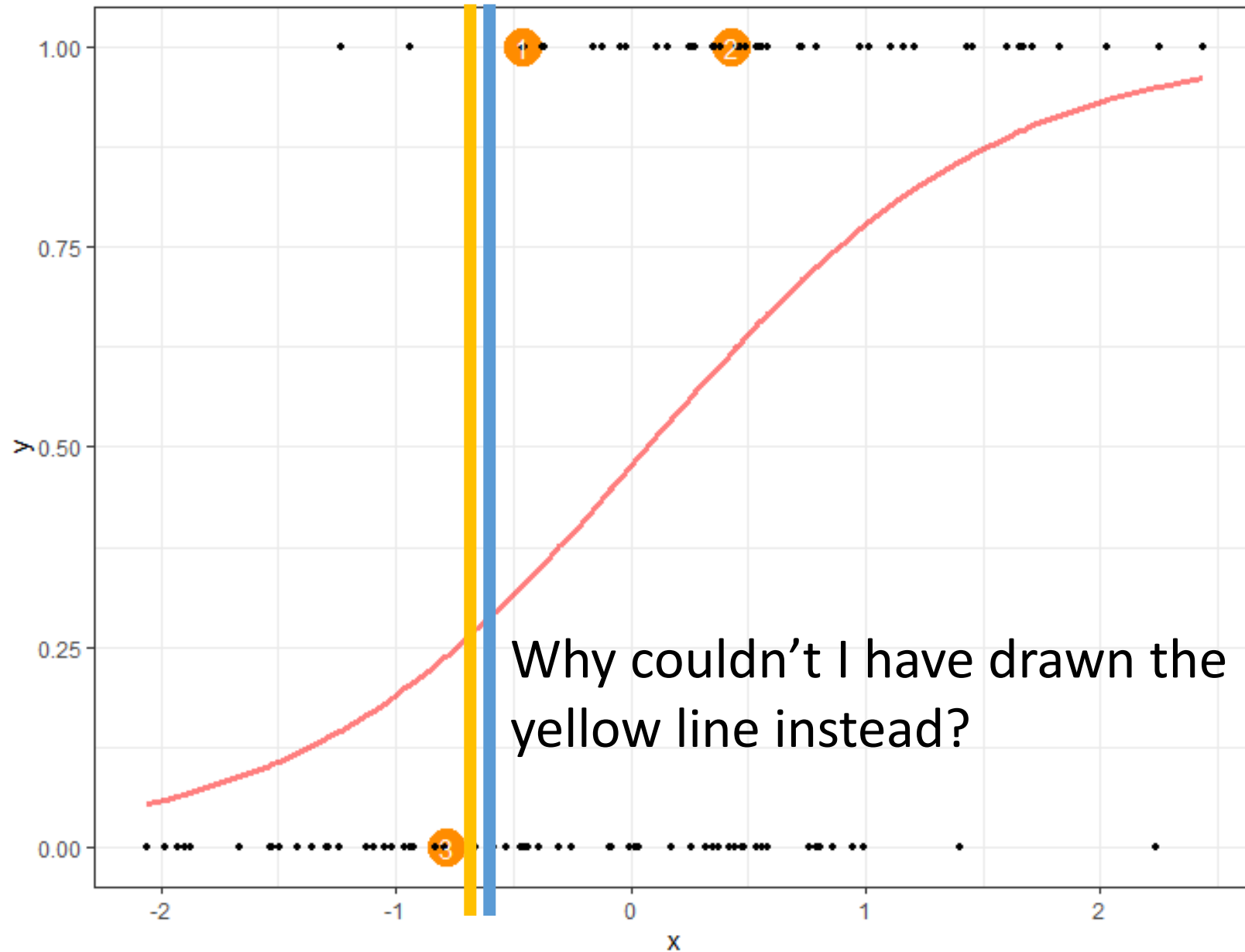




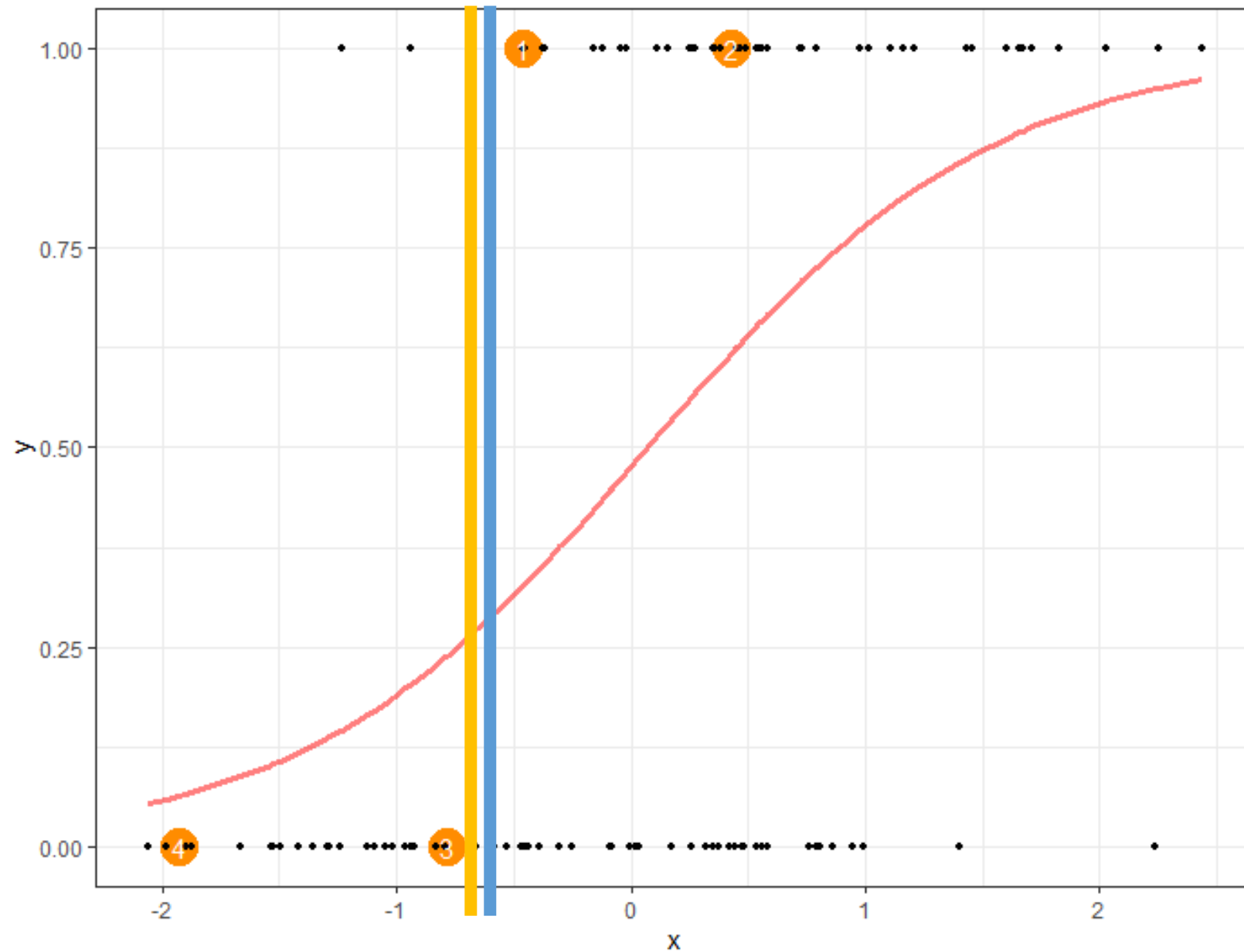
Let's try and see what this "definite boundary" looks like. Consider the first 3 observations, labeled below.



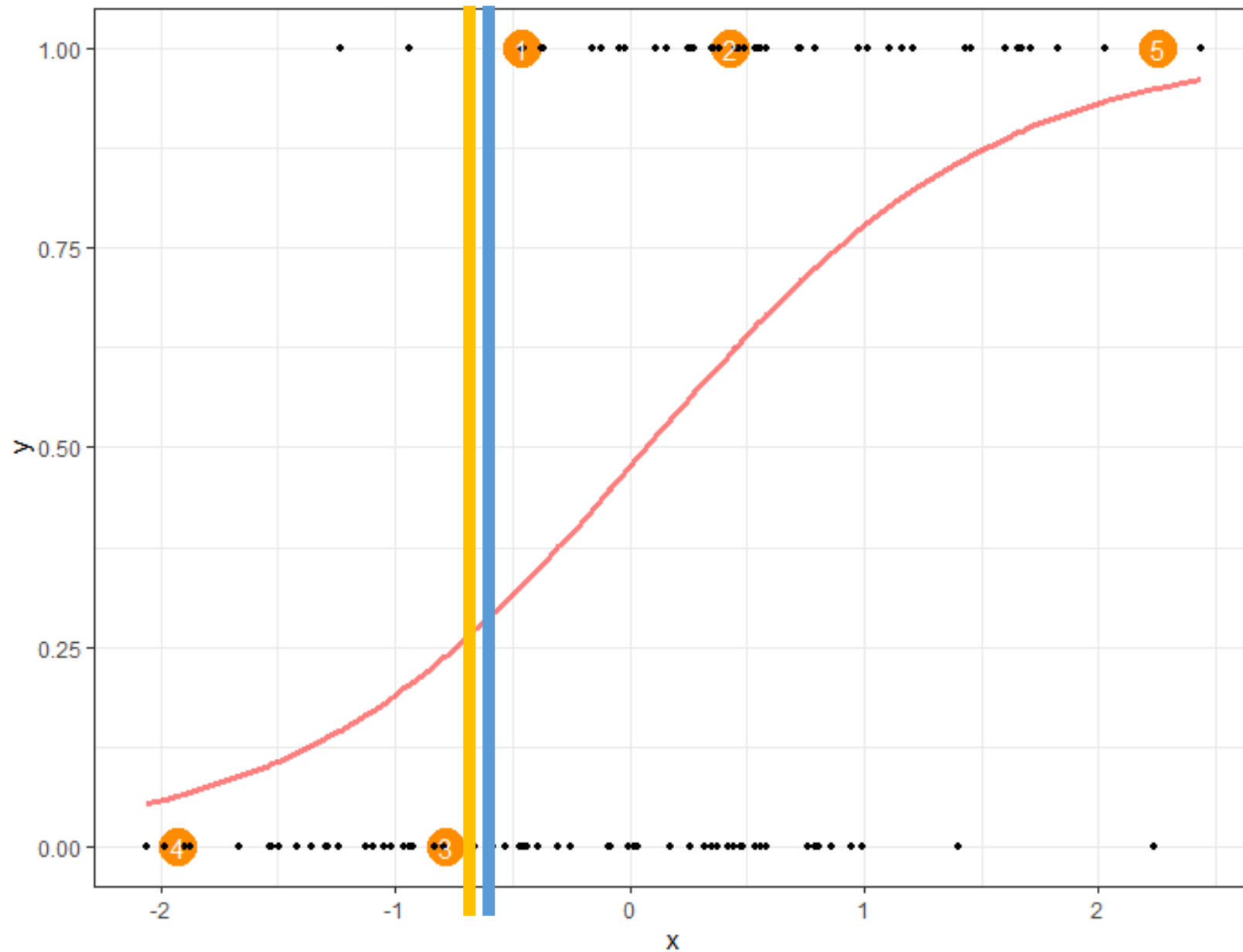
Let's try and see what this “definite boundary” looks like. Consider the first 3 observations, labeled below.



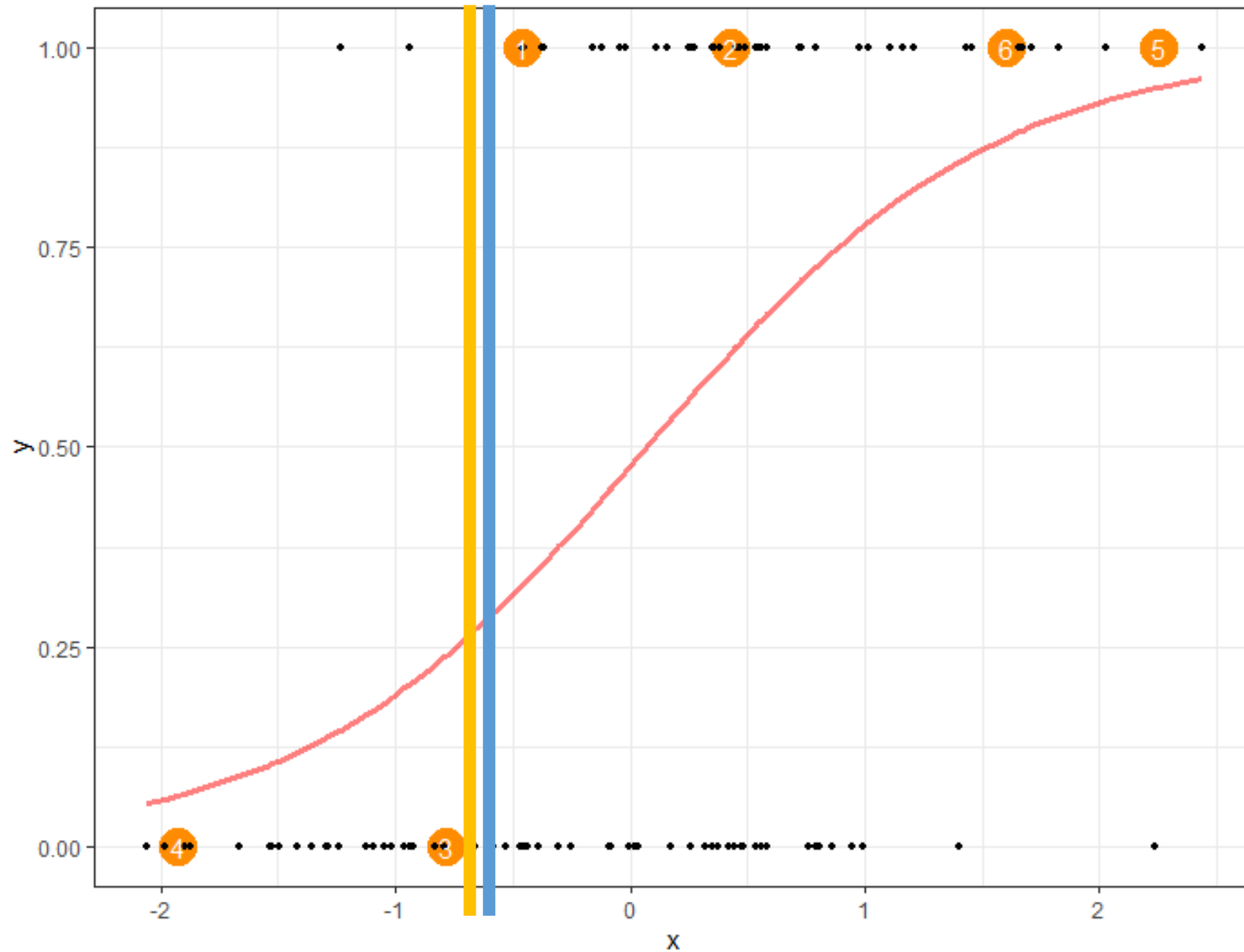
Now add in the 4th observation...the two classes are still separated by either line!



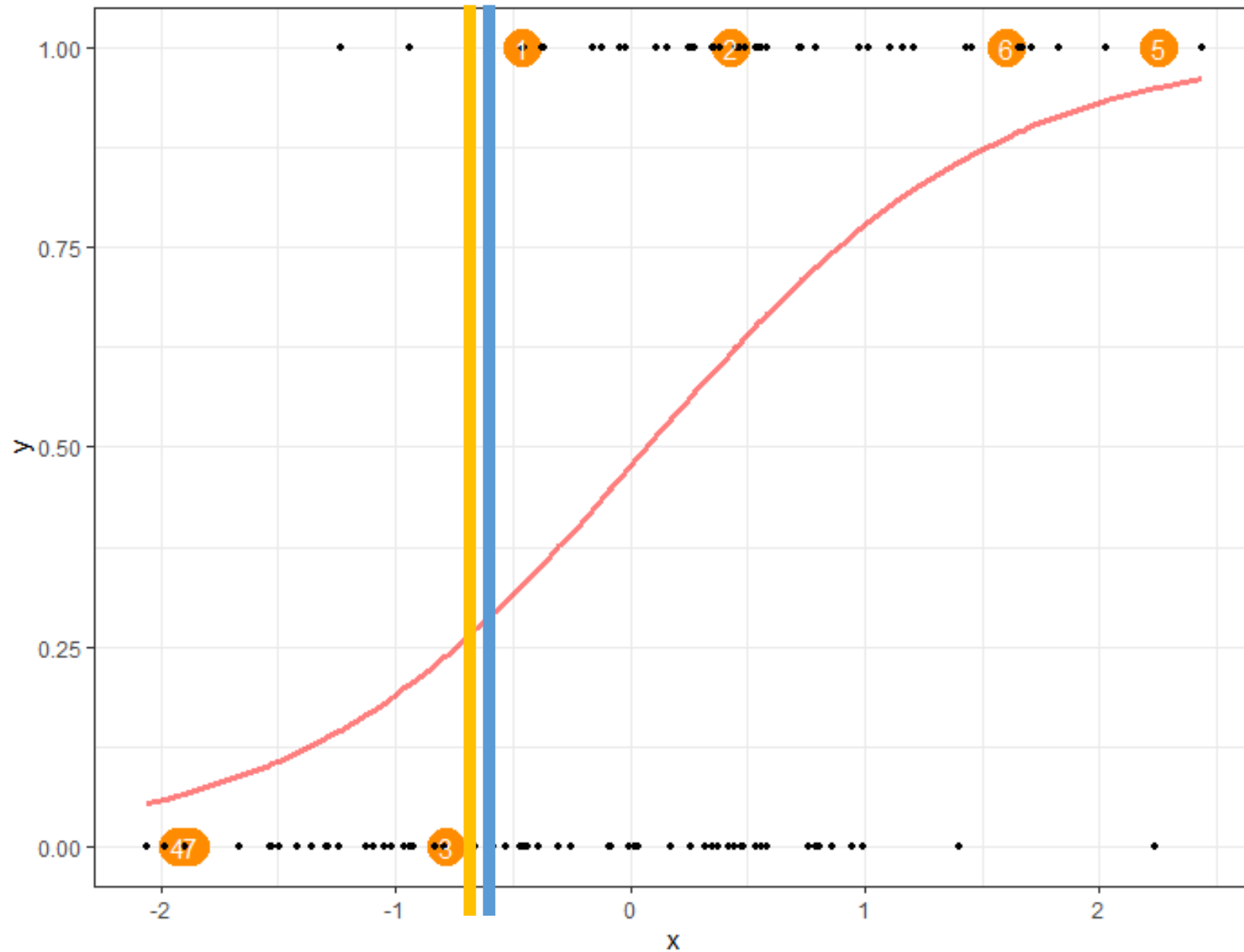
Sequentially add each observation.



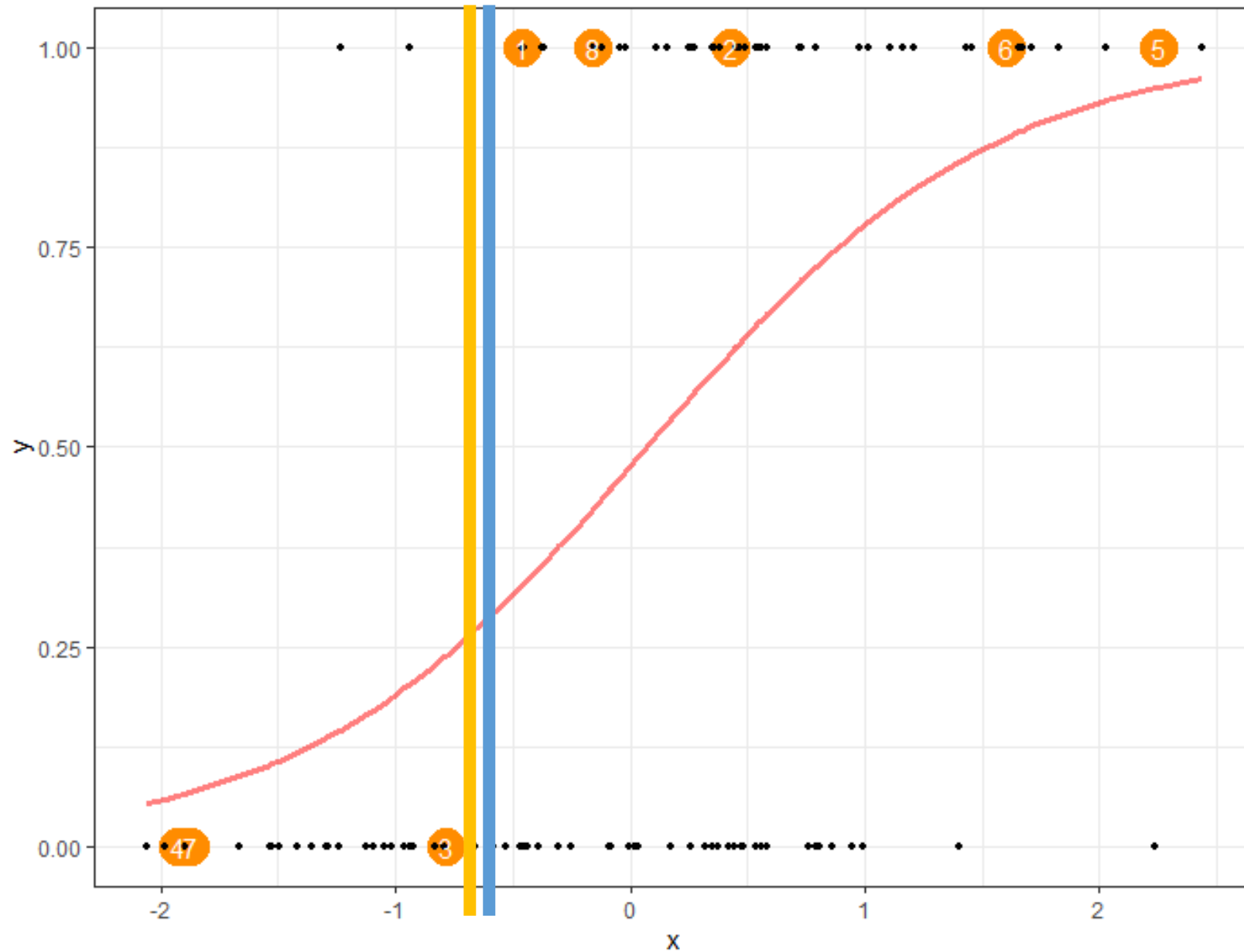
Sequentially add each observation.



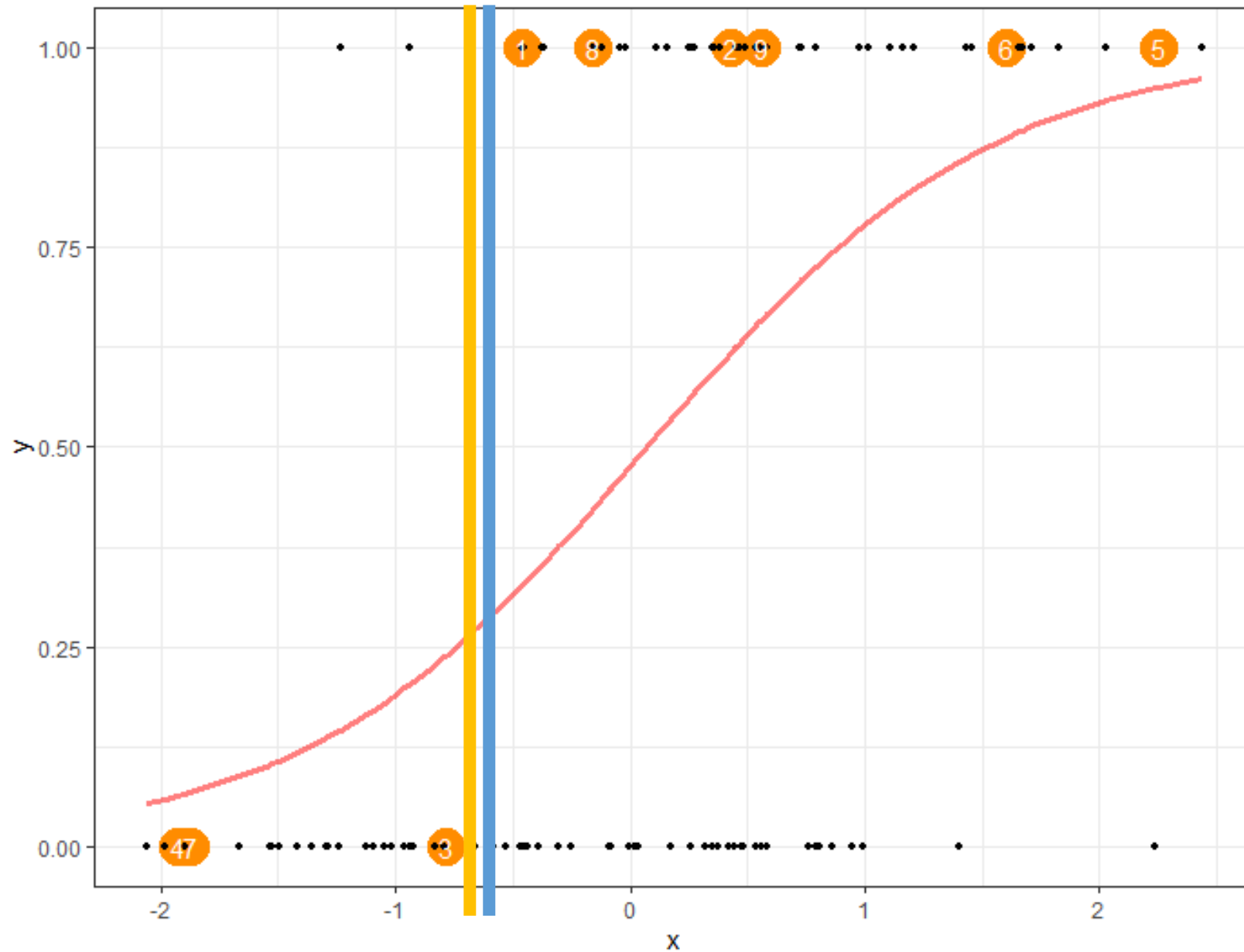
Sequentially add each observation.



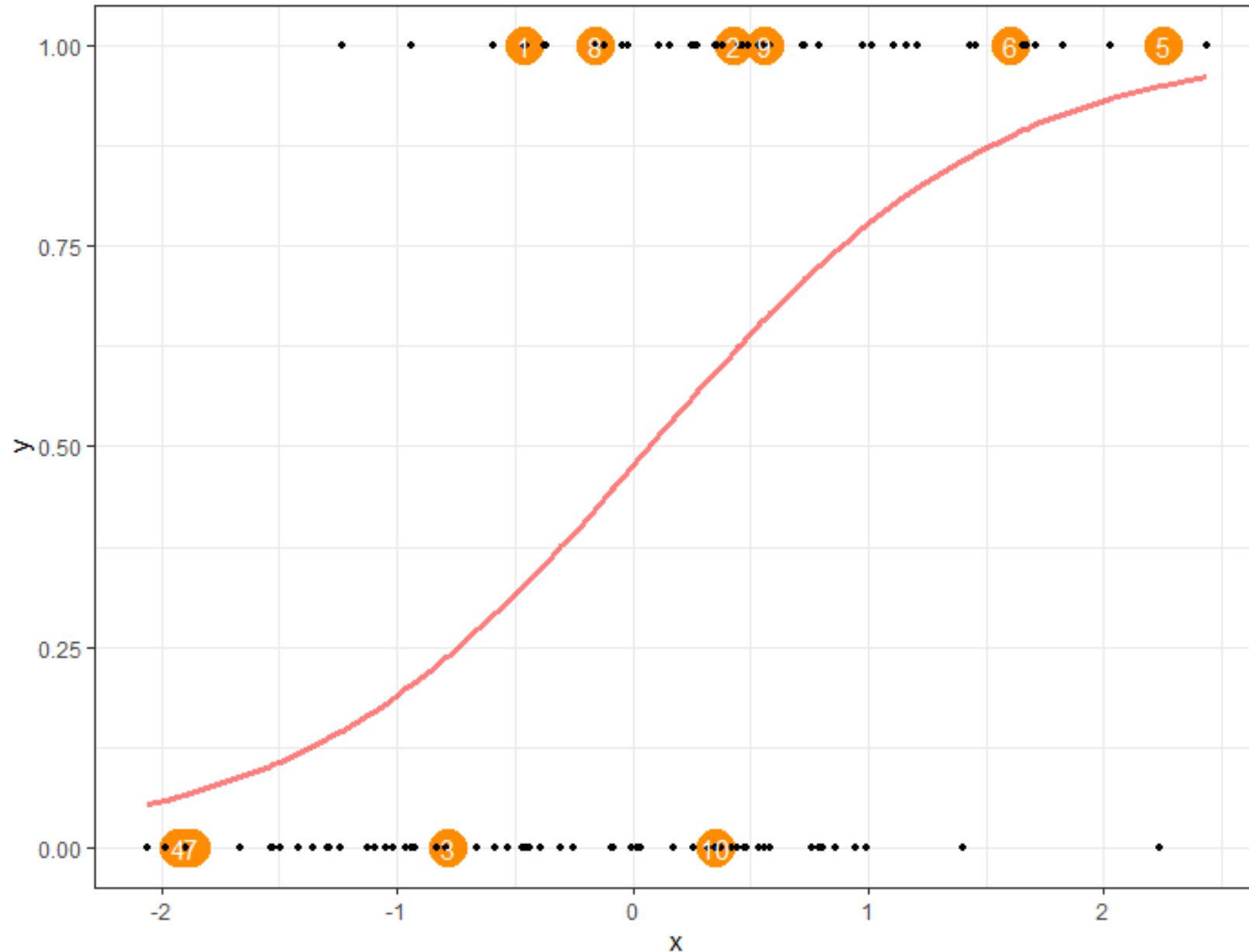
Sequentially add each observation.



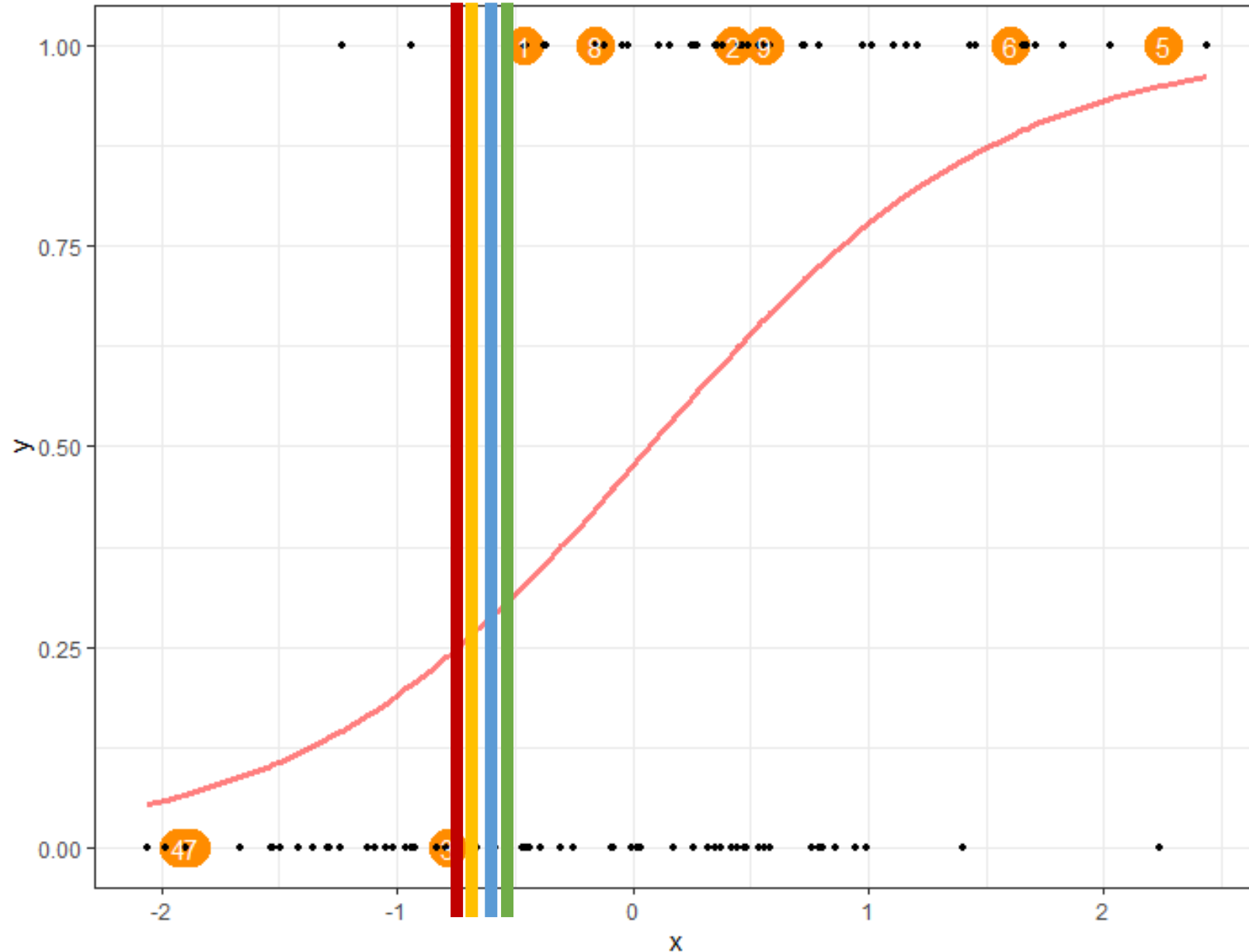
Sequentially add each observation.



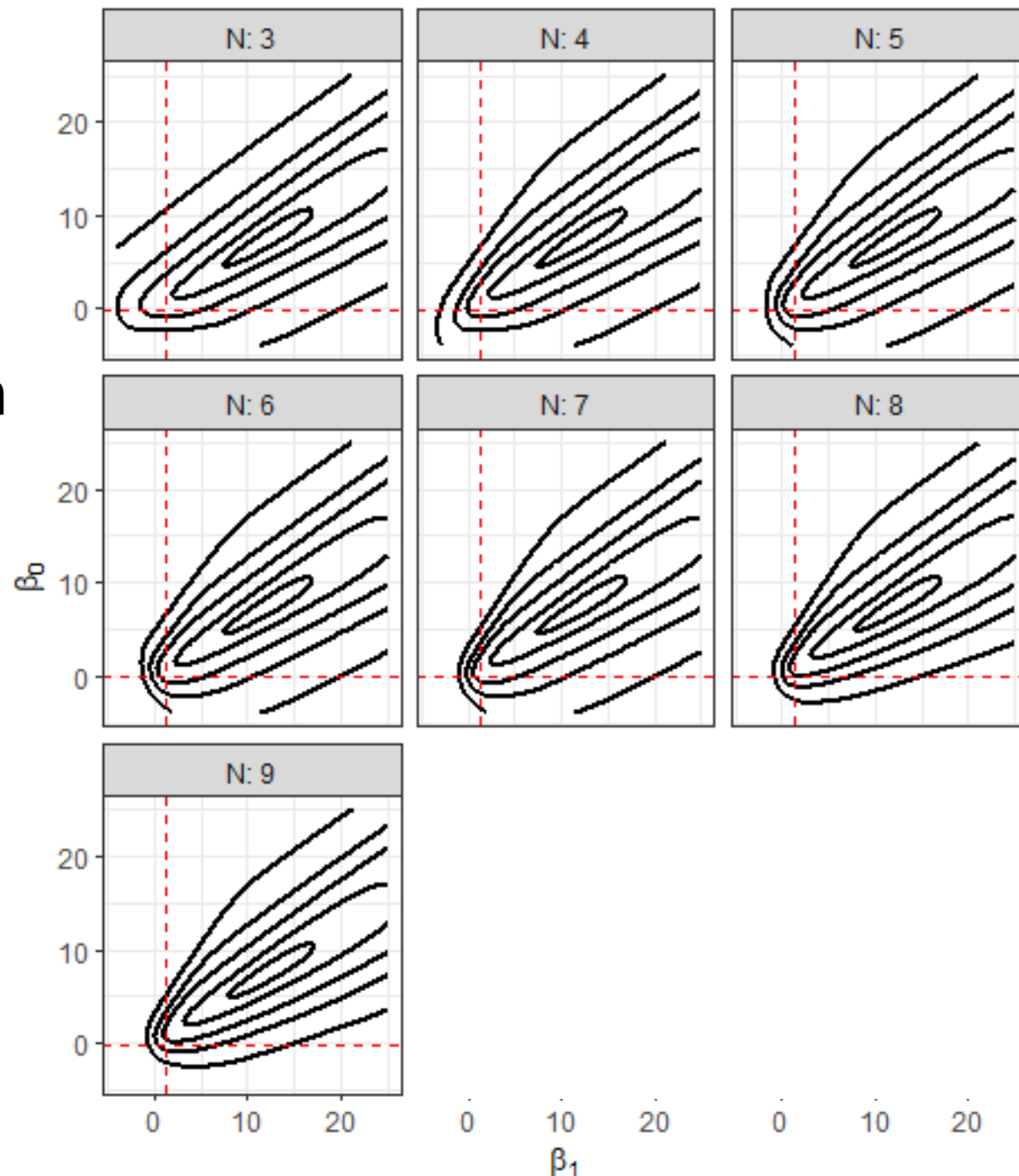
Finally...after observing 10 trials we can no longer “perfectly” divide or **LINEARLY**
SEPARATE the classes!!!



When we can **LINEARLY SEPARATE** the classes (events from non-events), there are **infinitely many lines that could be drawn** → the UNCERTAINTY is enormous!!!

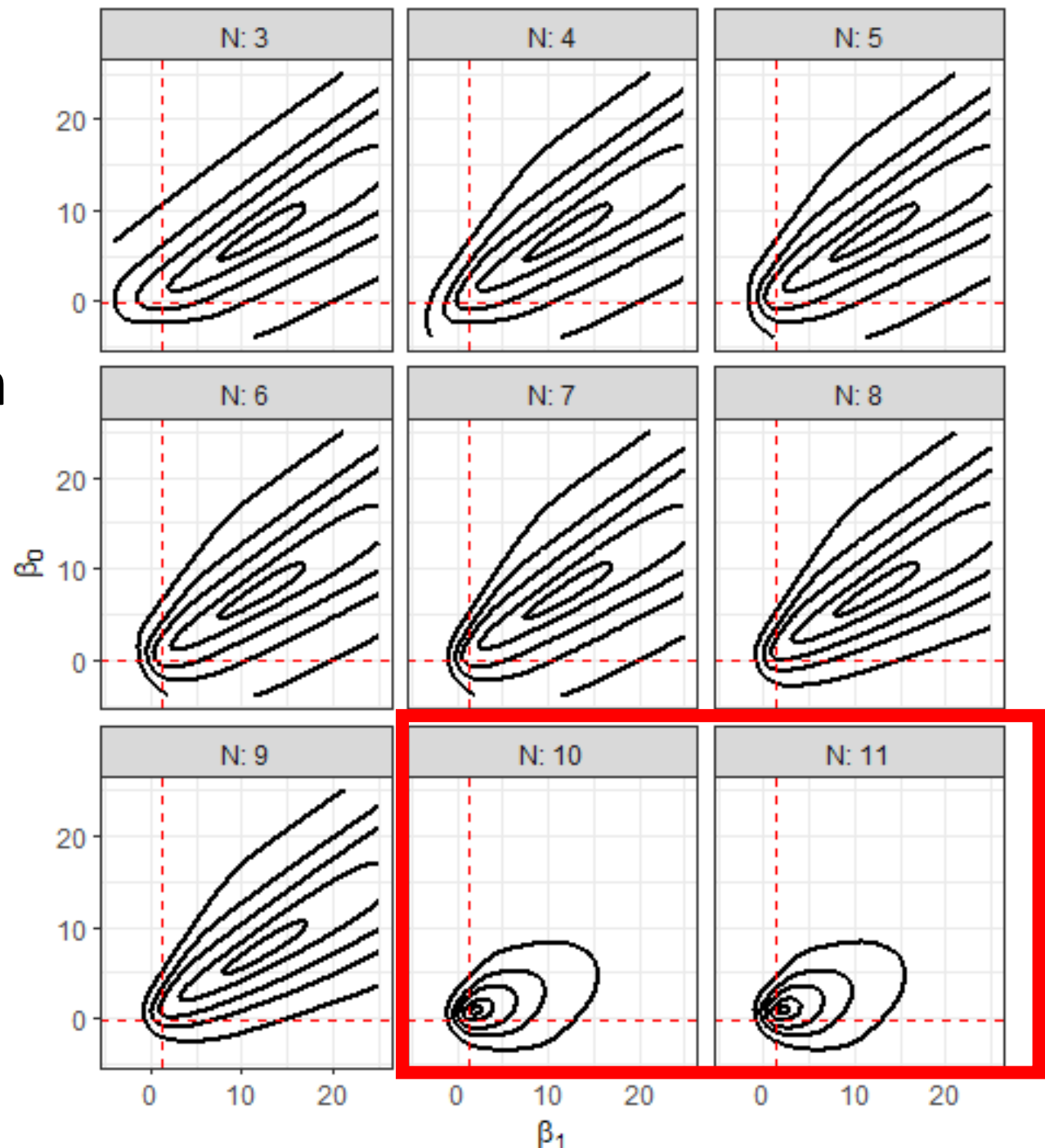


Being able to draw many linear separating lines is why the parameter uncertainty is so high when we use a very weak prior!



Being able to draw many linear separating lines is why the parameter uncertainty is so high when we use a very weak prior!

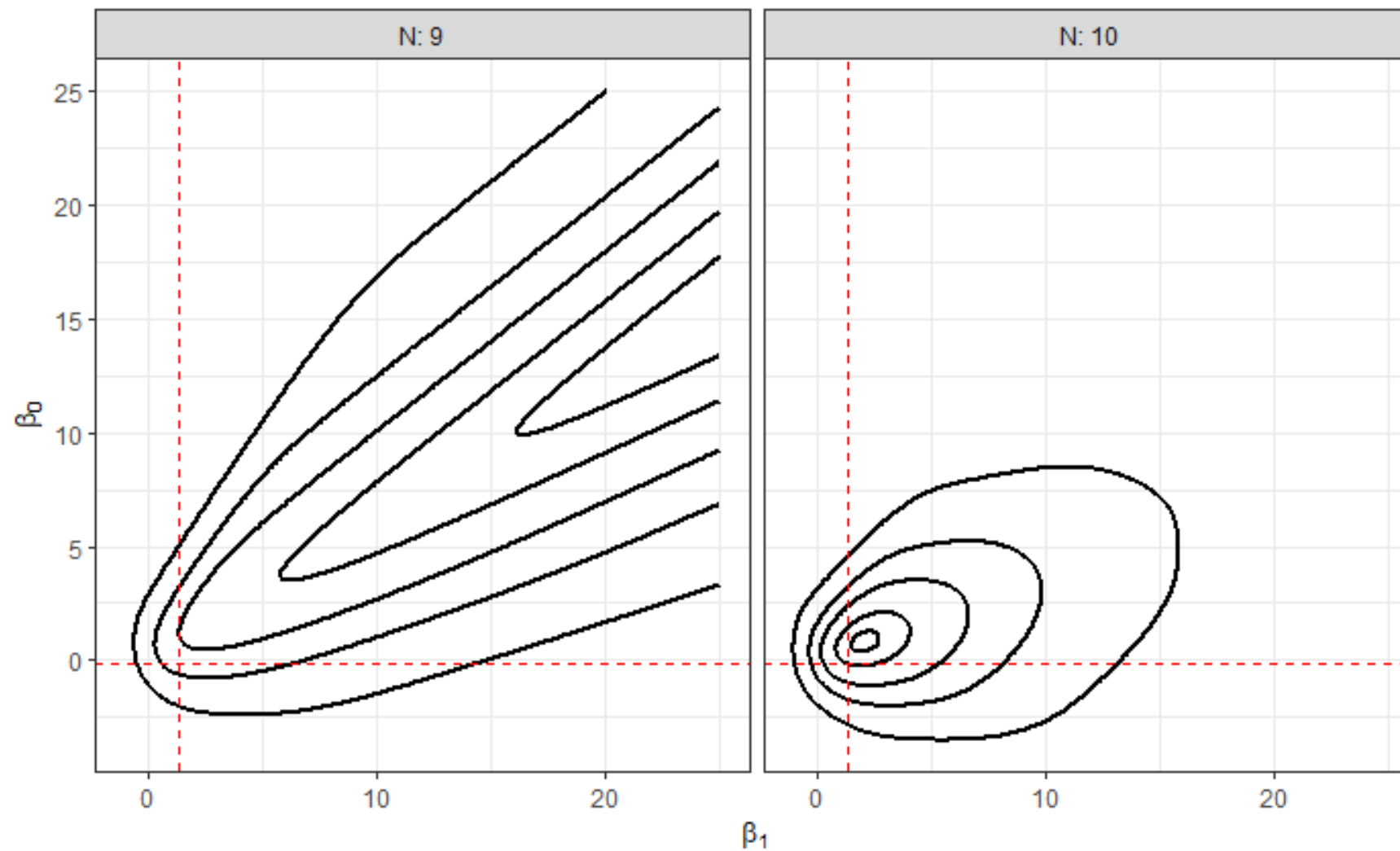
Notice that once we reach 10 observations, which breaks the separation the precision increases substantially.



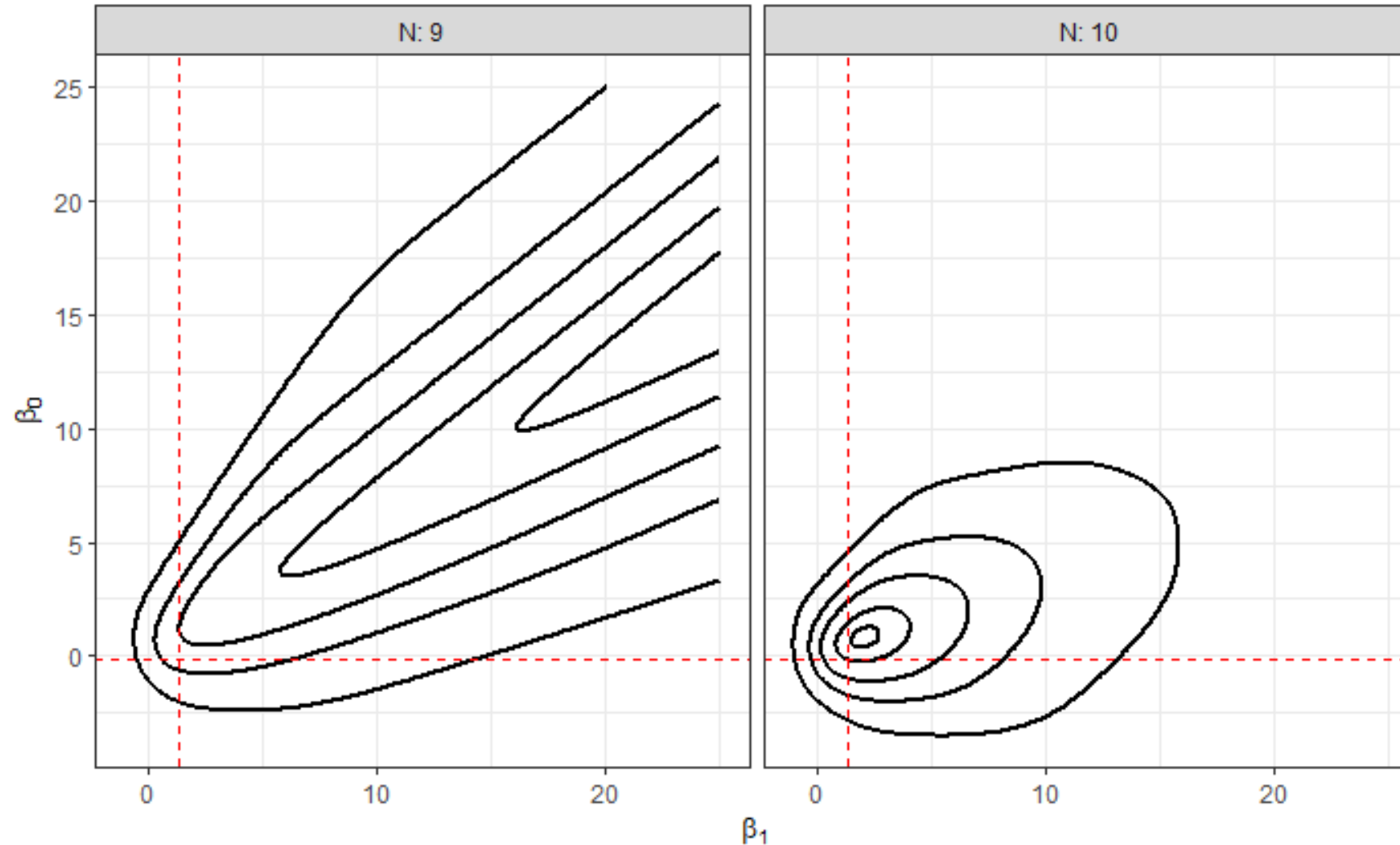
Logistic regression struggles with linearly separated classes

- We used a very weak prior with a prior standard deviation of 20.
- The posterior is very uncertain until the classes cannot be linearly separated.
- If we had tried to fit the model in a non-Bayesian setting by just finding the Maximum Likelihood Estimate the problem is even worse!

The log-likelihood surface over -4 to +25 in both parameters.

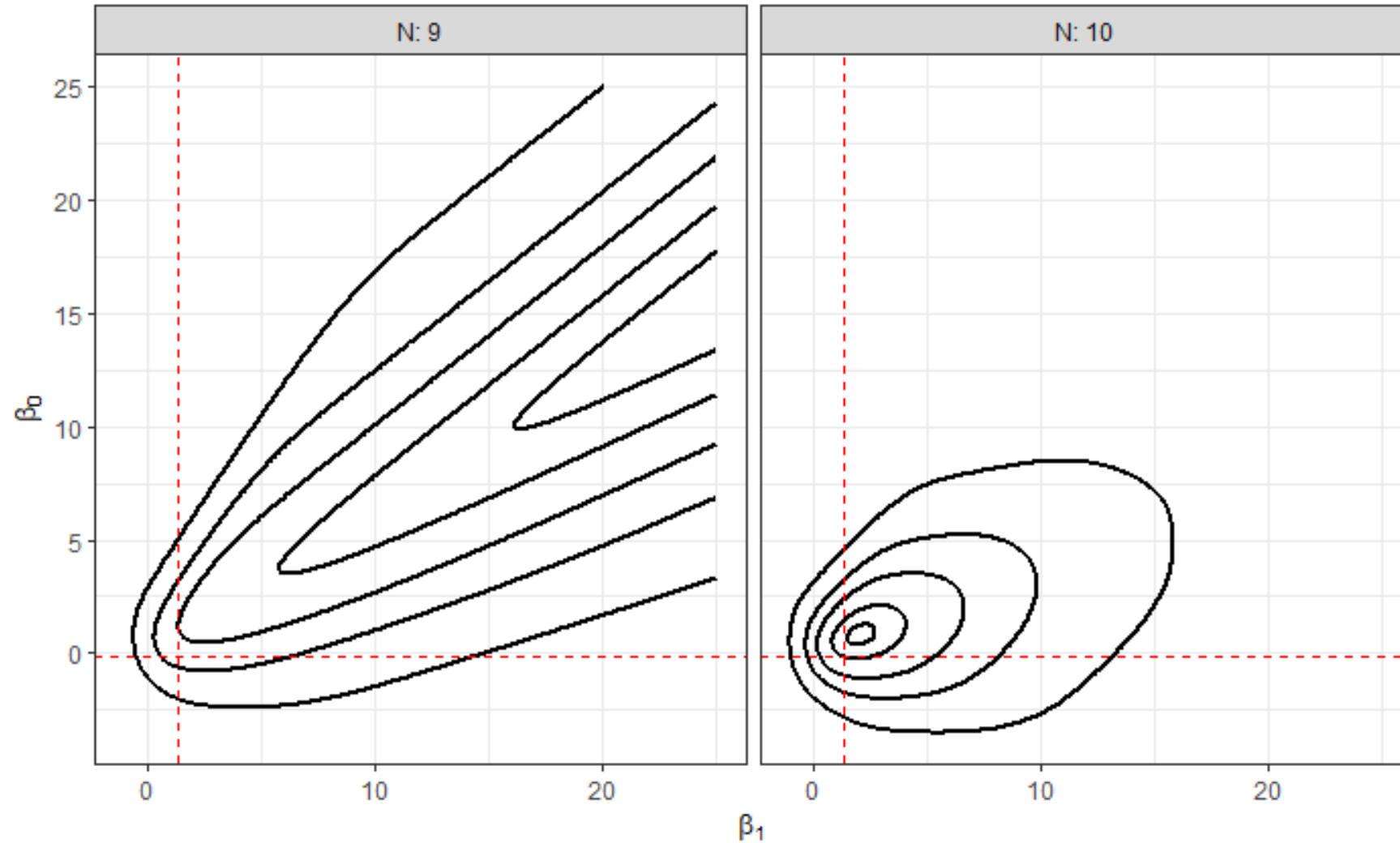


The log-likelihood surface over -4 to +25 in both parameters.



When we can linearly separate the classes, we still cannot “see” the likelihood mode.
Slopes and intercepts greater than 20 are driving the probability curve to a right-angle step change! 111

The log-likelihood surface over -4 to +25 in both parameters.



The problem goes away, when we cannot linearly separate the classes!

Non-Bayesian logistic regression struggles with linearly separated classes

- `glm()` in R fits logistic regression by setting `family = "binomial"` and uses Maximum Likelihood Estimation.
- `statsmodels.formula.api.glm()` in Python fits logistic regression with Maximum Likelihood Estimation.
- `sklearn.linear_model.LogisticRegression` in Python does not appear to suffer from linear separation issues...but that's because by default it does **NOT** use classic logistic regression!!!! It does **NOT** find the MLEs!!!!
 - By default `sklearn` uses the non-Bayesian RIDGE PENALTY to prevent extreme coefficient values!

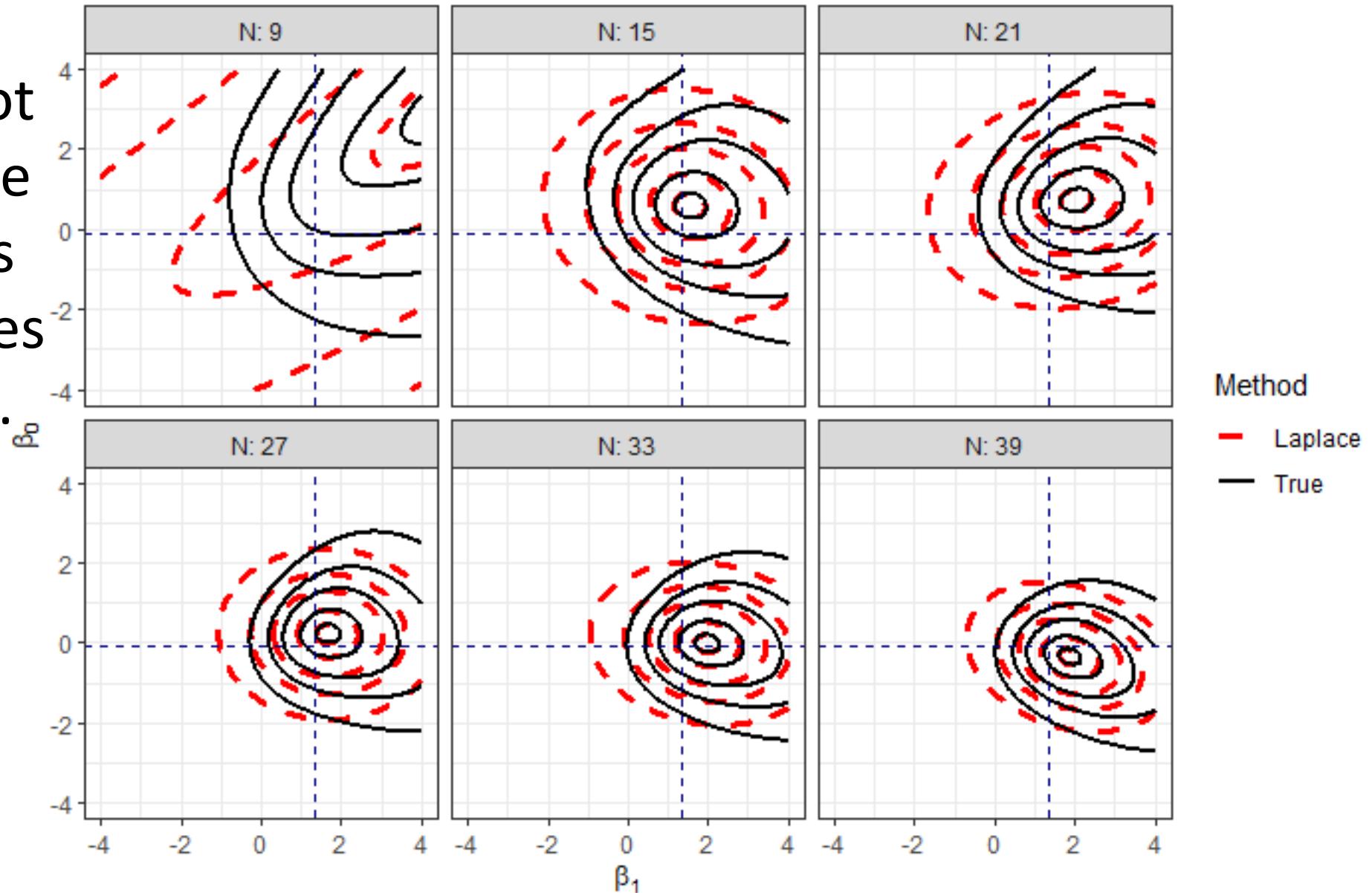
We saw that the log-posterior surface is UNIMODAL

- We also saw that as the sample size gets larger, the surface starts to look more circular...
- Can we apply the Laplace Approximation to logistic regression?

We saw that the log-posterior surface is UNIMODAL

- We also saw that as the sample size gets larger, the surface starts to look more circular...
- Can we apply the Laplace Approximation to logistic regression?
- Let's compare the MVN approximate posterior to the true posterior based on the diffuse prior.

The Laplace approximation is not perfect, but because the true posterior is unimodal, it matches the mode very well.



As the sample size increases, the approximation gets better and better!

Only the outer most tails are missed.

